



UNIVERSITE DE FIANARANTSOA

ECOLE NATIONALE D'INFORMATIQUE

MEMOIRE DE FIN D'ETUDES EN VUE D'OBTENTION DU DIPLOME DE
MASTER DE RECHERCHE

Mention : Intelligence Artificielle

Parcours : Gouvernance et Ingénierie des Données

Intitulée :

NATURAL LANGUAGE PROCESSING SUPPORT

Présenté par :

RAKOTOSON Hanitriniala Nicole

Encadreurs :

Dr DIMBIARISOA William Germain

Dr Stephane Ducasse

Année universitaire : 2024–2025

Remerciements

Je tiens à exprimer ma profonde gratitude à mes encadreurs pour leur accompagnement et leurs précieux conseils.

Table des figures

1.1	Algorithme de Viterbi	6
1.2	Structure de la RNN simple	13
1.3	La figure à gauche est une représentation compacte de la RNN simple et celle à droite est sa représentation déroulée dans le temps	14
5.1	Représentation du corpus "FRED TED" sous forme de liste doublement chaînée avec visualisation des paires et de leurs occurrences dans le processus de Byte Pair Encoding.	66
5.2	État de la liste doublement chaînée après fusion de la paire (d, _)	67
5.3	État de la liste doublement chaînée après fusion de la paire (e, d_)	69
5.4	Benchmark des temps d'entraînement des implémentations du BPE sur les noms de classes de Pharo en fonction de la taille du vocabulaire n	101
5.5	Benchmark des temps d'entraînement des implémentations du BPE sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n	102
6.1	Representation du corpus « fred fed ted bread and ted fed fred bread » en utilisant une liste doublement chainee	106
6.2	Représentation du corpus 'fred fed ted bread, and ted fed fred bread' en utilisant une liste doublement chaînée	107
6.3	État de la liste doublement chaînée après fusion de la paire (a, ##n)	108
6.4	État de la liste doublement chaînée après fusion de la paire (b, ##r)	108
6.5	Trie construit à partir du vocabulaire. La correspondance de la chaîne "abcdz" débute au nœud 0.	134
6.6	Trie construit à partir du vocabulaire. La correspondance de la chaîne "bcdz" débute au nœud 2.	134
6.7	Trie construit à partir du vocabulaire. La correspondance de la chaîne "cdz" débute au nœud 2.	135
6.8	Trie construit à partir du vocabulaire. La correspondance de la chaîne "dz" débute au nœud 2.	135
6.9	Trie construit à partir du vocabulaire $V = \{a, ##b, ##c, ##cdy, ##dz, abcdx\}$, avec les failure links et les failure pops associés à chaque nœud.	137
6.10	Benchmark des temps d'entraînement des implémentations du WordPiece sur les noms de classes de Pharo en fonction de la taille du vocabulaire n	157

6.11	Benchmark des temps d'entraînement des implémentations du WordPiece sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n	157
6.12	Heatmap des temps d'encodage de <code>WordPieceEncoder</code>	159
6.13	Heatmap des temps d'encodage de <code>WordPiece</code>	160
6.14	Facteur d'accélération de <code>WordPieceEncoder</code> par rapport à <code>WordPiece</code> en fonc- tion de la taille du corpus et de la taille du vocabulaire.	161
1.1		

Table des matières

Remerciements	i
1 Traitement Automatique de Langage Naturel	3
1.1 Importances de la NLP	3
1.1.1 Vie quotidienne	3
1.1.2 Santé	3
1.1.3 Entreprises	4
1.1.4 Education	4
1.2 Les algorithmes essentiels en TALN	4
1.2.1 Algorithmes de pretraitement	4
1.2.2 Frequence des mots	6
1.2.3 TF-IDF	6
1.2.4 Skip-gram	7
1.2.5 Régression logistique	8
1.2.6 Réseaux de neurones récurrents	12
2 Byte Pair Encoding	18
2.1 Présentation	18
2.1.1 Logique principale	18
2.1.2 Illustration à travers un exemple	19
2.1.3 Observation sur l'algorithme	24
2.2 Implémentation	25
2.2.1 Algorithme pseudo-code	25
2.2.2 Implémentation en Pharo	32
3 WordPiece	39
3.1 Présentation de WordPiece	39
3.1.1 Comprendre WordPiece	39
3.1.2 Logique de fonctionnement de WordPiece	39
3.1.3 Illustration à travers un exemple	40
3.2 Implémentation	48
3.2.1 Algorithmes pseudo-code	48
3.2.2 Implémentation en Pharo	54

4	Unigram	63
5	Optimisation de BPE	64
5.1	Idée de l'optimisation	64
5.1.1	Exemple	65
5.1.2	Fonctionnement de l'optimisation du BPE	69
5.2	Réalisation	74
5.2.1	Création des classes	74
5.2.2	Structures de données utilisées	77
5.2.3	Implémentation du prétraitement du texte	78
5.2.4	Initialisation des variables d'instance de BPETrain	80
5.2.5	Construction de la représentation du corpus dans BPETrain	81
5.2.6	Construction de vocabulary dans BPETrain	83
5.2.7	Construction des dictionnaires pairFrequencies et pairPositions dans BPE- Train	83
5.2.8	Réalisation d'une étape de fusion	86
5.2.9	Application des fusions apprises avec BPEEncoder	92
5.3	Optimisation de BPE par fusion de paires en lots	95
5.3.1	Idée sur l'optimisation de BPE par fusion de paires en lots	95
5.3.2	Fonctionnement	97
5.3.3	Implémentation	99
5.4	Benchmarking	100
5.4.1	Entraînement	100
5.4.2	Encodage	102
6	Optimisation de WordPiece	104
6.1	Optimisation de l'entraînement de WordPiece	104
6.1.1	Problème	104
6.1.2	Solution	105
6.1.3	Fonctionnement	108
6.1.4	Réalisation	115
6.2	Optimisation de l'encodage de WordPiece	133
6.2.1	Problème	133
6.2.2	Solution	136
6.2.3	Fonctionnement	142
6.2.4	Réalisation	144
6.3	Benchmarking	156
6.3.1	Entraînement	156
6.3.2	Encodage	158
6.4	Comparaison théorique	161

7	Implémentation	162
7.1	Implémentation de BPE	162
7.2	Implémentation de WordPiece	162
7.3	Implémentation de Unigram	162
8	Résultats et Analyse	163
8.1	Expérimentations	163
8.2	Comparaison des performances	163
9	Conclusion et Perspectives	164
A	Annexes	165

Introduction

Les machines ont toujours surpassé les humains dans les tâches de calcul numérique, mais elles ont longtemps été incapables d'égaliser nos capacités à comprendre ou à générer du texte. Aujourd'hui, grâce au TALN, les ordinateurs peuvent comprendre et générer du texte de manière automatisée.

Cependant les machines ne traitent pas directement le texte brut : celui-ci doit passer par plusieurs étapes de traitement avant d'être exploité par les modèles, notamment le prétraitement, l'extraction de caractéristiques, l'analyse et l'entraînement. La tokenisation, première étape du prétraitement, consiste à segmenter le texte en caractères, mots ou sous-mots. Bien que souvent négligée, elle influence directement la vitesse et la qualité de prédiction des modèles.

Parmi les méthodes les plus utilisées figurent BPE, WordPiece et Unigram. Ces algorithmes permettent de segmenter les mots en sous-unités, gérer les mots rares tout en conservant des séquences de longueur raisonnable et un vocabulaire de taille maîtrisée. Malgré leur disponibilité dans des bibliothèques comme Hugging Face Transformers et SentencePiece, ces algorithmes ne sont pas encore disponible dans Pharo.

Dans ce contexte, ce mémoire propose une implémentation naïve de ces trois algorithmes dans Pharo, suivie d'une optimisation et d'une étude comparative. L'objectif est de concevoir et d'implémenter les algorithmes BPE, WordPiece et Unigram, de réaliser un benchmarking de leurs performances en termes de vitesse et d'analyser la qualité des tokens générés. Cette démarche permet non seulement de combler un manque technique dans Pharo, mais aussi d'évaluer les différences entre ces méthodes.

Ce mémoire est organisé comme suit.

Le premier chapitre présente le contexte général du TALN et l'importance de la tokenisation.

Le deuxième chapitre introduit l'algorithme BPE ainsi que son implémentation naïve.

Le troisième chapitre présente l'algorithme WordPiece et son implémentation naïve.

Le quatrième chapitre est consacré à l'algorithme Unigram et à son implémentation naïve.

Le cinquième chapitre décrit l'optimisation de BPE, en présentant l'idée, la réalisation et les résultats du benchmarking.

Le sixième chapitre traite de l'optimisation de WordPiece, incluant la stratégie adoptée, l'implémentation et l'évaluation expérimentale.

Le septième chapitre présente l'optimisation de Unigram ainsi que son évaluation.

Enfin, le huitième chapitre propose une analyse comparative des tokens produits par les trois algorithmes sur les noms de méthodes en Pharo.

Chapitre 1

Traitement Automatique de Langage Naturel

1.1 Importances de la NLP

Les données textuelles sont générées quotidiennement par de multiples sources, telles que les réseaux sociaux, les blogs, les forums, les articles en ligne, les communications professionnelles, les assistants virtuels, les oeuvres scientifiques et culturelles, ou encore les avis consommateurs. L'ensemble de ces générateurs produits un volume massif de textes difficile à exploiter manuellement, rendant le TALN indispensable pour transformer ces textes en informations utiles, exploitables et organisées.

1.1.1 Vie quotidienne

Le NLP facilite l'interaction fluide entre l'homme et la machine. Grâce à lui, la barrière des langues peut être dépassée : les traducteurs automatiques comme Google Traduction permettent de comprendre et de communiquer dans différentes langues. Le NLP améliore également la recherche sur Internet, offrant résultats précis et adaptés. Les chatbots, applications du NLP, fournissent des réponses instantanées aux questions des utilisateurs à tout moment sans interruption. Enfin, le NLP permet la correction automatique de texte, le résumé de documents et la génération de contenus.

1.1.2 Santé

Dans le secteur de la santé, le NLP analyse les symptômes des patients, leurs antécédants médicaux et les rapports de laboratoire afin d'aider au diagnostic. Il permet de lire automatiquement des milliers de dossiers médicaux, d'extraire des informations importantes et d'identifier des tendances pour assister les professionnels de santé. L'identification des effets indésirables aux médicaments à partir de données textuelles est une tâche complexe ; les modèles NLP peuvent scanner les dossiers des patients, les réseaux sociaux et la littérature médicale pour détecter d'éventuels effets secondaires ou interactions médicamenteuses.[1]

1.1.3 Entreprises

De plus en plus d'entreprises utilisent le NLP pour automatiser certaines tâches, analyser les données et améliorer la gestion de relation client. Les systèmes NLP permettent de classifier automatiquement de grands volumes de documents. Ils sont également utilisés pour prédire les tendances boursières et les comportement des consommateurs en analysant les données issues des médias sociaux et de la presse, améliorant ainsi la satisfaction et la fidélisation des clients et orientant les stratégies marketing.

Le TALN enrichit l'informatique décisionnelle en exploitant les données non structurées, telles que les retours clients, les études de marché ou les réseaux sociaux. L'exploration de texte et les technologies du web sémantique sont utilisées pour améliorer l'analyse prédictive et la découverte de connaissances.[2]

1.1.4 Education

Dans le domaine de l'éducation, le NLP personnalise les parcours d'apprentissage en analysant les réponses des étudiants, en identifiant leurs forces et leurs faiblesses, et en proposant des contenus adaptés à leurs besoins individuels. Il aide à améliorer la prononciation, la traduction et la compréhension des langues, et peut répondre aux questions des étudiants à tout moment via des chatbots. [3]

1.2 Les algorithmes essentiels en TALN

1.2.1 Algorithmes de pretraitement

1.2.1.1 Tokenisation

La tokenisation consiste à segmenter un texte brut en unités appelées tokens. Ces unités peuvent correspondre à des mots, des expressions, des phrases, des sous-mots ou même des caractères, selon la méthode de tokenisation adoptée. Il y a deux grandes classes d'algorithmes de tokenisation : Tokenisation descendante : cette approche repose sur la définition préalable de règles linguistiques ou de normes de segmentation. Le texte est ensuite découpé en appliquant ces règles, qui peuvent être basées sur des espaces, des signes de ponctuation.

exemple : la phrase : "Le chat dort." devient : ["Le", "chat", "dort", "."].

Tokenisation ascendante : cette approche s'appuie sur des méthodes statistiques appliquées aux séquences de caractères afin de construire un vocabulaire de sous-mots. Le texte est ensuite segmenté en fonction de ces unités. Les algorithmes tels que Byte Pair Encoding ou Word-Piece ou Unigram illustrent ce type d'approche. Nous allons détailler ces algorithmes dans les chapitres suivantes.

exemple : la phrase : "Le chat dort." devient : ["Le", "ch", "at", "dor", "t", "."]

1.2.1.2 Hidden Markov Model pour Part-Of-Speech Tagging

Le HMM est un modele probabiliste de sequence. Etant donnee une sequence d'unites(mots, lettres, morphemes, etc), il calcule la probabilites de toutes les sequences possibles d'etiquettes et choisit la meilleure sequence d'etiquettes.

Le HMM est definit par cinq composants :

- Les etats Q : un ensemble d'etats $q_1, \dots, q_n$. Chaque etat correspond a une etiquette grammaticale.
- la matrice A contient les probabilites de passer d'un etat a un autre. Chaque valeur a_{ij} indique la probabilite de passer de l'etat i a l'etat j . La somme des probabilites de transition depuis un etat donne est egale à 1.
- Les observations O : la séquence de mots ou unités observables dans le texte. Chaque observation provient d'un vocabulaire donne.
- Les probabilites d'emissions B : elles indiquent la probabilité qu'un état génère une observation particulière.
- La distribution initiale π : elle donne la probabilité que la chaîne de Markov commence dans chaque état. Certains états peuvent avoir une probabilité initiale nulle s'ils ne peuvent pas commencer une séquence. La somme de toutes les probabilités initiales est égale à 1.

Pour pouvoir obtenir, la sequence d'etiquette correspondante a une sequence de mots nous allons proceder comme suit :

- Calculer les composantes A et B de HMM en comptant sur un corpus annote :
La matrice A contient les probabilites de transition entre etiquette $P(t_i|t_{i-1})$ qui representent la probabilite qu'une etiquette apparaisse etant donne l'etiquette precedente.
 $P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$ ou $C(t_{i-1}, t_i)$ est le nombre de fois ou l'etiquette t_{i-1} est suivi de t_i dans le corpus et $C(t_{i-1})$ est le nombre de fois ou l'etiquette t_{i-1} apparait.
 B contient les probabilites d'emissions $P(w_i|t_i)$ representant la probabilite, etant donne une etiquette, qu'elle soit associee a un mot donne. $P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$ ou $C(t_i, w_i)$ est le nombre de fois ou le mot w_i apparait avec l'etiquette t_i dans le corpus annote et $C(t_i)$ est le nombre de fois ou l'etiquette t_i apparait.
- Apres avoir calculer A et B , nous allons maintenant determiner la sequence la plus probable en utilisant l'algorithme de Viterbi illustre 1.1.

```

function VITERBI(observations of len  $T$ , state-graph of len  $N$ ) returns best-path, path-prob

create a path probability matrix viterbi[ $N, T$ ]
create a backpointer matrix backpointer[ $N, T$ ]
for each state  $s$  from 1 to  $N$  do ; initialization step
     $viterbi[s, 1] \leftarrow \pi_s * b_s(o_1)$ 
     $backpointer[s, 1] \leftarrow 0$ 
for each time step  $t$  from 2 to  $T$  do ; recursion step
    for each state  $s$  from 1 to  $N$  do
         $viterbi[s, t] \leftarrow \max_{s'=1}^N viterbi[s', t-1] * a_{s', s} * b_s(o_t)$ 
         $backpointer[s, t] \leftarrow \operatorname{argmax}_{s'=1}^N viterbi[s', t-1] * a_{s', s} * b_s(o_t)$ 

     $bestpathprob \leftarrow \max_{s=1}^N viterbi[s, T]$  ; termination step
     $bestpathpointer \leftarrow \operatorname{argmax}_{s=1}^N viterbi[s, T]$  ; termination step
    bestpath  $\leftarrow$  the path starting at state bestpathpointer, that follows backpointer[] to states back in time
    return bestpath, bestpathprob

```

Figure 17.10 Viterbi algorithm for finding the optimal sequence of tags. Given an observation sequence and an HMM $\lambda = (A, B)$, the algorithm returns the state path through the HMM that assigns maximum likelihood to the observation sequence.

FIGURE 1.1 – Algorithme de Viterbi

1.2.2 Frequence des mots

Le modele Frequence des mots represente chaque document d'un corpus sous la forme d'un vecteur de dimension egale a la taille du vocabulaire¹. Chaque dimension du vecteur correspond a un mot du vocabulaire, et la valeur associee indique le nombre d'occurences de ce mot dans le document. Il ignore l'ordre des mots.

Supposons que notre corpus est compose des deux phrases suivantes :

P_1 = "Le chat mange le poisson." et P_2 = "Le poisson nage dans l'eau."

Dans ce cas, notre vocabulaire V est donne par :

{Le , chat, mange, le, poisson, nage, dans, l', eau }.

Afin de representer ces deux phrases sous forme vectorielle, on fixe un ordonne les mots du vocabulaire comme suit :

(Le, chat, mange, le, poisson, nage, dans, l', eau).

Ainsi, le vecteur correspondant a P_1 (respectivement P_2) est (1, 1, 1, 1, 1, 0, 0, 0, 0) (respectivement (1, 0, 0, 0, 1, 1, 1, 1, 1)).

1.2.3 TF-IDF

Le TF-IDF est une methode d'extraction de caracteristiques de texte qui evalue l'importance d'un mot dans un document specifique par rapport a son occurence dans l'ensemble du corpus.

$$TF-IDF = TF(t, d) * IDF(t)$$

1. Le vocabulaire est l'ensemble des mots uniques du corpus

ou :

- $TF(t, d)$ est la fréquence d'apparition d'un mot t dans un document d spécifique.

$$TF(t, d) = \frac{\text{nombre total de mots dans le document } d}{\text{nombre d'occurrences du terme } t \text{ dans le document } d}$$

- $IDF(t, d)$ reflète l'importance de chaque terme dans un corpus de documents.

$$IDF(t) = \log\left(\frac{\text{nombre de documents contenant le terme } t \text{ dans le corpus}}{\text{nombre total de documents dans le corpus} + 1}\right)$$

Un TF-IDF élevé indique que le terme est fréquent dans le document mais rare dans l'ensemble du corpus.

1.2.4 Skip-gram

Skip-gram est une méthode permettant de calculer des embeddings de mots, c'est-à-dire des représentations vectorielles denses et continues des mots.

À partir d'un corpus de documents, le modèle apprend un vecteur pour chaque mot du vocabulaire en entraînant un classificateur binaire. Étant donné une paire (w, c) , composée d'un mot cible w et d'un mot candidat c , ce classificateur estime la probabilité que c apparaisse dans le contexte de w . Les poids appris servent ensuite d'embeddings.

L'algorithme parcourt le corpus et considère chaque mot comme un mot cible. Pour chaque mot cible, il prend en compte ses voisins dans une fenêtre de contexte de taille L , qui détermine le nombre de mots à gauche et à droite à considérer.

Par exemple, dans la phrase : « ... lemon, a [tablespoon of apricot jam, a] pinch ... », si le mot cible est apricot et que $L = 2$, les mots de contexte sont tablespoon, of, jam et a.

Skip-gram apprend deux matrices :

- W : embedding des mots en tant que cibles.
- C : embedding des mots en tant que contextes.

Ces matrices sont initialisées aléatoirement puis ajustées au cours de l'entraînement.

Le modèle skip-gram apprend ainsi deux embeddings distincts pour chaque mot i : un embedding cible w_i et un embedding de contexte c_i , stockés respectivement dans les matrices W et C . En pratique, il est courant de combiner ces deux représentations en utilisant la somme $w_i + c_i$ comme embedding final du mot. Alternativement, on peut ne conserver que l'embedding cible w_i et ignorer la matrice C . Pour chaque paire positive² (w, c_{pos}) , on génère k paires négatives³ (w, c_{neg}) , où c_{neg} est un mot tiré aléatoirement selon une distribution unigramme pondérée :

$$P_a(w) = \frac{\text{count}(w)^a}{\sum_{w'} \text{count}(w')^a}. \text{ En pratique, on prend souvent } a = 0.75.$$

L'objectif est de rapprocher les embeddings des paires positives et d'éloigner ceux des paires négatives.

Pour atteindre cet objectif, les embeddings sont ajustés de manière itérative au cours de l'entraînement. Plus précisément, à chaque étape, les vecteurs sont mis à jour en fonction des paires positives et négatives.

2. Une paire positive est un couple composé de w , un mot cible c'est-à-dire le mot que l'on considère, et c_{pos} , un vrai mot qui apparaît dans la fenêtre de contexte autour de w .

3. Une paire négative est un couple composé de w , un mot cible c'est-à-dire le mot que l'on considère, et c_{neg} , un mot tiré aléatoirement dans le vocabulaire (différent de w).

Les équations de mise à jour, de l'instant t à $t + 1$, sont données par :

$$c_{pos}^{t+1} = c_{pos}^t - [s(c_{pos}^t \cdot w^t) - 1]w^t.$$

$$c_{neg}^{t+1} = c_{neg}^t - [s(c_{neg}^t \cdot w^t)]w^t.$$

$$w^{t+1} = w^t - ([s(c_{pos}^t \cdot w^t) - 1]c_{pos}^t + \sum_{i=1}^k [s(c_{neg}^t \cdot w^t)]c_{neg}^i).$$

$s(x)$ est la fonction sigmoïde :

$$s(x) = \frac{1}{1 + \exp(-x)}.$$

1.2.5 Régression logistique

La régression logistique est un classificateur discriminatif⁴, probabiliste et supervisé. Elle est utilisée pour classer des données en deux ou plusieurs catégories (par exemple, positif ou négatif). Dans ce cadre, le modèle est entraîné à partir d'un ensemble d'exemples étiquetés X , c'est-à-dire que chaque observation x est associée à une étiquette y .

Chaque observation x est décrite par un vecteur de caractéristiques $x = [x_1, \dots, x_f]$ où f est le nombre de caractéristiques. Celles-ci peuvent être conçues manuellement ou apprises automatiquement à partir des données, notamment via des méthodes d'apprentissage de représentations ou des gabarits de caractéristiques.

L'objectif de la régression logistique est de prédire la classe $\hat{y}$ d'une nouvelle observation x à partir de ces caractéristiques. Elle résout cette tâche en apprenant, à partir d'un ensemble d'entraînement, un vecteur de poids $w = [w_1, \dots, w_f]$ et un terme de biais b . Chaque poids est un nombre réel et est associé à l'une des caractéristiques. Il représente l'importance de cette caractéristique dans la décision de classification.

Comment un classificateur prédit-il la sortie $\hat{y}$ pour une observation x donnée, et comment apprend-il ses paramètres w et b (poids et biais) à partir d'un jeu d'entraînement X ? Supposons que $X \in \mathbb{R}^{m \times f}$ (une matrice de taille $m \times f$), où m est le nombre d'observations et f le nombre de caractéristiques. On peut intégrer le biais b dans le vecteur de poids w afin de simplifier les calculs lors de la prédiction et de l'entraînement du modèle. Cela conduit à définir un nouveau vecteur de poids $\tilde{w} \in \mathbb{R}^{f+1}$. Pour ce faire, on ajoute une colonne de 1 à la matrice X , ainsi qu'une composante égale à 1 à chaque vecteur de caractéristiques x . On obtient alors une nouvelle matrice $\tilde{X} \in \mathbb{R}^{m \times (f+1)}$ et un vecteur $\tilde{x} \in \mathbb{R}^{f+1}$.

1.2.5.1 Cas de la régression logistique binaire

Le but de la régression logistique binaire est d'entraîner un classificateur capable de prendre une décision binaire à partir d'une observation. Pour une observation donnée, le modèle produit une sortie $\hat{y} \in [0, 1]$, qui représente la probabilité que l'observation appartienne à une classe

4. Un classificateur discriminatif cherche à apprendre à distinguer les classes sans forcément modéliser leur génération.

donnée (classe 0). Une fois cette probabilité calculée, une décision binaire est obtenue en appliquant un seuil (par exemple 0,5) : si $\hat{y}$ est supérieure à ce seuil, l'observation est classée dans la classe 1, sinon elle est classée dans la classe 0.

Pour une observation x , le modèle calcule un score :

$$z = \tilde{w} \cdot \tilde{x}$$

Ce score est ensuite transformé en probabilité à l'aide de la fonction sigmoïde :

$$\hat{y} = s(z) = \frac{1}{1 + \exp(-z)}$$

La prédiction de la classe est obtenue en appliquant un seuil à $\hat{y}$.

Pour un jeu de données X , le modèle calcule un vecteur de scores linéaires :

$$z = \tilde{X} \cdot \tilde{w}^T$$

où $\tilde{w}^T$ est la tranposée de w et z est un vecteur de taille m .

Ces scores sont ensuite transformés en probabilités à l'aide de la fonction sigmoïde appliquée élément par élément :

$$\hat{y} = \sigma(z) = \left[\frac{1}{1 + e^{-z_1}}, \dots, \frac{1}{1 + e^{-z_m}} \right]$$

où y est un vecteur de taille m représentant la probabilité d'appartenance de chaque observation à une classe.

La prédiction de la classe est obtenue en appliquant un seuil (par exemple 0,5) à chaque composante de $\hat{y}$.

Pour que le modèle réalise de bonnes prédictions, il est nécessaire d'apprendre des paramètres adaptés (poids et biais). Pour cela, ces paramètres sont initialisés de manière aléatoire. Ensuite, on mesure l'erreur entre les prédictions $\hat{y}$ et les vraies valeurs y à l'aide d'une fonction de perte. La fonction de perte utilisée pour la régression logistique binaire est la fonction de perte d'entropie croisée :

$$L_{CE}(\hat{y}, y) = -[y \log(s(\tilde{w} \cdot \tilde{x})) + (1 - y) \log(1 - s(\tilde{w} \cdot \tilde{x}))]$$

Cette fonction de perte quantifie l'écart entre les prédictions du modèle et les valeurs réelles. Un algorithme d'optimisation, généralement la descente de gradient, est ensuite utilisé pour minimiser cette perte. Les paramètres sont alors ajustés progressivement de sorte que les prédictions du modèle se rapprochent de plus en plus des valeurs réelles :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta \nabla L(s(\tilde{w} \cdot \tilde{x}), y)$$

où :

— $L(\hat{y}, y)$ est la fonction de perte qui mesure la proximité entre la prédiction $\hat{y}$ et la vraie

valeur y .

- $\nabla L(\hat{y}, y)$ est le gradient de la fonction de perte par rapport aux paramètres $\tilde{w}$, sachant que $\hat{y} = s(\tilde{w} \cdot \tilde{x})$.
- $\tilde{w}^t$ représente la valeur des paramètres à l'itération t .
- η est le taux d'apprentissage.

Comme $\nabla L_{CE}(s(\tilde{w} \cdot \tilde{x}), y) = -(y - \hat{y})\tilde{x}$ alors l'équation ci-dessus devient :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta(s(\tilde{w}^t \cdot \tilde{x}) - y)\tilde{x}$$

Il existe plusieurs façons de calculer cette mise à jour :

- **Stochastique** : elle sélectionne un seul exemple aléatoire x à chaque itération. Puis elle ajuste les paramètres :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta(s(\tilde{w}^t \cdot \tilde{x}) - y)\tilde{x}$$

- **Par lots** : elle utilise tout le jeu d'entraînement pour calculer une seule mise à jour des paramètres.
- **Par mini-lots** : elle utilise un sous-ensemble X' de $\tilde{X}$, de taille n , pour calculer une seule mise à jour des paramètres :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta X'^T (s(X' \cdot (\tilde{w}^t)^T) - y)$$

où $(\tilde{w}^t)^T$ (resp. X'^T) est la transposée de $(\tilde{w}^t)$ (resp. X')

1.2.5.2 Régression logistique multinomiale

La régression logistique multinomiale permet de réaliser une classification avec plus de deux classes.

Étant donné un jeu de données étiqueté X , soit $\mathcal{K}$ l'ensemble des classes possibles et $K = |\mathcal{K}|$ leur nombre. Pour chaque observation x , l'étiquette correcte y est représentée par un vecteur one-hot⁵ de taille K . Si la classe correcte est c , alors $y_c = 1$ et $y_k = 0$ pour tout $k \neq c$.

Le classificateur prend en entrée une observation x et produit en sortie un vecteur $\hat{y} \in \mathbb{R}^K$. Chaque composante $\hat{y}_k$ représente la probabilité que l'observation appartienne à la classe k , avec :

$$\sum_{k=1}^K \hat{y}_k = 1.$$

Dans la régression logistique multinomiale, on associe à chaque classe k un vecteur de poids $w_k \in \mathbb{R}^f$ et un biais b_k . Ces paramètres peuvent être regroupés sous forme matricielle dans une matrice $W \in \mathbb{R}^{K \times f}$ et un vecteur de biais $b \in \mathbb{R}^K$, afin de simplifier les calculs.

Afin de simplifier les calculs, le biais b est intégré dans la matrice des poids W ce qui nous donne une nouvelle matrice $\tilde{W} \in \mathbb{R}^{K \times (f+1)}$.

Pour une observation x , le modèle calcule un score :

5. Un vecteur one-hot est un vecteur contenant un seul élément égal à 1 et tous les autres égaux à 0.

$$z = \tilde{W} \cdot \tilde{x}^T$$

La prédiction est ensuite donnée par :

$$\hat{y} = \text{softmax}(z)$$

où :

$$\text{softmax}(z) = [\frac{\exp(z_1)}{\sum_{j=1}^K \exp(z_j)}, \dots, \frac{\exp(z_K)}{\sum_{j=1}^K \exp(z_j)}]$$

Chaque composante de $\hat{y}$ est comprise entre 0 et 1, et la somme des composantes du vecteur est égale à 1. Chaque composante k de $\hat{y}$ représente la probabilité que l'observation x appartienne à la classe k .

Pour un jeu de données X , on calcule alors les prédictions de tous les observations en une seule opération :

$$Z = \tilde{X} \cdot \tilde{W}^T$$

où $\tilde{W}^T$ est la transposée de $\tilde{W}$

Ensuite, on applique la fonction softmax ligne par ligne sur Z pour obtenir :

$$\hat{Y} = \text{softmax}(Z)$$

où $\hat{Y} \in \mathbb{R}^{m \times K}$, contenant pour chaque exemple une distribution de probabilité sur les classes. Comment les poids sont-ils appris ?

La fonction d'entropie croisée, utilisée pour mesurer l'écart entre la valeur prédite $\hat{y}$ par le modèle et la valeur réelle y , en régression logistique multinomiale pour une observation $\tilde{x}$ de $\tilde{X}$, est donnée par :

$$L_{CE}(\hat{y}, y) = - \sum_{i=1}^K y_i \log \hat{y}_i = - \log \hat{y}_k = - \log \left(\frac{\exp(\tilde{w}_k \cdot \tilde{x})}{\sum_{i=1}^K \exp(\tilde{w}_i \cdot \tilde{x})} \right)$$

où k est la classe correcte et $\tilde{w}_k$ représente les paramètres (poids et biais) associés à la classe k .

La fonction d'entropie croisée, utilisée pour mesurer l'écart entre les prédictions $\hat{Y}$ du modèle et les valeurs réelles Y , en régression logistique multinomiale sur l'ensemble du jeu d'entraînement $\tilde{X}$, est donnée par :

$$L_{CE}(\hat{Y}, Y) = - \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log \hat{y}_j^{(i)}$$

où :

— $\hat{Y} \in \mathbb{R}^{m \times K}$ est la matrice des prédictions ; chaque ligne i de $\hat{Y}$ correspond à une distribu-

tion de probabilités sur les K classes pour l'exemple $\tilde{x}^{(i)}$.

- $Y \in \mathbb{R}^{m \times K}$ est la matrice des étiquettes réelles ; chaque ligne i de Y est un vecteur one-hot représentant la classe réelle de l'exemple $\tilde{x}^{(i)}$.
- $y_j^{(i)}$ est la j -ème composante du vecteur cible $y^{(i)}$.
- $\hat{y}_j^{(i)}$ est la probabilité prédite par le modèle pour la classe j et l'exemple i .

On utilise ensuite un algorithme de descente de gradient pour minimiser cette fonction de perte.

Pour chaque classe k , le gradient de l'entropie croisée par rapport au vecteur de paramètres $\tilde{w}_k$ est donné par :

$$\nabla_{\tilde{w}_k} L_{CE}(\hat{Y}, Y) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_k^{(i)} - y_k^{(i)}) \tilde{x}^{(i)}$$

où :

- $\hat{y}_k^{(i)}$ est la probabilité prédite par le modèle pour la classe k et l'exemple i .
- $y_k^{(i)}$ est la valeur réelle (étiquette) pour la classe k et l'exemple i (issue d'un encodage one-hot).
- $\tilde{x}^{(i)}$ est le vecteur de caractéristiques de l'exemple i , éventuellement augmenté pour inclure le biais.

Les paramètres sont ensuite mis à jour selon la règle de descente de gradient :

$$\tilde{w}_k^{(t+1)} = \tilde{w}_k^{(t)} - \eta \frac{1}{m} \sum_{i=1}^m (\hat{y}_k^{(i)} - y_k^{(i)}) \tilde{x}^{(i)}$$

où :

- $\tilde{w}_k^{(t)}$ est le vecteur de paramètres de la classe k à l'itération t .
- $\tilde{w}_k^{(t+1)}$ est le vecteur de paramètres mis à jour après l'itération t .
- η est le taux d'apprentissage.

1.2.6 Réseaux de neurones récurrents

Les réseaux de neurones sont une approche d'apprentissage automatique inspirée du fonctionnement des neurones biologiques du cerveau humain. Ils apprennent à partir de données et peuvent être utilisés pour des tâches de classification, de régression ou de prédiction. Ils sont particulièrement adaptés à la modélisation de relations non linéaires complexes.

Un réseau de neurones artificiel est généralement composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. La profondeur d'un réseau correspond au nombre de couches de neurones qu'il contient ; la couche d'entrée n'est généralement pas comptée dans ce nombre. Chaque couche est composée d'un certain nombre d'unités appelées neurones artificiels. Le nombre de neurones d'une couche correspond à la taille de cette couche. La taille de la couche d'entrée est égale à la dimension du vecteur d'entrée x .

Dans un réseau de neurones classique entièrement connecté, les neurones d'une même couche ne sont généralement pas connectés entre eux. Les connexions se font entre couches successives : chaque neurone d'une couche est relié à tous les neurones de la couche suivante.

Chaque neurone d'une couche cachée ou de sortie reçoit en entrée un vecteur de valeurs réelles x , associé à un vecteur de poids w , ainsi qu'un biais b , généralement un scalaire. Le neurone calcule d'abord une combinaison linéaire :

$$z = w \cdot x + b$$

Puis une fonction d'activation non linéaire f est appliquée afin d'obtenir la sortie du neurone :

$$y = f(z)$$

Cette fonction d'activation peut être, par exemple, la fonction sigmoïde, la tangente hyperbolique ($\tanh$) ou la fonction ReLU (Rectified Linear Unit). La sortie d'une couche est donc un vecteur contenant les sorties de tous ses neurones.

Lorsqu'une entrée x est fournie au réseau, l'information se propage de la couche d'entrée vers les couches cachées puis vers la couche de sortie. Afin de simplifier les calculs, les poids des neurones d'une même couche sont regroupés dans une matrice de poids, et les biais dans un vecteur de biais. Ces paramètres sont initialisés aléatoirement puis ajustés pendant l'entraînement afin de minimiser l'erreur du modèle.

Un réseau de neurones récurrent est tout réseau de neurones qui contient un cycle dans ses connexions internes ce qui signifie que la valeur d'une certaine unite neuronale depend directement ou indirectement de ses propres sorties precedentes comme entree.[4]. Grâce à cette propriété, les RNN sont capables de conserver une information sur le contexte précédent. Ils sont donc particulièrement adaptés au traitement de données séquentielles telles que les séries temporelles, le texte, la parole ou les séquences biologiques.

Pour comprendre le fonctionnement des RNN, considérons le cas d'un RNN simple, c'est-à-dire un réseau récurrent qui associe une séquence d'entrée à une séquence de sortie de même longueur. Les éléments de la séquence sont traités un à un, dans l'ordre.

La figure 1.2 illustre la structure de ce réseau.

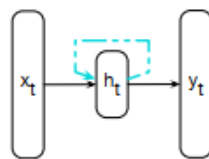


FIGURE 1.2 – Structure de la RNN simple

L'indice t représente le temps ou la position dans la séquence.

À chaque instant t :

- $x^{(t)}$ représente l'entrée du réseau ;
- $h^{(t)}$ représente la couche caché du réseau ;
- $y^{(t)}$ représente la sortie du réseau.

La couche cachée $h^{(t)}$ dépend à la fois de l'entrée actuelle $x^{(t)}$ et de la couche cachée précédente $h^{(t-1)}$. Ainsi, la couche caché $h^{(t)}$ influence donc directement la valeur de la sortie $y^{(t)}$ et

indirectement les valeurs des couches cachees futures et sorties futures .

Le réseau récurrent peut être représenté sous deux formes :

- une représentation compacte avec une boucle récurrente ;
- une représentation déroulée dans le temps, où chaque instant de temps est représenté explicitement.

Le déroulement du réseau est particulièrement utile pour appliquer l'algorithme de rétropropagation à travers le temps (Backpropagation Through Time ou BPTT), utilisé pour calculer les gradients et mettre à jour les poids. Ces deux représentations sont illustrées dans la figure 1.3.

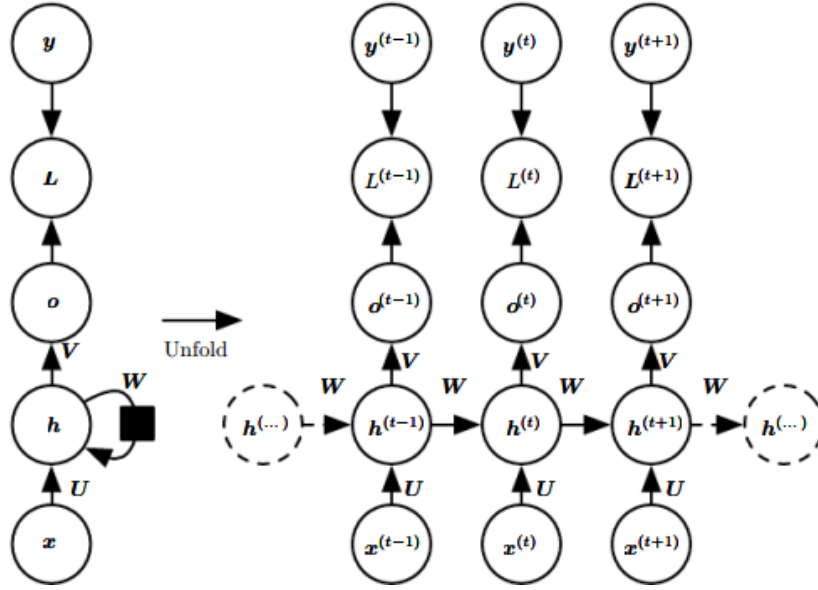


FIGURE 1.3 – La figure à gauche est une représentation compacte de la RNN simple et celle à droite est sa représentation déroulée dans le temps

D'après la figure ci-dessus, dans un RNN simple, le passage d'une séquence d'entrées à une séquence de sorties se fait de la manière suivante.

La matrice U relie l'entrée à la couche cachée, la matrice W relie l'état caché précédent à l'état caché courant, et la matrice V relie la couche cachée à la couche de sortie. Les mêmes matrices de poids sont utilisées à chaque instant de temps. Le calcul commence par l'initialisation de l'état caché initial, souvent :

$$h^{(0)} = 0$$

Ensuite, pour chaque instant t allant de 1 à T , où T est la longueur de la séquence, les calculs suivants sont effectués :

$$a^{(t)} = b + W \cdot h^{(t-1)} + U \cdot x^{(t)}$$

$$h^{(t)} = f(a^{(t)})$$

$$o^{(t)} = V \cdot h^{(t)} + c$$

$$y^{(t)} = g(o^{(t)})$$

où :

- f est la fonction d'activation non lineaire de la couche cachée.
- g est la fonction d'activation non lineaire de la couche de sortie.
- b est le vecteur de biais de la couche cachée.
- c est le vecteur de biais de la couche de sortie.

La séquence y est composé des $y^{(t)}$ pour t allant de 1 jusqu'à T .

L'objectif de l'entraînement est d'apprendre les matrices de poids W , U et V ainsi que les biais b et c . Pour cela, on utilise :

- un jeu de données d'entraînement ;
- une fonction de perte permettant de mesurer l'écart entre les prédictions du modèle et les valeurs réelles.

L'entraînement d'un réseau de neurones récurrent (RNN) se déroule généralement en deux étapes : la propagation avant et la rétropropagation à travers le temps.

La première étape, appelée propagation avant, consiste à faire passer chaque séquence d'entrée dans le réseau afin de calculer les sorties prédites. À chaque instant t , une perte locale $L^{(t)}$ est calculée en comparant la sortie prédite à la sortie réelle. La perte totale associée à une séquence est ensuite obtenue en additionnant les pertes sur tous les instants :

$$L = \sum_{t=1}^T L^{(t)}$$

où T désigne la longueur de la séquence.

La deuxième étape consiste à calculer les gradients de la perte totale par rapport aux paramètres du réseau (W , U , V , b et c). Ces gradients indiquent comment modifier les paramètres afin de réduire l'erreur du modèle.

Afin de faciliter le calcul des gradients, le réseau est déroulé dans le temps pour former un graphe computationnel. Cette représentation permet d'appliquer l'algorithme de rétropropagation à travers le temps.

Dans le graphe computationnel déroulé, les nœuds incluent les paramètres du réseau (W , U , V , b et c) ainsi que les variables dépendant du temps : $x^{(t)}$, $h^{(t)}$, $o^{(t)}$ et $L^{(t)}$.

Le calcul des gradients commence par les nœuds les plus proches de la fonction de perte. On calcule d'abord le gradient de la perte totale L par rapport à chaque perte locale $L^{(t)}$:

$$\frac{\partial L}{\partial L^{(t)}} = 1$$

Ensuite, on calcule le gradient de L par rapport à la sortie $o^{(t)}$ pour chaque instant t :

$$\nabla_{o^{(t)}} L = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o^{(t)}}$$

Au dernier instant T , l'état caché $h^{(T)}$ influence uniquement la sortie $o^{(T)}$. Son gradient est donc donné par :

$$\nabla_{h^{(T)}} L = V^\top \nabla_{o^{(T)}} L$$

Pour les instants précédents, chaque état caché $h^{(t)}$ influence à la fois la sortie courante $o^{(t)}$ et l'état caché suivant $h^{(t+1)}$. Le gradient doit donc être propagé à travers ces deux dépendances.

On itère ainsi de $t = T - 1$ jusqu'à $t = 1$ afin de calculer récursivement le gradient par rapport à chaque état caché :

$$\nabla_{h^{(t)}} L = \left(\frac{\partial h^{(t+1)}}{\partial h^{(t)}} \right)^\top \nabla_{h^{(t+1)}} L + \left(\frac{\partial o^{(t)}}{\partial h^{(t)}} \right)^\top \nabla_{o^{(t)}} L$$

Une fois les gradients calculés sur les nœuds internes du graphe computationnel, on peut calculer les gradients par rapport aux paramètres du réseau.

Le gradient par rapport au biais de sortie c est donné par :

$$\nabla_c L = \sum_t \left(\frac{\partial o^{(t)}}{\partial c} \right)^\top \nabla_{o^{(t)}} L$$

Le gradient par rapport au biais caché b est :

$$\nabla_b L = \sum_t \left(\frac{\partial h^{(t)}}{\partial b} \right)^\top \nabla_{h^{(t)}} L$$

Le gradient par rapport à la matrice de poids V est :

$$\nabla_V L = \sum_t (\nabla_{o^{(t)}} L) (h^{(t)})^\top$$

Le gradient par rapport à la matrice récurrente W est :

$$\nabla_W L = \sum_t \sum_i \frac{\partial L}{\partial h_i^{(t)}} \nabla_{W^{(t)}} h_i^{(t)}$$

Le gradient par rapport à la matrice d'entrée U est :

$$\nabla_U L = \sum_t \sum_i \frac{\partial L}{\partial h_i^{(t)}} \nabla_{U^{(t)}} h_i^{(t)}$$

Les gradients calculés sont ensuite utilisés pour mettre à jour les paramètres du réseau à l'aide d'un algorithme d'optimisation basé sur le gradient, comme la descente de gradient.

À l'itération $i + 1$, les paramètres sont mis à jour selon les équations suivantes :

$$c^{(i+1)} = c^{(i)} - \eta \nabla_c L$$

$$b^{(i+1)} = b^{(i)} - \eta \nabla_b L$$

$$V^{(i+1)} = V^{(i)} - \eta \nabla_V L$$

$$W^{(i+1)} = W^{(i)} - \eta \nabla_W L$$

$$U^{(i+1)} = U^{(i)} - \eta \nabla_U L$$

où η désigne le taux d'apprentissage.

Chapitre 2

Byte Pair Encoding

Le Byte Pair Encoding (BPE) est une technique de compression de données consistant à fusionner itérativement les paires d'octets ou de caractères consécutifs les plus fréquentes d'un texte ou d'une séquence de données en un nouveau symbole unique. Le processus est répété jusqu'à atteindre un nombre prédéfini de fusions ou jusqu'à ce qu'il ne reste plus de paires fréquentes.

Cette méthode a été décrite pour la première fois en 1994 par Philip Gage. Elle est ensuite devenue l'une des techniques de tokenisation en sous-mots les plus utilisées en traitement automatique du langage naturel (TALN).

2.1 Présentation

L'idée de l'algorithme est de représenter les paires les plus fréquentes par des marqueurs dédiés. Ce faisant, il identifie les séquences d'éléments (caractères) les plus fréquentes. Ensuite, en utilisant ces informations, il peut encoder des textes.

2.1.1 Logique principale

La logique de l'algorithme est illustrée par les étapes suivantes :

1. Construire un vocabulaire à partir d'un corpus de mots.
2. Identifier la paire la plus fréquente : compter les paires de symboles et sélectionner celle ayant la fréquence la plus élevée.
3. Produire un nouvel élément de vocabulaire pour la paire la plus fréquente : fusionner cette paire en un nouveau symbole et mettre à jour le corpus afin de refléter ce nouvel élément.
4. Répéter les étapes précédentes jusqu'à atteindre un nombre cible de fusions.
5. Enregistrer la table des fusions.
6. Encoder une chaîne de caractères à l'aide de la table des fusions.

2.1.2 Illustration à travers un exemple

```
b := BytePairEncoder new.  
b fromText: 'FRED FED TED BREAD, AND TED FED FRED BREAD'.
```

Inspection du corpus

Tout d'abord, une fois le texte d'entrée tokenisé, on obtient le vocabulaire suivant.

```
corpus  
#('F' 'E' 'D' ' ') 2  
#('B' 'R' 'E' 'A' 'D' ' ') 2  
#('A' 'N' 'D' ' ') 1  
#('T' 'E' 'D' ' ') 2  
#('F' 'R' 'E' 'D' ' ') 2
```

Lorsque l'on observe le système, on obtient les éléments suivants :

- un vocabulaire : 'T' 'A' 'F' 'D' 'B' ' ' 'R' 'N' 'E'
- un corpus : comme indiqué ci-dessus
- une liste de paires consécutives avec leurs fréquences
- une liste de fusions vide

```
voc  
'T' 'A' 'F' 'D' 'B' ' ' 'R' 'N' 'E'  
corpus  
#('F' 'E' 'D' ' ') 2  
#('B' 'R' 'E' 'A' 'D' ' ') 2  
#('A' 'N' 'D' ' ') 1  
#('T' 'E' 'D' ' ') 2  
#('F' 'R' 'E' 'D' ' ') 2  
pairs  
#('N' 'D') 1  
#('B' 'R') 2  
#('A' 'N') 1  
#('A' 'D') 2  
#('F' 'E') 2  
#('D' ' ') 9  
#('R' 'E') 4  
#('E' 'A') 2  
#('T' 'E') 2  
#('E' 'D') 6
```

```
#('F' 'R') 2
merges
```

On observe que la paire la plus fréquente est `#('A' 'B')`. Elle sera donc celle qui sera fusionnée.

Itération 1

Maintenant, examinons la situation avec la première fusion.

b mergeOneStep

On observe que la paire la plus fréquente `#('D' '_')` a été fusionnée. Cela signifie que :

- Elle a été ajoutée au vocabulaire : ici, nous obtenons par exemple `'D_'` comme nouvel élément. À noter que si les caractères étaient encodés par des nombres, les éléments fusionnés seraient simplement représentés par un nouvel identifiant numérique.
- Elle a été ajoutée à la liste des fusions.
- Le corpus a été mis à jour afin de refléter la présence du nouvel élément du vocabulaire. Par exemple, `#('F' 'E' 'D' '_')` est désormais encodé comme `#('F' 'E' 'D_')`.
- Toutes les paires ont été recalculées à partir de la version mise à jour du corpus. Ainsi, la paire la plus fréquente devient maintenant `#('E' 'D_')`.

```
voc
'F' 'R' 'T' 'B' 'N' 'D_' '_ ' 'E' 'D' 'A'
corpus
#('F' 'E' 'D_') 2
#('A' 'N' 'D_') 1
#('B' 'R' 'E' 'A' 'D_') 2
#('T' 'E' 'D_') 2
#('F' 'R' 'E' 'D_') 2
pairs
#('E' 'D_') 6
#('B' 'R') 2
#('A' 'N') 1
#('N' 'D_') 1
#('F' 'E') 2
#('E' 'A') 2
#('R' 'E') 4
#('T' 'E') 2
#('F' 'R') 2
#('A' 'D_') 2
merges
#('D' '_')->'D_'
```

Itération 2

L'algorithme répète le même processus. Ainsi, la fusion $\#('E' \text{ 'D_'}) \rightarrow 'ED_'$ est ajoutée à la liste des fusions. Le vocabulaire, le corpus et les paires sont ensuite mis à jour.

```
voc
  'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' ' ' 'E' 'D' 'A'
corpus
  #('T' 'ED_') 2
  #('F' 'R' 'ED_') 2
  #('A' 'N' 'D_') 1
  #('B' 'R' 'E' 'A' 'D_') 2
  #('F' 'ED_') 2
pairs
  #('T' 'ED_') 2
  #('B' 'R') 2
  #('A' 'N') 1
  #('N' 'D_') 1
  #('R' 'E') 2
  #('E' 'A') 2
  #('F' 'R') 2
  #('A' 'D_') 2
  #('R' 'ED_') 2
  #('F' 'ED_') 2
merges
  #('D' ' ')->'D_'
  #('E' 'D_')->'ED_'
```

On observe maintenant qu'il existe plusieurs paires ayant la fréquence la plus élevée. L'algorithme en sélectionnera une parmi elles.

Itération 3

Voici l'itération suivante. La fusion $\#('T' \text{ 'ED_'}) \rightarrow 'TED_'$ a été ajoutée à la liste des fusions. Il est à noter qu'il s'agit de la première paire apparaissant dans le journal présenté ici, mais cela est simplement dû au fait que la logique d'affichage utilise la même implémentation de type *bag* ainsi que la même logique de tri que celle utilisée par l'algorithme lorsqu'il recherche l'association la plus fréquente. Cela n'est pas lié à l'algorithme lui-même. Une autre paire ayant la même fréquence aurait pu être sélectionnée.

```
voc
  'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' ' ' 'E' 'TED_' 'D' 'A'
corpus
```

```

#('F' 'R' 'ED_') 2
#('A' 'N' 'D_') 1
#('B' 'R' 'E' 'A' 'D_') 2
#('TED_') 2
#('F' 'ED_') 2
pairs
#('B' 'R') 2
#('A' 'N') 1
#('N' 'D_') 1
#('R' 'E') 2
#('E' 'A') 2
#('F' 'R') 2
#('A' 'D_') 2
#('R' 'ED_') 2
#('F' 'ED_') 2
merges
#('D' '_' )->'D_'
#('E' 'D_')->'ED_'
#('T' 'ED_')->'TED_'

```

Itération 4 Voici une autre itération au cours de laquelle la paire #('B' 'R') a été sélectionnée.

```

voc
'BR' 'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' '_' 'E' 'TED_' 'D' 'A'
corpus
#('F' 'R' 'ED_') 2
#('F' 'ED_') 2
#('A' 'N' 'D_') 1
#('TED_') 2
#('BR' 'E' 'A' 'D_') 2
pairs
#('BR' 'E') 2
#('A' 'N') 1
#('N' 'D_') 1
#('E' 'A') 2
#('F' 'R') 2
#('F' 'ED_') 2
#('R' 'ED_') 2
#('A' 'D_') 2
merges
#('D' '_' )->'D_'

```

```
#('E' 'D_')->'ED_'
#('T' 'ED_')->'TED_'
#('B' 'R')->'BR'
```

Jusqu'à ce qu'aucune fusion supplémentaire ne soit possible.

L'algorithme continue comme expliqué jusqu'à ce qu'il n'y ait plus de paires à fusionner, c'est-à-dire lorsque la fréquence des paires est égale à un. On obtient la situation suivante.

```
oc
  'BR' 'BREA' 'R' 'TED_' 'ED_' 'E' 'N' 'FR' 'A' 'BRE' '_' 'F' 'D_' 'BREAD_' 'D' 'FRED_'
corpus
#('FRED_') 2
#('FED_') 2
#('BREAD_') 2
#('A' 'N' 'D_') 1
#('TED_') 2
pairs
#('A' 'N') 1
#('N' 'D_') 1
merges
#('D' '_' )->'D_'
#('E' 'D_')->'ED_'
#('T' 'ED_')->'TED_'
#('B' 'R')->'BR'
#('BR' 'E')->'BRE'
#('BRE' 'A')->'BREA'
#('F' 'R')->'FR'
#('BREA' 'D_')->'BREAD_'
#('F' 'ED_')->'FED_'
#('FR' 'ED_')->'FRED_'
```

Utilisation des fusions

Nous pouvons maintenant encoder un texte. La logique de cette étape est la suivante :

- tokeniser le texte ;
- parcourir la liste des fusions dans l'ordre et appliquer chaque fusion l'une après l'autre sur le résultat de l'application précédente ;
- une fois arrivé à la fin de la liste des fusions, on obtient le texte encodé.

Imaginons que nous voulions encoder 'FRED EATS BREAD'. Lors de la phase de tokenisation, nous obtenons la liste suivante :

```
'F' 'R' 'E' 'D' '_' 'E' 'A' 'T' 'S' '_' 'B' 'R' 'E' 'A' 'D_'
```

À noter que l'on pourrait également travailler au niveau des mots et utiliser une structure de type Bag afin de factoriser le traitement pour des mots similaires.

Ensuite, nous demandons à l'algorithme d'encoder le texte. On obtient la liste d'éléments suivante : 'FRED_' 'E' 'A' 'T' 'S' ' ' 'BREAD'

```
self encode: 'FRED EATS BREAD'  
> an OrderedCollection('FRED_' 'E' 'A' 'T' 'S' ' ' 'BREAD_')
```

Ce que l'on observe, c'est que les mots fréquents ont été substitués. Ainsi, si on l'examine sous une forme numérique, on obtient :

```
'ABRC' asArray collect: [ :each | each digitValue -9 ]  
> #(1 2 18 3)
```

```
'FRED EATS BREAD' asArray collect: [ :each | each digitValue -9 ]  
> #(6 18 5 4 -10 5 1 20 19 -10 2 18 5 1 4)
```

```
'EATS_' asArray collect: [ :each | each digitValue -9 ]  
> #(5 1 20 19 -10)
```

Par conséquent, le texte passe de 15 à 7 éléments (car 'FRED_' et 'BREAD_' comptent chacun comme un seul élément).

2.1.3 Observation sur l'algorithme

L'exécution que nous avons présentée correspond à une implémentation naïve et simple. Elle reconstruit l'ensemble complet des paires à chaque itération. De plus, une seule paire est fusionnée à la fois.

Ce que montre cette exécution, c'est que le vocabulaire, le corpus et les paires doivent être mis à jour à chaque fusion.

Plusieurs paires pourraient être fusionnées au cours d'un même passage si l'algorithme prend en compte le fait que les paires fusionnées simultanément ne doivent ni commencer ni se terminer par un élément similaire.

Lorsque l'on dispose des paires suivantes :

```
#('N' 'D') 1  
#('B' 'R') 2  
#('A' 'N') 1  
#('A' 'D') 2  
#('F' 'E') 2  
#('D' ' ') 9  
#('R' 'E') 4  
#('E' 'A') 2
```

```
#('T' 'E') 2
#('E' 'D') 6
#('F' 'R') 2
```

Lors de la fusion de `#('D' ' ') 9`, l'algorithme ne devrait pas fusionner les paires se terminant par 'D' ou commençant par ' ', mais il pourrait en fusionner d'autres, comme 'R' 'E'. Or, la fusion simultanée de plusieurs paires conduit à des encodages différents.

2.2 Implémentation

L'objectif cette section est d'implémenter l'algorithme Byte Pair Encoding (BPE) en Pharo. Le fonctionnement de BPE est décrit la section **1.3.1.1**.

2.2.1 Algorithme pseudo-code

L'algorithme 1 prend en entrée un texte ainsi que le nombre de fusions souhaité ou la taille du vocabulaire souhaitée. Il permet l'entraînement de l'algorithme BPE sur le texte donné en entrée et retourne :

- un dictionnaire des fusions ;
- ainsi que le vocabulaire appris.

Le dictionnaire des fusions contient :

- comme clés, les nouveaux symboles créés lors des différentes opérations de fusion ;
- et comme valeurs, l'ordre dans lequel ces symboles ont été appris pendant l'entraînement.

L'algorithme commence par initialiser le vocabulaire à l'aide de la fonction `InitializeVocabulary()`. Cette fonction construit l'ensemble des symboles présents dans le texte d'entrée. Le vocabulaire contient également le symbole spécial ' ', utilisé pour indiquer la fin d'un mot dans BPE.

Ensuite, la fonction `ComputeWordsFrequencies()` calcule la fréquence de chaque mot du texte tout en transformant chaque mot en une liste de caractères. Les mots prétraités ainsi que leurs fréquences sont ensuite enregistrés dans un dictionnaire représentant le corpus.

Après cette étape, l'algorithme effectue zéro, une ou plusieurs itérations jusqu'à ce que le nombre de fusions souhaité la taille du vocabulaire souhaitée soit atteint.

À chaque itération :

- les fréquences des paires de symboles consécutifs présentes dans le corpus sont calculées à l'aide de la fonction `ComputePairs()` ;
- la paire possédant la fréquence maximale est ensuite sélectionnée ;
- les deux symboles composant cette paire sont fusionnés afin de créer un nouveau symbole.
- le nouveau symbole obtenu est ajouté au dictionnaire des fusions, où la valeur associée correspond au numéro de l'itération courante, ainsi qu'au vocabulaire.
- le corpus est mis à jour à l'aide de la fonction `UpdateCorpus()`, qui remplace toutes les occurrences de la paire sélectionnée par le nouveau symbole fusionné.

Algorithm 1: TrainBPE

Input: text, mergeCount

Output: mergesDictionary, vocabulary

```
1 vocabulary  $\leftarrow$  InitializeVocabulary(text);
2 corpus  $\leftarrow$  computeWordsFrequency(text);
3 mergesDictionary  $\leftarrow$  empty dictionary;
4  $i \leftarrow$  mergeCount - Size(vocabulary);
5  $j \leftarrow 1$ ;
6 while  $i > 0$  do
7    $pairs \leftarrow$  ComputePairs(corpus);
8    $bestPair \leftarrow$  Max(pairs);
9    $newSymbol \leftarrow bestPair[1] + bestPair[2]$ ;
10  mergesDictionary[ $newSymbol$ ]  $\leftarrow j$ ;
11  Add(vocabulary,  $newSymbol$ );
12  corpus  $\leftarrow$  UpdateCorpus(corpus,  $bestPair$ ,  $newSymbol$ );
13   $j = j + 1$ ;
14   $i \leftarrow i - 1$ ;
15 return mergesDictionary, vocabulary
```

L’algorithme 2 prend en entrée un texte et retourne un dictionnaire représentant le texte prétraité. Dans ce dictionnaire :

- les clés correspondent à des listes de caractères représentant les mots du corpus ;
- les valeurs correspondent aux fréquences d’apparition de ces mots dans le texte.

L’algorithme parcourt le texte caractère par caractère afin d’identifier les mots. Chaque mot représenté sous forme d’une liste de caractères représentant sa décomposition initiale utilisée par BPE.

L’espace joue le rôle de séparateur de mots. Lorsqu’un espace est rencontré pendant le parcours du texte, l’algorithme considère que le mot courant est terminé. La liste de caractères construite jusqu’à ce point est alors ajoutée au dictionnaire, puis une nouvelle liste est initialisée pour traiter le mot suivant.

De manière générale, les espaces ainsi que les signes de ponctuation peuvent être utilisés comme séparateurs de mots lors du prétraitement d’un texte. Toutefois, dans cet algorithme, seuls les espaces sont considérés comme séparateurs.

Avant l’ajout d’une liste de caractères dans le dictionnaire, le symbole spécial ‘_’ est ajouté à la fin de la liste de caractères afin d’indiquer la frontière de fin de mot. Ce symbole permet à BPE de distinguer les caractères appartenant à des mots différents et de préserver les limites des mots lors des opérations de fusion.

Après la construction de la liste de caractères représentant le mot, l’algorithme vérifie si ce mot existe déjà dans le dictionnaire :

- s’il existe, sa fréquence est incrémentée de 1 ;

— sinon, sa fréquence est initialisée à 1.

Ce processus est répété pour chaque mot du texte. À la fin du parcours, l'algorithme retourne un dictionnaire contenant l'ensemble des mots du corpus sous forme de listes de caractères ainsi que leurs fréquences d'apparition.

Algorithm 2: ComputeWordsFrequencies

Input: text

Output: corpus

```
1 corpus  $\leftarrow \{\}$ ;
2 word  $\leftarrow$  empty list;
3  $i \leftarrow 1$ ;
4 while  $i \leq \text{Size}(\text{text})$  do
5   if  $\text{text}[i] \neq ' '$  then
6      $\text{Add}(\text{word}, \text{text}[i])$ ;
7   if  $\text{text}[i] = ' '$  and  $\text{Size}(\text{word}) \neq 0$  then
8      $\text{Add}(\text{word}, \_)$ ;
9     if  $\text{word} \in \text{corpus}$  then
10       $\text{corpus}[\text{word}] \leftarrow \text{corpus}[\text{word}] + 1$ ;
11    else
12       $\text{corpus}[\text{word}] \leftarrow 1$ ;
13     $\text{word} \leftarrow$  empty list;
14    $i \leftarrow i + 1$ ;
15 if  $\text{Size}(\text{word}) > 0$  then
16    $\text{Add}(\text{word}, \_)$ ;
17   if  $\text{word} \in \text{corpus}$  then
18      $\text{corpus}[\text{word}] \leftarrow \text{corpus}[\text{word}] + 1$ ;
19   else
20      $\text{corpus}[\text{word}] \leftarrow 1$ ;
21 return corpus
```

L'algorithme 3 prend en entrée un dictionnaire représentant le texte, dans lequel chaque clé correspond à une liste de symboles (représentant un mot) et chaque valeur correspond à la fréquence d'apparition de ce mot dans le texte. Il retourne un dictionnaire des paires de symboles, où les clés sont les paires de symboles adjacents et les valeurs sont leurs fréquences dans le texte.

L'algorithme parcourt chaque entrée du dictionnaire. Pour chaque liste de symboles associée à un mot, il identifie toutes les paires de symboles adjacents dans la liste.

Chaque paire est ensuite ajoutée au dictionnaire des paires. Si la paire existe déjà, sa fréquence est incrémentée de la fréquence du mot correspondant dans le texte. Sinon, elle est initialisée avec cette même valeur.

Algorithm 3: ComputePairs

Input: corpus**Output:** pairs

```
1 pairs  $\leftarrow$  empty dictionary;
2 foreach  $(key, value) \in \text{corpus}$  do
3    $i \leftarrow 1$ ;
4   while  $i < \text{Size}(key)$  do
5      $pair \leftarrow [key[i], key[i + 1]]$ ;
6     if  $pair \in \text{pairs}$  then
7        $\text{pairs}[pair] \leftarrow \text{pairs}[pair] + value$ ;
8     else
9        $\text{pairs}[pair] \leftarrow value$ 
10     $i \leftarrow i + 1$ 
11 return pairs
```

L’algorithme 4 prend en entrée un dictionnaire représentant le corpus, dans lequel chaque clé correspond à une liste de symboles (représentant un mot) et chaque valeur correspond à la fréquence d’apparition de ce mot dans le texte, ainsi qu’une paire de symboles et un nouveau symbole issu de leur fusion.

Il retourne un dictionnaire mis à jour, dans lequel les occurrences de la paire de symboles fournie en entrée sont remplacées par le nouveau symbole dans toutes les listes de symboles du dictionnaire.

L’algorithme parcourt chaque élément du dictionnaire. Pour chaque liste de symboles associée à un mot, il construit une nouvelle liste en remplaçant toutes les occurrences consécutives de la paire de symboles spécifiée par le symbole fusionné.

Lors du parcours de la liste, si les deux symboles consécutifs correspondent exactement à la paire à fusionner, alors le nouveau symbole est ajouté à la nouvelle liste et l’indice est avancé de deux positions. Sinon, le symbole courant est ajouté à la nouvelle liste et l’algorithme avance d’une seule position.

Une fois la liste entièrement traitée, la nouvelle liste de symboles est insérée dans un nouveau dictionnaire avec la même fréquence que celle de la liste initiale.

Ce processus permet de mettre à jour le corpus après chaque fusion.

Algorithm 4: UpdateCorpus

Input: corpus, mostFrequentPair, newSymbol

Output: updatedCorpus *corpus1*

```
1 updatedCorpus  $\leftarrow$  empty dictionary;
2 foreach  $(key, value) \in \text{corpus}$  do
3    $i \leftarrow 1$ ;
4    $word \leftarrow$  empty list;
5   while  $i < \text{Size}(key)$  do
6     if  $key[i] = \text{mostFrequentPair}[1]$  and  $key[i + 1] = \text{mostFrequentPair}[2]$  then
7       Add(word, newSymbol);
8        $i \leftarrow i + 2$ ;
9     else
10      Add(word, key[i]);
11       $i \leftarrow i + 1$ ;
12   updatedCorpus[word]  $\leftarrow$  value;
13 return updatedCorpus
```

L’algorithme 5 prend en entrée un texte ainsi qu’un dictionnaire contenant les fusions apprises lors de l’entraînement de BPE, avec leur ordre d’ajout dans le dictionnaire. Il retourne une liste de tokens représentant le texte segmenté en sous-mots.

Cet algorithme permet de tokeniser un texte en appliquant les fusions apprises lors de la phase d’entraînement du modèle BPE. La liste de sortie est initialisée vide et sera progressivement remplie par les sous-mots obtenus.

Dans un premier temps, un prétraitement est appliqué au texte. Celui-ci est découpé en une liste de mots à l’aide de la fonction `Split()`. Chaque mot est représenté sous forme d’une liste de caractères. Cette représentation permet de conserver l’ordre des caractères et des mots dans le texte. De manière générale, les espaces, les tabulations et les signes de ponctuation peuvent être utilisés comme séparateurs de mots, mais dans cet algorithme, seul l’espace est considéré comme séparateur.

Après ce prétraitement, l’algorithme applique de manière itérative les règles de fusion contenues dans le dictionnaire des fusions. À chaque itération, la fonction `FindPairToMerge()` identifie la paire de symboles présente dans le texte prétraité qui correspond à une fusion connue et ayant la priorité la plus élevée (c’est-à-dire la plus ancienne selon l’ordre des fusions).

Si aucune paire à fusionner n’est trouvée, le processus s’arrête. Sinon, la paire identifiée est fusionnée pour former un nouveau symbole.

La liste de mots est ensuite mise à jour à l’aide de la fonction `UpdateCorpus()`, qui remplace toutes les occurrences de la paire par le nouveau symbole.

Ce processus est répété jusqu’à ce qu’il n’y ait plus de paire de symboles pouvant être fusionnée.

Enfin, une fois qu’il n’y a plus de paire à fusionner, l’algorithme parcourt les éléments de la

liste des mots. Chaque élément est une liste de symboles. Ces symboles sont ensuite ajoutés à la liste finale des tokens, en conservant leur ordre.

Le résultat final est une liste ordonnée de sous-mots représentant la tokenisation complète du texte selon le modèle BPE appris.

Algorithm 5: Tokenize

Input: text, mergesDictionary

Output: tokenizedText

```

1 corpus  $\leftarrow$  Split(text);
2 a = True;
3 tokenizedText  $\leftarrow$  empty list;
4 while a = True do
5   pair  $\leftarrow$  FindPairToMerge(corpus, mergesDictionary);
6   if pair = null then
7     a  $\leftarrow$  False;
8   else
9     newSymbol  $\leftarrow$  pair[1] + pair[2];
10    corpus  $\leftarrow$  UpdateCorpus(corpus, pair, newSymbol);
11 i = 1;
12 while i  $\leq$  Size(corpus) do
13   j  $\leftarrow$  1;
14   symbols  $\leftarrow$  corpus[i];
15   while j  $\leq$  Size(symbols) do
16     Add(tokens, symbols[j]);
17     j  $\leftarrow$  j + 1;
18   i  $\leftarrow$  i + 1;
19 return tokens

```

L'algorithme 6 prend en entrée un texte et retourne une liste de mots représentés sous forme de liste de caractères. Chaque élément de cette liste correspond à un mot du texte, lui-même représenté comme une liste de caractères. Cette structure permet de conserver à la fois l'ordre des mots dans le texte et l'ordre des caractères dans chaque mot.

La liste de sortie est initialisée vide au début de l'algorithme. Ensuite, l'algorithme parcourt le texte caractère par caractère afin de construire progressivement les mots.

Pour chaque caractère rencontré, s'il ne s'agit pas d'un espace, il est ajouté à la liste courante de caractères représentant le mot en cours de construction. Lorsqu'un espace est rencontré et que la liste courante n'est pas vide, cela signifie que le mot est terminé.

Dans ce cas, le symbole spécial '_' est ajouté à la fin de la liste de caractères afin de marquer explicitement la frontière de fin de mot. Cette liste est ensuite ajoutée à la liste principale contenant tous les mots du texte, puis une nouvelle liste est initialisée pour traiter

le mot suivant.

De manière générale, les espaces, la ponctuation et les tabulations peuvent être utilisés comme séparateurs de mots lors du prétraitement. Toutefois, dans cet algorithme, seul l'espace est utilisé comme séparateur.

Le résultat final est une liste ordonnée de mots du texte, chacun représenté sous forme de liste de caractères annotée avec le symbole de fin de mot.

Algorithm 6: Split

Input: text

Output: splittedText

```
1 splittedText  $\leftarrow$  empty list;
2 tokens  $\leftarrow$  empty list;
3 i  $\leftarrow$  1;
4 while  $i \leq \text{Size}(\text{text})$  do
5   if  $\text{text}[i] \neq ' '$  then
6      $\text{Add}(\text{tokens}, \text{text}[i])$ 
7   if  $\text{text}[i] = ' '$  and  $\text{Size}(\text{tokens}) \neq 0$  then
8      $\text{Add}(\text{tokens}, '_')$ 
9      $\text{Add}(\text{corpus}, \text{tokens})$ 
10     $\text{tokens} \leftarrow$  empty list
11     $i \leftarrow i + 1$ 
12 return corpus
```

L'algorithme 7 prend en entrée une liste de mots, où chaque mot est représenté sous forme de liste de symboles, ainsi qu'un dictionnaire des fusions issu de l'apprentissage BPE. Il retourne la paire de symboles à fusionner selon l'ordre défini dans le dictionnaire des fusions.

L'objectif de cet algorithme est d'identifier, dans la liste de mots, la paire de symboles adjacents correspondant à la prochaine fusion à appliquer, c'est-à-dire celle ayant la priorité la plus élevée selon l'ordre des fusions appris.

Pour cela, l'algorithme parcourt chaque liste de symboles de la liste de mots et examine toutes les paires de symboles consécutifs. Chaque paire candidate est recherchée dans le dictionnaire des fusions afin de vérifier si elle a été apprise lors de l'entraînement BPE.

Si la paire existe dans le dictionnaire des fusions, son score (correspondant à son ordre de fusion) est récupéré. La paire retenue est celle ayant le score le plus petit, car elle correspond à la fusion la plus ancienne et donc la plus prioritaire.

L'algorithme met ainsi à jour progressivement la paire sélectionnée en comparant les scores de toutes les paires rencontrées dans la liste de mots.

À la fin du parcours de toutes les listes, la paire sélectionnée est retournée comme la prochaine paire à fusionner dans le processus de tokenisation BPE.

Algorithm 7: FindPairToMerge

Input: corpus, mergesDictionary**Output:** pair

```
1  $i \leftarrow 1$ ;  
2  $bestScore \leftarrow +\infty$ ;  
3  $pair \leftarrow \text{null}$ ;  
4 while  $i \leq \text{Size}(\text{corpus})$  do  
5    $tokens \leftarrow \text{Corpus}[i]$ ;  
6    $j \leftarrow 1$ ;  
7   while  $j < \text{Size}(tokens)$  do  
8      $symbol \leftarrow tokens[j] + tokens[j + 1]$ ;  
9     if  $symbol \in \text{mergesDictionary}$  then  
10       $score \leftarrow \text{mergesDictionary}[symbol]$ ;  
11      if  $score < bestScore$  then  
12         $bestScore \leftarrow score$ ;  
13         $pair \leftarrow [tokens[j], tokens[j + 1]]$ ;  
14      $j \leftarrow j + 1$ ;  
15    $i \leftarrow i + 1$ ;  
16 return  $pair$ 
```

2.2.2 Implémentation en Pharo

Nous allons maintenant présenter l'implémentation de l'algorithme de tokenisation BPE en Pharo. La version présentée dans ce chapitre est une version naïve, qui sera ensuite optimisée dans le chapitre suivant. Cette implémentation est disponible sur ce lien : <https://github.com/Ducasse/BytePairEncoder/blob/master/src/BytePairEncoder/BytePairEncoder.class.st>.

Création de classe

Une classe a été créée afin de regrouper les variables et les méthodes nécessaires à l'implémentation du BPE. Elle permet de stocker les données du modèle, telles que le vocabulaire et les règles de fusion, ainsi que les opérations utilisées durant l'entraînement et la tokenisation.

Une classe `BytePairEncoder` a ainsi été créée avec les variables d'instance suivantes : `words`, `vocabulary`, `pairs` et `merges`.

```
Class {  
  #name : 'BytePairEncoder',  
  #superclass : 'Object',  
  #instVars : [  
    'words',  
    'pairs',  
    'vocabulary',  
    'merges'
```

```
],
#category : 'BytePairEncoder',
#package : 'BytePairEncoder'
}
```

Structures de données utilisées

- Une instance de la classe **Set** est utilisée pour représenter le vocabulaire, constitué des caractères initiaux du corpus ainsi que des symboles issus des fusions successives.
- Chaque mot du corpus est représenté sous forme de séquence ordonnée de caractères à l'aide d'une instance de la classe **OrderedCollection**, afin de préserver l'ordre des symboles.
- Une instance de la classe **Bag** est utilisée pour stocker les mots du corpus et leurs fréquences d'occurrence, chaque mot étant représenté comme une séquence de caractères.
- Une instance de la classe **Bag** est également utilisée pour comptabiliser les paires consécutives de caractères ou de sous-mots ainsi que leurs occurrences dans le corpus.
- Une instance de la classe **OrderedCollection** est utilisée pour conserver l'historique des fusions, en enregistrant chaque paire fusionnée ainsi que le symbole résultant.

Remarque : La classe **Bag** est utilisée car elle permet d'associer automatiquement les occurrences à chaque élément lors de leur insertion. Il suffit d'ajouter les éléments pour que leurs fréquences soient mises à jour : elle facilite le comptage des fréquences de chaque élément. De plus, elle fournit des opérations utiles comme la récupération des éléments les plus fréquents. L'accès aux éléments et à leurs occurrences est également simple et efficace.

Initialisation des variables d'instance

La méthode `initialize` permet d'initialiser les variables d'instance : `words`, `vocabulary` et `merges`. La variable `pairs` n'est pas initialisée dans cette méthode, car elle est recalculée à chaque étape de l'entraînement.

```
{ #category : 'initialization' }
BytePairEncoder >> initialize [

    super initialize.
    words := Bag new.
    "We use a bag so that we can have the occurrences at the world level."

    vocabulary := Set new.
    merges := OrderedCollection new
]
```

La structure de données **Bag** est utilisée pour `words` afin de stocker les mots du corpus avec leurs fréquences. Cela facilite le comptage des occurrences et permet d'éviter de répéter plusieurs fois les mêmes traitements sur des mots identiques.

`vocabulary` sert à stocker les symboles appris durant l'entraînement. Il sera ensuite utilisé par le modèle de langage. Au début, il est initialisé comme une instance de `Set`, puis les caractères présents dans le corpus y sont ajoutés, y compris le symbole spécial `"_"` indiquant la fin d'un mot. Ensuite, à chaque étape de l'entraînement, le nouveau symbole obtenu par la fusion de la paire la plus fréquente est ajouté au vocabulaire.

`merges` est une instance de `OrderedCollection`, utilisée pour stocker les règles de fusion apprises durant l'entraînement. L'ordre des fusions est important, car ces règles devront être réappliquées dans le même ordre lors de la tokenisation de nouveaux textes.

Prétraitement du texte

Le texte est d'abord découpé en mots, puis chaque mot est transformé en une collection ordonnée de caractères grâce à la méthode `textToWords` :

```
{ #category : 'compute' }
BytePairEncoder >> textToWords: aText [
    "Split the text into words. Then turn a word as a collection of strings."

    | rawWords |
    rawWords := aText substrings: {
        Character space.
        Character cr.
        Character tab . $. . $? . $! . $: . $; }.
    ^ (rawWords collect: [ :each |
        | col |
        col := OrderedCollection new: each size + 1.
        each do: [ :aChar | col add: aChar asString ].
        col add: '_'.
        col ])
]
```

Cette méthode sera ensuite utilisée par `prepareWordsFromText` : pour ajouter les mots transformés dans la variable `words` :

```
{ #category : 'compute' }
BytePairEncoder >> prepareWordsFromText: aText [
    "Split the text into words. Then build the corpus of words by representing a word as a

    words addAll: (self textToWords: aText)
]
```

Ainsi, chaque mot est représenté comme une collection ordonnée de chaînes de caractères. Par exemple, le mot `"low"` devient `#('l' 'o' 'w' '_')`. Cette représentation facilite les opérations de fusion durant l'entraînement du BPE.

Construction du vocabulaire

Le vocabulaire est construit grâce à la méthode `computeVocabulary` en parcourant chaque élément de `words` :

```
{ #category : 'vocabulary' }
BytePairEncoder >> computeVocabulary [
    "we do not add characters but string because in the merge phase we will have sequences

    words do: [ :each |
    each do: [ :aChar |
    self addToVocabulary: aChar asString ]].
]
```

Chaque caractère est converti en chaîne de caractères avant son ajout au vocabulaire, car les opérations de fusion en Pharo s'effectuent sur des chaînes de caractères.

Calcul de la fréquence de chaque paire existante du corpus

Les paires de symboles adjacents sont calculées grâce à la méthode `computePairs` en parcourant chaque élément de `words` :

```
BytePairEncoder >> computePairs [

    pairs := Bag new.
    words associationsDo: [ :association |
    | word |
    word := association key.
    word overlappingPairsWithIndexDo: [ :a :b :index |
    pairs add: { a . b } withOccurrences: association value ]
]
```

La méthode `overlappingPairsWithIndexDo:` parcourt une collection par paires d'éléments adjacents et exécute un bloc pour chaque paire et l'indice du premier élément de la paire.

Initialisation du pipeline BPE

Les méthodes `prepareWordsFromText:`, `computeVocabulary` et `computePairs` sont regroupées dans `fromText:` afin de préparer le texte avant l'entraînement, d'initialiser le vocabulaire et de calculer les fréquences initiales des paires. Cette méthode reçoit un texte en entrée.

```
{ #category : 'api' }
BytePairEncoder >> fromText: aString [

    self prepareWordsFromText: aString.
    self computeVocabulary.
    self computePairs.
]
```

Entraînement du BPE

Après cette phase de préparation, l'entraînement du BPE consiste à :

- déterminer la paire la plus fréquente ;
- fusionner cette paire ;
- ajouter le nouveau symbole au vocabulaire ;
- enregistrer la règle de fusion ;
- mettre à jour le corpus ;
- recalculer les fréquences des paires.

Ces opérations sont regroupées dans la méthode `mergeOneStep` :

```
{ #category : 'merges' }
BytePairEncoder >> mergeOneStep [

    | newVocabularyItem pair |
    pair := pairs sortedCounts first value.
    newVocabularyItem := pair first, pair second.
    self addToVocabulary: newVocabularyItem.
    self addToMergesPair: pair mergedInto: newVocabularyItem.
    self updateCorpusFrom: pair to: newVocabularyItem.
    self computePairs.
]
```

Cette méthode est exécutée plusieurs fois jusqu'à atteindre la taille de vocabulaire souhaitée ou jusqu'à ce qu'il n'existe plus de paires à fusionner.

La mise à jour du corpus est réalisée grâce à la méthode `updateCorpusFrom:to:` :

```
{ #category : 'merges' }
BytePairEncoder >> updateCorpusFrom: pair to: newVocabularyItem [

    | bag |
    bag := Bag new.
    words keys copy do:
    [ :word |
        bag
        add: (word copyReplaceAll: pair with: {newVocabularyItem})
        withOccurrences: (words occurrencesOf: word)
    ].
    words := bag
]
```

Cette méthode remplace toutes les occurrences de la paire donnée par le nouveau symbole fusionné dans chaque mot du corpus.

Après chaque mise à jour du corpus, les fréquences des paires sont recalculées en exécutant de nouveau `computePairs`.

Application des règles de fusion apprises durant l'entraînement

Une fois l'entraînement terminé, les règles de fusion stockées dans `merges` peuvent être appliquées à un nouveau texte grâce à la méthode `encode` :

```
{ #category : 'api' }
BytePairEncoder >> encode: aString [
    "this encoding does not considering overlapping merges.

    if we have
    a,b -> 1
    b,c -> 2
    and a string
    abc
    it will produce
    1c
    and not 12
    "

    | wrds |
    wrds := (self textToWords: aString) flatCollect: [ :each | each ].
    wrds := wrds asOrderedCollection.

    merges do: [ :aMerge |
        | key first second rempl i |
        key := aMerge key.
        first := key first.
        second := key second.
        rempl := aMerge value.
        i := 1.
        [ i < wrds size ] whileTrue: [
            (((wrds at: i) = first)
            and: [ (wrds at: i + 1) = second ])
            ifTrue: [ wrds removeAt: i; at: i put: rempl.
                i := i + 1 ].
            i := i + 1 ].
        ].
        ^ wrds
    ]
}
```

Avant d'appliquer les règles de fusion, le texte est prétraité avec `textToWords`: afin d'être

transformé en collections ordonnées de symboles. La méthode `flatCollect` : est ensuite utilisée pour aplatir ces collections et obtenir une unique séquence de symboles représentant le texte.

La méthode `flatCollect` : n'a pas été utilisée durant l'entraînement afin d'éviter la création de paires composées de symboles appartenant à des mots différents. Cela permet de préserver les frontières entre les mots et d'empêcher la fusion de symboles provenant de mots distincts.

Chapitre 3

WordPiece

3.1 Présentation de WordPiece

WordPiece est un algorithme de tokenisation en sous-mots. Il découpe les mots en caractères ou en sous-mots selon les vocabulaires obtenus lors de l'entraînement de WordPiece à partir d'un corpus. Cet algorithme a été développé initialement par Google et utilisé pour le pré-entraînement du modèle BERT. L'algorithme présenté ici est inspiré des explications proposées dans le cours de Hugging Face.

3.1.1 Comprendre WordPiece

L'idée principale de WordPiece est de sélectionner, à chaque itération, la paire de symboles ayant le score le plus élevé. Ce score est calculé à partir de la formule suivante :

$$score = \frac{\text{fréquence de la paire}}{\text{fréquence du premier élément de la paire} * \text{fréquence du second élément de la paire}}$$

La paire sélectionnée est ensuite fusionnée pour former un nouveau symbole ajouté au vocabulaire. Le vocabulaire obtenu après les itérations est ensuite utilisé pour encoder du texte.

3.1.2 Logique de fonctionnement de WordPiece

Le fonctionnement de WordPiece peut être résumé par les étapes suivantes :

1. Prétraiter le texte : chaque mot est transformé en une collection ordonnée de caractères. Les caractères situés au milieu d'un mot sont préfixés par le symbole spécial "##".
2. Initialiser le vocabulaire : le vocabulaire initial est constitué des caractères présents dans le corpus. Les caractères apparaissant au milieu des mots sont ajoutés avec le préfixe "##".
3. Calculer la fréquence de chaque symbole (caractère ou sous-mot) du corpus.
4. Calculer la fréquence de chaque paire de symboles consécutifs.
5. Calculer le score de chaque paire.

6. Sélectionner la paire ayant le score maximal.
7. Fusionner cette paire : si le second élément de la paire commence par "##", ce préfixe est supprimé avant la fusion. Le résultat de la fusion devient un nouveau symbole.
8. Ajouter le nouveau symbole au vocabulaire.
9. Mettre à jour le corpus en remplaçant chaque occurrence de la paire par sa fusion.
10. Répéter les étapes précédentes jusqu'à atteindre la taille de vocabulaire souhaitée ou jusqu'à ce qu'il n'existe plus de paires à fusionner.
11. Le vocabulaire appris est utilisé pour encoder du texte. Pour chaque mot, WordPiece recherche le plus long préfixe présent dans le vocabulaire. Le reste du mot est ensuite traité de la même manière jusqu'à ce que le mot soit entièrement tokenisé.

3.1.3 Illustration à travers un exemple

```
|w| w := WordPiece new.
w fromText: 'FRED FED TED BREAD, AND TED FED FRED BREAD'.
```

Vérification du corpus

Après avoir tokenisé notre texte, nous obtenons ce changement :

```
corpus
#('T' '##E' '##D') 2
#('B' '##R' '##E' '##A' '##D') 2 #('F' '##E' '##D') 2
#('F' '##R' '##E' '##D') 2
#('A' '##N' '##D') 1
```

Lorsque nous examinons le système, nous obtenons les éléments suivants :

- Un vocabulaire : '##D' 'F' '##A' '##R' '##,' 'T' 'B' '##N' '##E' 'A'
- Un corpus : comme présenté ci-dessus
- Une liste de paires consécutives avec leurs
- Une liste de paires consecutives avec leurs scores

```
voc
'##D' 'F' '##A' '##R' 'T' 'B' '##N' '##E' 'A'
corpus
#('T' '##E' '##D') 2
#('B' '##R' '##E' '##A' '##D') 2
#('F' '##E' '##D') 2
#('F' '##R' '##E' '##D') 2
#('A' '##N' '##D') 1
```

pairs

```
#('##E' '##A') 2
#('F' '##E') 2
#('A' '##N') 1
#('F' '##R') 2
#('T' '##E') 2
#('##E' '##D') 6
#('##N' '##D') 1
#('B' '##R') 2
#('##A' '##D') 2
#('##R' '##E') 4
```

letter frequency

```
'##D' 9
'##E' 8
'##R' 4
'##A' 2
'F' 4
'B' 2
'A' 1
'##N' 1
'T' 2
```

scores

```
#('##N' '##D') (1/9)
#('F' '##R') (1/8)
#('##R' '##E') (1/8)
#('##E' '##D') (1/12)
#('B' '##R') (1/4)
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)
#('A' '##N') 1
```

Nous observons que la paire ayant le score maximale est : $\#('A', '##N')$. Cette paire sera la seule qui sera fusionnée. Après sa fusion cette paire donne le symbole 'AN'.

Itération 1

Examinons la situation après la première fusion.

w mergeOneStep

Nous voyons que la paire la plus fréquente $\#('A' \ '##N')$ a été fusionnée. Cela signifie que :

- Sa fusion a été ajoutée au vocabulaire : nous pouvons maintenant voir 'AN' comme un nouvel élément.
- Le corpus a été mis à jour afin de refléter la présence de nouvel élément du vocabulaire : $\#('A' \ '##N' \ '##D')$ est désormais représenté par $\#('AN' \ '##D')$;
- toutes les paires ont été recalculées à partir de la version mise à jour du corpus. Ainsi, la paire la plus fréquente devient maintenant $\#('B' \ '##R')$.

voc

'##D' 'F' '##A' '##R' 'T' 'B' '##N' 'AN' '##E' 'A'

corpus

$\#('T' \ '##E' \ '##D')$ 2

$\#('B' \ '##R' \ '##E' \ '##A' \ '##D')$ 2

$\#('F' \ '##E' \ '##D')$ 2

$\#('F' \ '##R' \ '##E' \ '##D')$ 2

$\#('AN' \ '##D')$ 1

pairs

$\#('##E' \ '##A')$ 2

$\#('F' \ '##E')$ 2

$\#('F' \ '##R')$ 2

$\#('T' \ '##E')$ 2

$\#('##E' \ '##D')$ 6

$\#('AN' \ '##D')$ 1

$\#('B' \ '##R')$ 2

$\#('##A' \ '##D')$ 2

$\#('##R' \ '##E')$ 4

letter frequency

'##D' 9

'AN' 1

'##E' 8

'##R' 4

'##A' 2

'F' 4

'B' 2

'T' 2

scores

$\#('F' \ '##R')$ (1/8)

$\#('##R' \ '##E')$ (1/8)

$\#('##E' \ '##D')$ (1/12)

$\#('B' \ '##R')$ (1/4)

$\#('AN' \ '##D')$ (1/9)

```
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)
```

Itération 2

L'algorithme répète le même processus. Ainsi, la fusion 'BR' est ajoutée au vocabulaire. Le corpus et les paires sont ensuite mis à jour.

voc

```
'BR' '##D' 'F' '##A' '##R' 'T' 'B' '##N' 'AN' '##E' 'A'
```

Corpus

```
#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('F' '##R' '##E' '##D') 2
#('AN' '##D') 1
```

pairs

```
#('##E' '##A') 2
#('F' '##E') 2
#('F' '##R') 2
#('T' '##E') 2
#('##E' '##D') 6
#('AN' '##D') 1
#('##D' '##,') 1
#('BR' '##E') 2
#('##A' '##D') 2
#('##R' '##E') 2
```

letter frequency

```
'##D' 9
'AN' 1
'##E' 8
'##A' 2
'BR' 2
'F' 4
'##R' 2
'T' 2
```

scores

```
#('F' '##R') (1/4)
#('##R' '##E') (1/8)
#('BR' '##E') (1/8)
```

```

#('##E' '##D') (1/12)
#('AN' '##D') (1/9)
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)

```

Itération 3

Voici l'itération suivante. Le resultat de la fusion 'FR' de la paire #('F' '##R') a été ajoutée à la vocabulaire.

voc

```
'BR' '##D' 'F' '##A' '##R' 'FR' 'T' 'B' '##N' 'AN' '##E' 'A'
```

corpus

```

#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('AN' '##D') 1
#('FR' '##E' '##D') 2

```

Pairs

```

#('##E' '##A') 2
#('F' '##E') 2
#('FR' '##E') 2
#('T' '##E') 2
#('##E' '##D') 6
#('AN' '##D') 1
#('BR' '##E') 2
#('##A' '##D') 2

```

letter frequency

```

'##D' 9
'AN' 1
'##E' 8
'##A' 2
'BR' 2
'F' 2
'FR' 2
'T' 2

```

scores

```

#('FR' '##E') (1/8)
#('AN' '##D') (1/9)
#('##A' '##D') (1/9)

```

```

#('##E' '##A') (1/8)
#('T' '##E') (1/8)
#('F' '##E') (1/8)
#('##E' '##D') (1/12)
#('BR' '##E') (1/8)

```

Itération 4

Voici l'itération suivante. La fusion 'FRE' de #('FR' '##E') est ajoutée à la liste des fusions. Elle apparaît en première position uniquement à cause du tri utilisé pour l'affichage, qui est le même que celui utilisé pour sélectionner la paire la plus fréquente. Cela n'a pas d'impact sur l'algorithme : d'autres paires de même fréquence auraient pu être choisies.

voc

```
'BR' '##D' 'F' '##A' '##R' 'FR' 'T' 'B' 'FRE' '##N' 'AN' '##E' 'A'
```

corpus

```

#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('AN' '##D') 1
#('FRE' '##D') 2

```

pairs

```

#('##E' '##A') 2
#('F' '##E') 2
#('T' '##E') 2
#('##E' '##D') 4
#('FRE' '##D') 2
#('AN' '##D') 1
#('BR' '##E') 2
#('##A' '##D') 2

```

letter frequency

```

'##D' 9
'AN' 1
'##E' 6
'##A' 2
'BR' 2
'F' 2
'FRE' 2
'T' 2

```

scores

```

#('BR' '##E') (1/6)
#('AN' '##D') (1/9)

```

```
#('##A' '##D') (1/9)
#('##E' '##A') (1/6)
#('T' '##E') (1/6)
#('F' '##E') (1/6)
#('##E' '##D') (2/27)
#('FRE' '##D') (1/9)
```

Jusqu'à ce qu'a un certain seuil

L'algorithme se déroule comme décrit précédemment jusqu'à ce que le score de chaque paire devienne inférieur à un seuil défini. On obtient alors la situation suivante.

voc

```
'BR' 'BREA' 'TE' 'FRED' '##N' '##E' 'TED' 'FR' 'A' 'AN' 'BRE' 'BREAD' 'FED' 'F' '##A'
```

corpus

```
#('AN' '##D') 1
#('FED') 2
#('FRED') 2
#('BREAD') 2
#('TED') 2
```

pairs

```
#('AN' '##D') 1
```

letter frequency

```
'FRED' 2
'BREAD' 2
'FED' 2
'TED' 2
'AN' 1
'##D' 1
```

scores

```
#('AN' '##D') 1
```

Utilisation du vocabulaire pour encoder du texte

La logique de cette étape est la suivante :

Le texte est d'abord découpé en une collection ordonnée de mots M . Soit T la liste des tokens du texte et t la liste des tokens associés à un mot. Au début, T est vide. Pour chaque mot du texte, t est initialisée à vide.

Supposons que le mot courant à tokeniser soit m .

1. On recherche m dans le vocabulaire.

— Si m appartient au vocabulaire, il est ajouté à t .

- Tous les éléments de t sont ensuite ajoutés à T .
 - Le mot suivant dans M devient alors le nouveau mot courant à tokeniser, et le processus recommence à l'étape 1.
2. Si m n'appartient pas au vocabulaire, on recherche le plus long préfixe de m présent dans le vocabulaire.
- (a) Si un préfixe valide est trouvé :
- ce préfixe est ajouté à t ;
 - le reste du mot est préfixé par le symbole spécial **##** ;
 - ce reste devient alors le nouveau mot courant m à tokeniser.
- (b) Si aucun préfixe valide n'est trouvé :
- le symbole spécial **<UNK>** est ajouté à T ;
 - la tokenisation du mot courant est arrêtée ;
 - le processus passe au mot suivant dans M .
3. Ce processus est répété jusqu'à ce que le mot soit entièrement tokenisé ou qu'aucun préfixe du mot courant ne soit trouvé dans le vocabulaire.

Exemple : imaginons que nous voulons encoder le texte 'FRED EATS BREAD'. Lors de la phase de découpage du texte en séquences de mots, nous obtenons :

#('FRED' 'EATS' 'BREAD')

Ensuite, nous demandons à l'algorithme d'encoder le texte et nous obtenons la liste de tokens suivante :

```
w encode: 'FRED EATS BREAD'
> an OrderedCollection('FRED' '<UNK>' 'BREAD')
```

Les mots FRED et BREAD sont directement retournés par WordPiece, car ils appartiennent au vocabulaire appris durant l'entraînement.

En revanche, le mot EATS est remplacé par le symbole spécial **<UNK>**, car aucun préfixe valide de ce mot (EATS, EAT, EA ou E) n'existe dans le vocabulaire.

Observation de l'algorithme

WordPiece et BPE réalisent des opérations similaires durant l'entraînement. Cependant, contrairement à BPE qui sélectionne la paire la plus fréquente, WordPiece calcule un score pour chaque paire de symboles avant de choisir celle à fusionner. Cela nécessite également le calcul de la fréquence de chaque symbole du corpus.

Comme BPE, WordPiece ne fusionne qu'une seule paire à chaque itération. Après chaque fusion, le corpus est mis à jour, puis les fréquences des symboles, les fréquences des paires et les scores des paires sont recalculés.

Le prétraitement du texte et la phase d'encodage diffèrent également entre BPE et WordPiece. WordPiece utilise notamment le symbole spécial `##` pour représenter les symboles (caractères ou sous-mots) situés au milieu d'un mot.

Lors de l'encodage d'un mot, WordPiece n'utilise pas une liste de règles de fusion pour encoder un texte. Il utilise principalement le vocabulaire appris durant l'entraînement pour tokeniser les mots. Il recherche itérativement le plus long préfixe du mot présent dans le vocabulaire. Si aucun préfixe valide n'est trouvé pour le mot courant ou pour le reste du mot à tokeniser, le mot entier est remplacé par le symbole spécial `<UNK>`.

3.2 Implémentation

L'objectif de ce chapitre est d'implémenter WordPiece en Pharo. Le détail sur le fonctionnement de WordPiece est décrit dans la section 3.1.2.

Nous avons vu précédemment que WordPiece et BPE effectuent des opérations similaires. Elles se différencient principalement par :

- le calcul des fréquences des symboles,
- le calcul des scores des paires,
- le prétraitement du texte,
- la fusion des symboles,
- et l'encodage.

Nous allons définir les algorithmes en pseudo-code pour :

- le prétraitement,
- le calcul des fréquences des symboles,
- le calcul des scores des paires,
- la fusion des symboles,
- l'entraînement,
- et l'encodage.

3.2.1 Algorithmes pseudo-code

L'algorithme 9 prend en entrée un texte et produit en sortie un dictionnaire dont les clés représentent les mots du corpus et les valeurs représentent leurs fréquences d'apparition dans le texte.

Cet algorithme effectue d'abord un prétraitement des mots en les découpant en une liste de caractères. Le premier caractère est conservé tel quel, tandis que les caractères suivants sont précédés du symbole spécial `'##'`. Ce symbole permet d'indiquer qu'un caractère ou un sous-mot ne se situe pas au début du mot, conformément au principe utilisé par WordPiece.

Chaque mot prétraité est ensuite ajouté au dictionnaire du corpus. Avant l'ajout, l'algorithme vérifie si le mot existe déjà dans le dictionnaire :

- s'il existe, sa fréquence est incrémentée de 1 ;
- sinon, sa fréquence est initialisée à 1.

Algorithm 8: Preprocess

Input: text**Output:** corpus

```
1 corpus  $\leftarrow$  empty dictionary;
2 word  $\leftarrow$  empty list;
3  $i \leftarrow 1$ ;
4 while  $i \leq \text{Length}(\text{text})$  do
5   if  $\text{text}[i] = " "$  then
6     if  $\text{Length}(\text{word}) > 0$  then
7       if  $\text{word} \in \text{corpus}$  then
8          $\text{corpus}[\text{word}] \leftarrow \text{corpus}[\text{word}] + 1$ 
9       else
10         $\text{corpus}[\text{word}] \leftarrow 1$ 
11      word  $\leftarrow$  empty list
12   else
13     if  $\text{Length}(\text{word}) = 0$  then
14       Add(word, text[i])
15     else
16       Add(word, '###' + text[i])
17    $i \leftarrow i + 1$ 
18 if  $\text{Length}(\text{word}) > 0$  then
19   if  $\text{word} \in \text{corpus}$  then
20      $\text{corpus}[\text{word}] \leftarrow \text{corpus}[\text{word}] + 1$ 
21   else
22      $\text{corpus}[\text{word}] \leftarrow 1$ 
23 return corpus
```

L’**algorithme 10** prend en entrée un dictionnaire d_e obtenu après l’application de l’**algorithme 9** et produit en sortie un dictionnaire d_s .

Dans le dictionnaire d_s , les clés représentent les symboles présents dans le corpus prétraité par l’**algorithme 9**. Ces symboles peuvent être des caractères ou des sous-mots obtenus lors des différentes étapes de fusion de WordPiece. Les valeurs associées à ces clés correspondent à leurs fréquences totales d’apparition dans le corpus.

L’algorithme parcourt chaque élément de d_e , où chaque entrée est constituée :

- d’une liste de symboles représentant un mot prétraité ;
- et de la fréquence d’apparition de ce mot dans le texte.

Pour chaque symbole contenu dans une liste de symboles, l’algorithme vérifie si ce symbole existe déjà dans le dictionnaire d_s :

- si le symbole existe déjà, sa fréquence est incrémentée par la fréquence de la liste de symboles qui le contient dans d_e ;
- sinon, sa fréquence est initialisée avec cette même valeur.

Ainsi, le dictionnaire d_s contient la fréquence globale de chaque symbole dans l'ensemble du corpus. Ces fréquences sont ensuite utilisées lors du calcul des scores des paires de symboles dans l'algorithme WordPiece.

Algorithm 9: ComputeSymbolFrequencies

Input: corpus

Output: symbolFrequencies

```

1 symbolFrequencies  $\leftarrow$  empty dictionary;
2 foreach  $\underline{word, frequency \in \text{corpus}}$  do
3    $i \leftarrow 1$ 
4   while  $i \leq \text{Length}(word)$  do
5      $symbol \leftarrow word[i]$ 
6     if  $\underline{symbol \in \text{symbolFrequencies}}$  then
7        $\text{symbolFrequencies}[symbol] \leftarrow \text{symbolFrequencies}[symbol] + frequency$ 
8     else
9        $\text{symbolFrequencies}[symbol] \leftarrow 1$ 
10     $i \leftarrow i + 1$ 
11 return  $\underline{\text{symbolFrequencies}}$ 

```

L'algorithme 11 calcule le score de chaque paire de symboles présente dans le corpus. Il prend en entrée deux dictionnaires :

- un dictionnaire contenant les symboles ainsi que leurs fréquences dans le corpus ;
- et un dictionnaire contenant les paires de symboles ainsi que leurs fréquences d'apparition dans le corpus.

L'algorithme produit en sortie un dictionnaire dont les clés représentent les paires de symboles et les valeurs représentent leurs scores. Pour chaque paire de symboles, le score est calculé à partir de la fréquence de la paire ainsi que des fréquences individuelles des deux symboles qui la composent. Les scores obtenus sont ensuite utilisés pour sélectionner la meilleure paire de symboles à fusionner lors de l'étape d'apprentissage de WordPiece. La paire possédant le score le plus élevé est considérée comme la plus pertinente à fusionner afin de créer un nouveau sous-mot.

Algorithm 10: ComputeScores

Input: symbolFrequencies, pairFrequencies

Output: scores

```
1 scores  $\leftarrow$  empty dictionary;
2 foreach pair, frequency  $\in$  pairFrequencies do
3   | symbol1  $\leftarrow$  pair[1];
4   | symbol2  $\leftarrow$  pair[2];
5   | scores[pair]  $\leftarrow \frac{frequency}{symbolFrequencies[symbol1] * symbolFrequencies[symbol2]}$ 
6 return scores
```

L’algorithme 12 prend en entrée une paire de symboles et retourne un nouveau symbole obtenu par la fusion des deux éléments composant cette paire.

Avant d’effectuer la fusion, l’algorithme vérifie si le deuxième symbole de la paire est préfixé par le symbole spécial ‘##’. Si ce préfixe est présent, il est supprimé.

Après cette étape, les deux symboles sont concaténés pour former un nouveau symbole. Ce nouveau symbole correspond au sous-mot créé lors du processus d’apprentissage de WordPiece et sera ajouté au vocabulaire.

Algorithm 11: MergePair

Input: pair

Output: newSymbol

```
1 symbol1  $\leftarrow$  pair[1];
2 symbol2  $\leftarrow$  pair[2];
3 if symbol2 starts with ‘##’ then
4   | symbol2  $\leftarrow$  Substring(symbol2, 3, Length(symbol2))
5 newSymbol  $\leftarrow$  symbol1 + symbol2
6 return newSymbol
```

L’algorithme 13 permet l’entraînement de WordPiece sur un texte. Il prend en entrée un texte ainsi qu’une taille de vocabulaire souhaitée ou un seuil appliqué au score des paires de symboles, utilisé comme condition d’arrêt de l’apprentissage.

L’algorithme produit en sortie un vocabulaire contenant :

- les symboles initiaux obtenus après le prétraitement du texte ;
- ainsi que les nouveaux symboles créés au cours des différentes opérations de fusion.

L’apprentissage débute par le prétraitement du texte à l’aide de la fonction `Preprocess()` (algorithme 9). Cette étape transforme chaque mot du corpus en une liste de symboles. Ensuite, la fonction `InitializeVocabulary()` initialise le vocabulaire avec l’ensemble des symboles présents dans le corpus prétraité.

À chaque itération, et jusqu’à ce que la taille souhaitée du vocabulaire soit atteinte ou que le score maximal des paires devienne inférieur au seuil fixé, l’algorithme effectue les opérations suivantes :

- calcul des fréquences des symboles à l'aide de la fonction `ComputeSymbolFrequencies()` (**algorithme 10**);
- calcul des fréquences des paires de symboles à l'aide de la fonction `ComputePairFrequencies()` (**algorithme 3**);
- calcul du score de chaque paire grâce à la fonction `ComputeScores()` (**algorithme 11**).
- après le calcul des scores, l'algorithme sélectionne la paire possédant le score maximal. Les deux symboles composant cette paire sont ensuite fusionnés à l'aide de la fonction `MergePair()` (**algorithme 12**) afin de produire un nouveau symbole.
- le nouveau symbole obtenu est ajouté au vocabulaire.
- enfin, le corpus est mis à jour grâce à la fonction `UpdateCorpus()` (**algorithme 4**), qui remplace toutes les occurrences de la paire sélectionnée par le symbole fusionné correspondant.

Ce processus itératif permet de construire progressivement un vocabulaire composé de sous-mots fréquents et représentatifs du corpus d'apprentissage.

Algorithm 12: TrainWordPiece

Input: text, vocabularySize

Output: vocabulary

```

1 corpus ← Preprocess(text);
2 vocabulary ← InitializeVocabulary[corpus];
3 i ← vocabularySize - Length(vocabulary)
4 if i ≤ 0 then
5   return vocabulary
6 while i > 0 do
7   symbolFrequencies ← ComputeSymbolFrequencies(corpus);
8   pairFrequencies ← ComputePairs(corpus);
9   scores ← ComputeScores(symbolFrequencies, pairFrequencies);
10  bestPair ← Max(scores);
11  newSymbol ← MergePair(bestPair);
12  Add(vocabulary, newSymbol);
13  corpus ← UpdateCorpus(corpus, bestPair, newSymbol);
14  i ← i - 1
15 return vocabulary
```

L'**algorithme 14** prend en entrée un texte ainsi que le vocabulaire obtenu lors de l'entraînement de WordPiece et retourne une liste de symboles représentant le texte encodé.

Cet algorithme permet de représenter le texte sous la forme d'une suite de sous-mots appartenant au vocabulaire appris pendant la phase d'entraînement.

Le prétraitement effectué durant l'encodage diffère de celui réalisé pendant l'apprentissage. En effet, dans cette phase, le texte n'est pas transformé en listes de caractères préfixés par le symbole spécial '#'. Le texte est simplement découpé en mots à l'aide de la fonction `Split()`,

qui utilise l'espace comme séparateur. Cette fonction retourne alors une liste contenant les différents mots du texte.

Pour chaque mot obtenu, l'algorithme applique ensuite la fonction `EncodeWord()` (**algorithme 15**), qui retourne une liste de sous-mots correspondant au mot encodé. Cette fonction permet de décomposer chaque mot en une suite de sous-mots existants dans le vocabulaire `WordPiece`. Le découpage est réalisé en recherchant successivement les plus longs sous-mots appartenant au vocabulaire.

Une fois le mot encodé, tous les sous-mots obtenus sont ajoutés à la liste finale des symboles représentant le texte. Ce processus est répété pour chacun des mots du texte.

Après l'encodage de tous les mots, l'algorithme retourne la liste complète des sous-mots composant le texte. Cet algorithme permet ainsi de transformer le texte donné en entrée en une représentation basée sur les sous-mots du vocabulaire appris lors de l'entraînement.

Algorithm 13: EncodeText

Input: text, vocabulary

Output: tokens

```

1 tokens  $\leftarrow$  empty list;
2 words  $\leftarrow$  Split(text, ' ');
3 i  $\leftarrow$  1
4 while i  $\leq$  Length(words) do
5   subTokens  $\leftarrow$  EncodeWord(words[i], vocabulary);
6   AddAll(tokens, subTokens);
7   i  $\leftarrow$  i + 1
8 return tokens
```

L'**algorithme 15** prend en entrée un mot ainsi que le vocabulaire obtenu après l'entraînement de `WordPiece`. Il retourne une liste de symboles représentant le mot encodé.

Cet algorithme permet de découper le mot donné en entrée en une suite de sous-mots appartenant au vocabulaire appris lors de l'entraînement. Au début de l'exécution, la liste de symboles est initialisée à une liste vide.

L'algorithme effectue ensuite une ou plusieurs itérations afin de segmenter progressivement le mot en sous-mots. À chaque itération, il recherche le plus long préfixe du mot courant appartenant au vocabulaire, conformément au principe du "longest match first" utilisé par `WordPiece`.

Si aucun préfixe du mot n'existe dans le vocabulaire, l'algorithme retourne le symbole spécial `['[UNK]']`, indiquant que le mot ne peut pas être représenté à l'aide du vocabulaire appris.

Lorsqu'un préfixe valide est trouvé, celui-ci est ajouté à la liste des symboles. La partie correspondante est ensuite retirée du mot à l'aide de la fonction `SubString()`, qui permet d'extraire une sous-chaîne à partir d'une position donnée jusqu'à la fin du mot.

Si le mot restant possède encore des caractères, le préfixe spécial `'###'` lui est ajouté afin d'indiquer que les prochains sous-mots ne correspondent pas à un début de mot. Le même

processus est ensuite répété sur le mot restant jusqu'à son encodage complet.

L'algorithme s'arrête lorsque :

- soit aucun préfixe valide n'est trouvé dans le vocabulaire ;
- soit la longueur du mot restant devient égale à 0, ce qui signifie que le mot a été entièrement encodé.

Le résultat final est une liste de sous-mots représentant le mot d'entrée selon le vocabulaire WordPiece.

Algorithm 14: EncodeWord

Input: word, vocabulary

Output: tokens

```
1 tokens ← empty list;
2 while Length(word) > 0 do
3   i ← Length(word);
4   subWord ← SubString(word, 1, i);
5   while i > 0 and subWord not in vocabulary do
6     i ← i - 1;
7     subWord ← SubString(subWord, 1, i);
8   if i = 0 then
9     return ['[UNK]']
10  Add(tokens, subWord);
11  word ← SubString(word, i + 1, Length(word));
12  if Length(word > 0) then
13    word ← '###' + word
14 return tokens
```

3.2.2 Implémentation en Pharo

Dans cette section, nous présentons une implémentation de WordPiece en Pharo permettant le prétraitement du texte, l'entraînement ainsi que l'encodage d'un texte. L'implémentation décrite dans ce chapitre correspond à une version naïve et est disponible à l'adresse suivante : <https://github.com/Ducasse/BytePairEncoder/blob/master/src/BytePairEncoder/WordPiece.class.st>. Une version optimisée de cette implémentation sera présentée dans le chapitre suivant.

Création de la classe

WordPiece peut être considéré comme une extension de l'algorithme BPE et, dans une approche orientée objet, comme une sous-classe de celui-ci. En effet, les deux algorithmes partagent une structure générale commune comprenant le prétraitement du texte, la construction du vocabulaire, le calcul des fréquences des symboles ainsi que le processus itératif de fusion des symboles (section 2.1.3).

Cependant, WordPiece spécialise le comportement de BPE en modifiant certaines étapes clés. Notamment, la sélection des paires à fusionner ne repose plus uniquement sur leur fré-

quence, mais sur un score calculé à partir des fréquences des symboles. De plus, le mécanisme de prétraitement et d'encodage diffère, notamment avec l'utilisation du marqueur '##' pour indiquer qu'un symbole ne correspond pas au début d'un mot, ainsi qu'un encodage réalisé mot par mot.

Dans cette perspective, WordPiece hérite des attributs et des méthodes de BPE, ce qui permet de réutiliser l'ensemble des fonctionnalités de base de cet algorithme. Toutefois, il introduit également des attributs supplémentaires et redéfinit certaines méthodes héritées afin d'adapter le comportement de BPE à ses propres spécificités.

Ainsi, l'algorithme WordPiece contient notamment les variables d'instance `pairScores`, qui stocke les scores associés aux paires de symboles, et `letterFrequency`, qui permet de conserver les fréquences des symboles du corpus. Ces attributs jouent un rôle essentiel dans le calcul des scores utilisés lors de la sélection des paires à fusionner.

```
BytePairEncoder << #WordPiece
  slots: { #pairScores . #letterFrequency };
  package: 'BytePairEncoder'
```

Les méthodes ajoutées dans WordPiece sont les suivantes :

- `computeLetterFrequency`, qui permet de calculer les fréquences des symboles utiles au calcul du score des paires de symboles ;
- `encodeWord:`, qui permet d'encoder individuellement chaque mot d'un texte ;
- `merge:and:`, qui permet de fusionner deux symboles ;
- `findPairToMerge:`, qui permet de rechercher la paire possédant le score maximal dans `pairScores`.

Certaines méthodes héritées de BPE ont également été redéfinies :

- `textToWords:`, afin d'ajouter le symbole spécial '##' aux symboles qui ne correspondent pas au début d'un mot ;
- `computePairs`, afin de calculer, en plus des fréquences, le score associé à chaque paire ;
- `mergeOneStep`, car le processus de sélection et de fusion des symboles a été modifié ;
- `encode:`, car l'encodage est réalisé mot par mot.

Les méthodes héritées de BPE dont le comportement reste inchangé sont directement réutilisées dans WordPiece :

- `prepareWordsFromText:`
- `computeVocabulary`
- `fromText:`
- `addToVocabulary:`
- `updateCorpusFrom:to:`

Structures de données utilisées

- Une instance de la classe **Bag** a été utilisée pour stocker les symboles présents dans le corpus d'entraînement ainsi que leurs fréquences.

- Une instance de la classe `Dictionary` a été utilisée pour stocker les associations entre les paires de symboles et leurs scores.

Remarque : la classe **Bag** a été utilisée car elle facilite le comptage des occurrences des éléments. Lorsqu'un élément est ajouté dans un **Bag**, son nombre d'occurrences est automatiquement mis à jour. Les éléments stockés dans un **Bag** sont associés à leur fréquence d'apparition, ce qui permet de représenter efficacement les symboles ainsi que leurs occurrences dans le corpus.

Prétraitement du texte

La méthode `textToWord:` prend en entrée un texte, puis le découpe en mots. Les ponctuations, les tabulations, les espaces ainsi que les espaces multiples sont utilisés comme séparateurs de mots. Chaque mot est ensuite représenté comme une collection ordonnée de ses caractères. La méthode `textToWord:` ajoute également le symbole '##' aux caractères qui ne correspondent pas au début d'un mot. Enfin, elle retourne une collection ordonnée de ces collections ordonnées de caractères.

```
textToWords: aText
| rawWords |

rawWords := aText substrings: {
Character space.
Character cr.
Character tab .
$. .
$? .
$! .
$: .
$;}.
^ (rawWords collect: [ :each |

| col |
col := OrderedCollection new.

col add: each first asString.

2
to: each size
do: [ :i|

col add: '##',( each at: i ) asString ] .

col ] )
```

Elle sera ensuite utilisée par la méthode `prepareWordsFromText` : afin d'ajouter les éléments de la collection ordonnée qu'elle retourne dans la variable d'instance `words`, dans le but de pouvoir compter les fréquences de ces éléments. La méthode `prepareWordsFromText` : est présentée dans la partie « **Prétraitement du texte** » de la sous-section **2.2.2**.

Comptage des fréquences des symboles dans le texte

La méthode `computeLetterFrequency` permet de calculer la fréquence d'apparition des symboles présents dans le corpus d'entraînement. Pour cela, elle parcourt chaque élément de la variable d'instance `words`, qui contient les mots prétraités sous forme de collections ordonnées de symboles. Chaque symbole rencontré est ensuite ajouté à la variable d'instance `letterFrequency`. Les fréquences obtenues seront ensuite utilisées pour le calcul des scores des paires de symboles lors du processus de fusion de `WordPiece`.

```
computeLetterFrequency
```

```
letterFrequency := Bag new .
```

```
words do: [ :each |
```

```
    each do: [ :aChar |
```

```
        letterFrequency add: aChar asString ]].
```

Comptage des scores des paires de symboles

La méthode `computePairs`, présentée dans la partie "Calcul de la fréquence de chaque paire existante du corpus", a été modifiée afin de permettre le calcul du score des paires de symboles présentes dans le corpus. `words`(sous-section **2.2.2**).

Dans un premier temps, la méthode `computeLetterFrequency` est appelée afin de calculer la fréquence de chaque symbole du corpus. Ensuite, la méthode `computePairs` définie dans la classe `BPE` est exécutée afin de calculer la fréquence de chaque paire de symboles existante.

Une fois ces fréquences obtenues, un dictionnaire `pairScores` est initialisé pour stocker les scores associés à chaque paire. Pour chaque paire de symboles présente dans la variable d'instance `pairs`, le calcul suivant est effectué :

- on récupère la fréquence de la paire dans `pairs` ;
- on récupère les fréquences des deux symboles composant cette paire à partir de la variable d'instance `letterFrequency` ;
- on calcule le score en divisant la fréquence de la paire par le produit des fréquences de ses deux symboles. Ce calcul peut être formalisé ainsi :

$$score(p_1, p_2) = \frac{freq(p_1, p_2)}{freq(p_1) * freq(p_2)}$$

Enfin, chaque paire est associée à son score et stockée dans la variable d'instance `pairScores`, sous forme de dictionnaire. Cette structure permet ensuite de sélectionner efficacement les paires ayant les scores les plus pertinents lors du processus de fusion de WordPiece.

```
computePairs
```

```
pairScores := Dictionary new.
```

```
super computePairs.
```

```
pairs associationsDo: [ :association |
```

```
pairScores add:(
```

```
Association
```

```
key: association key
```

```
value: (
```

```
association value / ((letterFrequency occurrencesOf: association key first) *  
(letterFrequency occurrencesOf: association key second)))
```

```
]
```

Recherche de la paire à fusionner

Dans WordPiece, la paire fusionnée à chaque itération est celle possédant le score maximal dans `pairScores`. La méthode `findPairToMerge` permet d'identifier cette paire.

```
findPairToMerge
```

```
^ (pairScores associations detectMax: [:assoc | assoc value]) key
```

Fusion entre deux symboles

La méthode `merge:and:` permet de fusionner deux symboles donnés en entrée afin de former un nouveau symbole. Cette opération est utilisée lors du processus itératif de fusion des paires dans l'algorithme WordPiece.

Avant la concaténation, la méthode vérifie si le second symbole est préfixé par le symbole spécial `'##'`, utilisé dans WordPiece pour indiquer qu'un symbole ne correspond pas au début d'un mot. Si ce préfixe est présent, il est supprimé avant la fusion afin de conserver une représentation correcte du nouveau symbole. Dans le cas contraire, les deux symboles sont directement concaténés.

Par exemple, la fusion des symboles `'a'` et `'##n'` produit le symbole `'an'`.

```
merge: s1 and: s2
```

```
( s2 beginsWith: '##' )  
  ifTrue: [ ^ s1, ( s2 allButFirst: 2 ) ]  
  ifFalse: [ ^ s1, s2 ]
```

Entraînement de WordPiece

Après le prétraitement du texte — comprenant le découpage du texte en mots, l'ajout du symbole spécial '##' aux caractères qui ne correspondent pas au début d'un mot, le calcul des scores des paires de symboles ainsi que l'initialisation du vocabulaire — l'entraînement de WordPiece est réalisé de manière itérative à l'aide de la méthode `mergeOneStep`.

Dans un premier temps, la méthode `findPairToMerge` est utilisée afin de rechercher, dans `pairScores`, la paire de symboles possédant le score maximal.

Ensuite, les deux symboles composant cette paire sont fusionnés à l'aide de la méthode `merge:and:`, ce qui permet de produire un nouveau symbole. Ce nouveau symbole est alors ajouté au vocabulaire grâce à la méthode `addToVocabulary:`.

Après l'ajout du nouveau symbole au vocabulaire, le corpus représenté par la variable d'instance `words` est mis à jour à l'aide de la méthode `updateCorpusFrom:to:`. Cette mise à jour consiste à remplacer toutes les occurrences de la paire fusionnée par le nouveau symbole créé.

Enfin, les fréquences et les scores des paires de symboles sont recalculés à l'aide de la méthode `computePairs`, afin de préparer l'itération suivante du processus d'entraînement.

```
mergeOneStep
```

```
| newVocabularyItem pair |
```

```
pair := self findPairToMerge .
```

```
newVocabularyItem := self merge: pair first and: pair second.
```

```
self addToVocabulary: newVocabularyItem.
```

```
self updateCorpusFrom: pair to: newVocabularyItem.
```

```
self computePairs.
```

Encodage d'un mot à l'aide du vocabulaire appris

La méthode `encodeWord:` permet d'encoder un mot à l'aide du vocabulaire appris durant l'entraînement de WordPiece. Elle prend en entrée un mot et retourne une collection ordonnée de symboles correspondant à la tokenisation de ce mot.

L'encodage est réalisé de manière itérative tant que le mot n'est pas entièrement tokenisé. À chaque étape, la méthode recherche dans le vocabulaire le plus long préfixe du mot courant.

Si aucun préfixe du mot courant n'est présent dans le vocabulaire, l'algorithme interrompt l'encodage et retourne le symbole spécial <UNK>, indiquant que le mot ne peut pas être représenté à l'aide du vocabulaire appris.

Dans le cas contraire, le préfixe trouvé est ajouté à une collection de tokens initialisée au début de la méthode. La partie correspondante est ensuite retirée du mot courant afin de poursuivre l'encodage sur la sous-chaîne restante. Si cette sous-chaîne n'est pas vide, le préfixe spécial '##' lui est ajouté afin d'indiquer que les symboles suivants ne correspondent pas au début d'un mot.

Le processus est répété jusqu'à ce que l'ensemble du mot soit tokenisé. La méthode retourne finalement la collection ordonnée des symboles obtenus.

Par exemple, si le vocabulaire contient les symboles 'play' et '##ing', alors le mot 'playing' pourra être encodé sous la forme :

##('play' '##ing')

encodeWord: aWord

```
| tokens word i |
```

```
word := aWord.
```

```
tokens := OrderedCollection new.
```

```
[ word isEmpty ] whileTrue: [
```

```
  i := 0.
```

```
  [ i < word size and: [ (vocabulary includes: (word allButLast: i)) not ] ]
```

```
  whileTrue: [
```

```
    i := i + 1
```

```
  ].
```

```
( i >= word size ) ifTrue: [
```

```
  ^ OrderedCollection with: '<UNK>'
```

```
].
```

```
tokens add: (word allButLast: i).
```

```
word := word allButFirst: word size - i.
```

```
( word isEmpty ) ifTrue: [
```

```
word := '##', word].].
```

```
^ tokens
```

Encodage d'un texte à partir du vocabulaire appris

La méthode `encode:` permet d'encoder un texte donné en entrée à l'aide du vocabulaire appris lors de l'entraînement de WordPiece. Elle retourne une liste ordonnée de sous-mots correspondant à la tokenisation complète du texte.

Dans un premier temps, le texte est découpé en une liste de mots à l'aide de séparateurs tels que les espaces, les retours à la ligne, les tabulations ainsi que certaines ponctuations. Ensuite, une collection est initialisée afin de stocker les sous-mots obtenus. La méthode parcourt alors chaque mot de la liste issue du découpage du texte et applique la méthode `encodeWord:` à chacun d'eux. Cette méthode réalise l'encodage d'un mot en sous-unités en utilisant le vocabulaire appris.

Le résultats de chaque appel à `encodeWord:` est ensuite ajoutés à la collection de sous-mots. Enfin, les différentes collections produites pour chaque mot sont aplaties à l'aide de `flatCollect:` afin d'obtenir une seule collection ordonnée de sous-mots représentant l'ensemble du texte.

La méthode retourne finalement cette collection ordonnée de sous-mots, qui constitue la représentation encodée du texte.

```
encode: aString
```

```
| rawWords tokens |
```

```
rawWords := aString substrings: {  
    Character space.  
    Character cr.  
    Character tab.  
    $..  
    $?.  
    $!  
    $:  
    $;  
    }.
```

```
tokens := OrderedCollection new.
```

```
rawWords do: [ :each |  
    tokens add: (self encodeWord: each)
```

```
].
```

```
tokens := tokens flatCollect: [ :each | each ].
```

```
^ tokens
```

Chapitre 4

Unigram

Chapitre 5

Optimisation de BPE

À chaque étape de l'entraînement du BPE, une mise à jour globale du corpus est généralement effectuée, ce qui implique le recalcul des fréquences de toutes les paires de symboles présentes dans le corpus. Cette opération peut devenir coûteuse en temps de calcul, en particulier sur de très grands corpus.

L'idée de l'optimisation consiste à éviter ce recalcul complet à chaque itération. Au lieu de mettre à jour l'ensemble du corpus, on limite les mises à jour uniquement aux zones du corpus affectées par la fusion en cours, c'est-à-dire les mots contenant la paire fusionnée.

Lorsqu'une paire de symboles est fusionnée, elle est remplacée par un nouveau symbole, ce qui modifie localement la structure des mots. Par conséquent, les fréquences de certaines paires doivent être ajustées : certaines disparaissent, d'autres apparaissent, et certaines voient leur fréquence diminuer.

Les mises à jour concernent donc principalement les paires situées dans le voisinage immédiat de la fusion.

Cette approche permet de recalculer uniquement les modifications locales des fréquences, réduisant ainsi significativement le coût computationnel tout en conservant un comportement équivalent à celui du BPE classique.

5.1 Idée de l'optimisation

Le principal problème du BPE naïf est qu'à chaque étape de fusion, l'ensemble du corpus doit être parcouru afin de mettre à jour sa représentation et de recalculer les fréquences de toutes les paires de symboles. Cette approche devient rapidement coûteuse lorsque la taille du corpus augmente, car la majorité des mots et des paires présents dans le corpus ne sont pas affectés par la fusion en cours.

L'optimisation du BPE repose sur une observation simple : lorsqu'une paire de symboles est fusionnée, les modifications sont limitées aux mots contenant une occurrence de cette paire. À l'intérieur de ces mots, seules les occurrences de la paire fusionnée ainsi que les paires formées avec les symboles situés immédiatement à gauche et à droite de ses occurrences peuvent être affectées par l'opération. Toutes les autres paires du corpus conservent leur fréquence et ne

nécessitent donc aucune mise à jour. Il devient ainsi inutile de recalculer les fréquences de l'ensemble des paires après chaque fusion ; seules les paires réellement affectées doivent être mises à jour.

Pour exploiter efficacement cette propriété, le corpus est représenté à l'aide de structures de données permettant d'accéder directement aux occurrences des paires de symboles ainsi qu'à leurs voisins. Plus précisément, le corpus est stocké dans une liste doublement chaînée dans laquelle chaque nœud contient un symbole. Afin de préserver les frontières entre les mots, un symbole spécial `endOfWord` est ajouté à la fin de chaque mot. Ce symbole permet d'identifier explicitement les limites des mots et garantit que les opérations de fusion restent limitées aux symboles appartenant à un même mot, empêchant ainsi toute fusion entre symboles issus de mots différents.

La liste doublement chaînée préserve l'ordre des symboles tout en permettant d'effectuer localement les modifications induites par une fusion. Lorsqu'une référence vers un nœud est connue, il est possible d'accéder directement à ce nœud, de modifier sa valeur, de le supprimer et encore d'accéder à ses voisins gauche et droit sans parcourir l'ensemble du corpus.

Afin d'éviter toute recherche coûteuse dans la liste chaînée, deux dictionnaires sont maintenus tout au long de l'exécution de l'algorithme :

- Le dictionnaire des fréquences des paires, qui associe à chaque paire de symboles son nombre d'occurrences dans le corpus. Ce dictionnaire permet d'identifier la paire à fusionner.
- Le dictionnaire des positions des paires associe à chaque paire l'ensemble de ses occurrences dans la liste doublement chaînée. Chaque occurrence est représentée par un couple de références vers les deux nœuds contenant respectivement le premier et le second symbole de la paire. Cette représentation permet d'accéder directement aux deux symboles concernés par une fusion sans effectuer de parcours supplémentaire dans la liste.

Lorsqu'une paire de symboles est sélectionnée pour la fusion, ses occurrences sont obtenues directement à partir du dictionnaire des positions. Pour chacune d'elles, la liste doublement chaînée permet d'accéder immédiatement aux symboles concernés ainsi qu'à leurs voisins. La fusion est alors réalisée localement en remplaçant les deux symboles par un nouveau symbole fusionné et en supprimant le nœud devenu inutile.

Les voisins des occurrences fusionnées permettent ensuite d'identifier précisément les paires dont les fréquences doivent être modifiées. Certaines paires disparaissent du corpus, d'autres voient leur fréquence diminuer ou augmenter, tandis que de nouvelles paires sont créées. Les dictionnaires des fréquences et des positions sont alors mis à jour uniquement pour ces paires affectées.

5.1.1 Exemple

Supposons que nous disposions du texte « FRED TED ». La figure 5.1 illustre sa représentation à l'aide d'une liste doublement chaînée. Chaque nœud de cette liste contient un symbole du texte. Afin de préserver les frontières entre les mots, un symbole spécial `endOfWord` est

ajouté à la fin de chaque mot.

Dans cette figure, les paires colorées en rouge correspondent à la paire sélectionnée pour la fusion, tandis que les nœuds rouges représentent les symboles qui composent cette paire. Les paires colorées en bleu correspondent aux paires dont la fréquence sera affectée par la fusion. Les nœuds bleus représentent les voisins des symboles impliqués dans la fusion et permettent d'identifier les nouvelles paires qui seront créées ou les anciennes paires dont la fréquence devra être mise à jour.

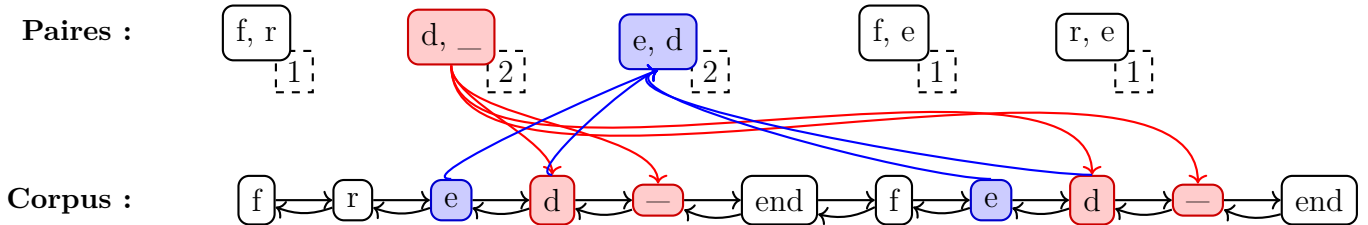


FIGURE 5.1 – Représentation du corpus "FRED TED" sous forme de liste doublement chaînée avec visualisation des paires et de leurs occurrences dans le processus de Byte Pair Encoding.

D'après la figure, deux paires possèdent la fréquence maximale, égale à 2 : (d, _) et (e, d). Supposons que la paire (d, _) soit sélectionnée pour la fusion. Cette paire apparaît deux fois dans le corpus.

La première étape consiste à consulter le dictionnaire des positions afin de récupérer l'ensemble des couples de références associés à la paire (d, _). Chaque couple de références pointe vers les deux nœuds contenant respectivement le premier et le second symbole d'une occurrence de cette paire dans la liste doublement chaînée. Ces couples permettent ainsi d'accéder directement à toutes les occurrences de la paire (d, _) sans parcourir l'ensemble du corpus.

Pour la première occurrence, le voisin situé à gauche de la paire (d, _) est le symbole e. La paire (e, d) est donc affectée par la fusion. Sa fréquence est décrétementée de 1 dans le dictionnaire des fréquences. Si cette fréquence devient nulle après la mise à jour, la paire est supprimée du dictionnaire.

Le couple de références représentant cette occurrence de la paire (e, d) est également supprimé de l'ensemble associé à cette paire dans le dictionnaire des positions. Si cet ensemble devient vide, la paire est supprimée du dictionnaire des positions.

Le voisin situé à droite de cette occurrence de (d, _) est le symbole endOfWord. Aucune paire n'est donc affectée à droite et aucune nouvelle paire ne sera créée de ce côté.

La fusion est ensuite réalisée dans la liste doublement chaînée. La valeur du nœud contenant d est remplacée par le symbole fusionné d_. Le nœud contenant _ est ensuite supprimé de la liste. Les liens entre les nœuds voisins sont alors mis à jour afin de préserver la structure de la liste doublement chaînée.

Une nouvelle paire est créée à gauche de l'occurrence fusionnée : (e, d_). Cette paire est ajoutée au dictionnaire des fréquences avec une fréquence de 1. Elle est également ajoutée au dictionnaire des positions avec comme valeur associée un ensemble contenant un couple de références vers les deux nœuds représentant cette occurrence, c'est-à-dire le nœud contenant e

et le nœud contenant $d_$.

Les mêmes opérations sont ensuite effectuées pour la seconde occurrence de $(d, _)$. Son voisin gauche est également e , ce qui affecte une seconde occurrence de la paire (e, d) . La fréquence de cette paire est donc à nouveau décrétementée de 1. Comme sa fréquence valait déjà 1 après la mise à jour précédente, elle devient nulle et la paire est supprimée du dictionnaire des fréquences.

Le couple de références représentant cette seconde occurrence de la paire (e, d) est également supprimé de l'ensemble associé à cette paire dans le dictionnaire des positions. Cet ensemble devenant vide, la paire est alors supprimée du dictionnaire des positions.

La fusion est alors réalisée dans la liste doublement chaînée de la même manière que pour la première occurrence : la valeur du nœud contenant d devient $d_$, le nœud contenant $_$ est supprimé et les liens entre les nœuds voisins sont mis à jour.

La paire $(e, d_)$ est ensuite ajoutée aux dictionnaires. Comme cette paire a déjà été créée lors du traitement de la première occurrence, sa fréquence est incrémentée de 1 dans le dictionnaire des fréquences, ce qui lui donne une fréquence totale de 2. Dans le dictionnaire des positions, le couple de références correspondant à cette seconde occurrence de la paire $(e, d_)$ est ajouté à l'ensemble déjà associé à cette paire.

Une fois toutes les occurrences de $(d, _)$ traitées, la paire fusionnée $(d, _)$ n'existe plus dans le corpus. Elle est donc supprimée du dictionnaire des fréquences ainsi que du dictionnaire des positions.

La figure 5.2 illustre l'état de la liste doublement chaînée et des dictionnaires après l'ensemble de ces mises à jour.

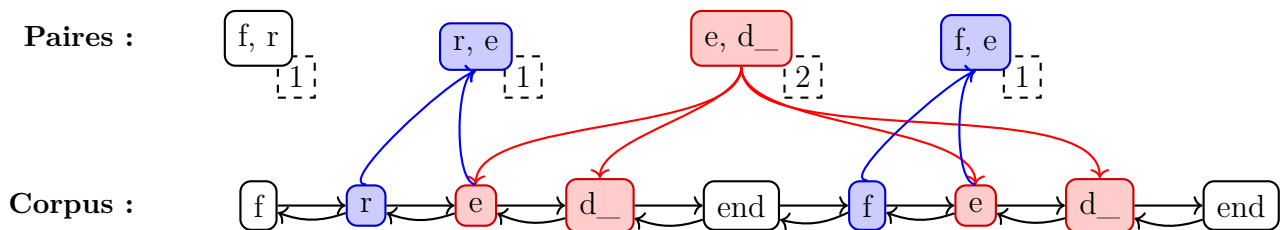


FIGURE 5.2 – État de la liste doublement chaînée après fusion de la paire $(d, _)$

La paire possédant désormais la fréquence maximale est $(e, d_)$. Sa fréquence est égale à 2 ; elle apparaît donc deux fois dans le corpus.

En consultant le dictionnaire des positions, on récupère l'ensemble des couples de références associé à cette paire. Chaque couple de références correspond à une occurrence de la paire et contient des références vers les deux nœuds contenant respectivement les symboles e et $d_$. Cela permet d'accéder directement aux occurrences de $(e, d_)$ dans la liste doublement chaînée.

On traite d'abord la première occurrence.

Le voisin situé à gauche de cette occurrence est le symbole r (coloré en bleu sur la figure). La paire (r, e) est donc affectée par la fusion. Sa fréquence est décrétementée de 1 dans le dictionnaire des fréquences. Comme cette paire n'apparaît qu'une seule fois dans le corpus, sa fréquence devient nulle après la mise à jour. Elle est donc supprimée du dictionnaire des

fréquences.

Le dictionnaire des positions doit également être mis à jour. Le couple de références correspondant à cette occurrence de la paire (r, e) est supprimé de l'ensemble associé à cette paire. Comme cet ensemble devient vide après la suppression, la paire (r, e) est également supprimée du dictionnaire des positions.

Le voisin situé à droite de cette occurrence est le symbole `endOfWord`. Ce symbole marquant la frontière du mot, aucune paire n'est affectée à droite et aucune mise à jour supplémentaire n'est nécessaire dans les dictionnaires.

La fusion peut alors être effectuée dans la liste doublement chaînée. La valeur du nœud contenant e est remplacée par le symbole fusionné $ed_$. Le nœud suivant, contenant $d_$, est ensuite supprimé. Les liens entre les nœuds voisins sont mis à jour afin de préserver la structure de la liste doublement chaînée : le nœud suivant de $ed_$ devient l'ancien nœud suivant du nœud contenant $d_$, et le lien précédent de ce nœud est mis à jour pour pointer vers $ed_$.

Une nouvelle paire est alors créée à gauche de l'occurrence fusionnée : $(r, ed_)$. Cette paire est ajoutée au dictionnaire des fréquences avec une fréquence égale à 1. Elle est également ajoutée au dictionnaire des positions avec comme valeur associée un ensemble contenant un couple de références vers les deux nœuds représentant cette occurrence, c'est-à-dire les nœuds contenant respectivement r et $ed_$.

Comme le voisin droit de $ed_$ est `endOfWord`, aucune nouvelle paire n'est créée à droite.

On traite ensuite la seconde occurrence de $(e, d_)$.

Le voisin situé à gauche de cette occurrence est le symbole f (également coloré en bleu sur la figure). La paire (f, e) est donc affectée par la fusion. Sa fréquence est décrétementée de 1 dans le dictionnaire des fréquences. Si cette fréquence devient nulle après la mise à jour, la paire est supprimée du dictionnaire.

Le dictionnaire des positions est également mis à jour. Le couple de références correspondant à cette occurrence de la paire (f, e) est supprimé de l'ensemble associé à cette paire. Si cet ensemble devient vide après la suppression, la paire est supprimée du dictionnaire des positions.

Le voisin situé à droite de cette occurrence est également `endOfWord`. Aucune paire n'est donc affectée à droite et aucune mise à jour supplémentaire n'est nécessaire.

La fusion est ensuite réalisée dans la liste doublement chaînée. La valeur du nœud contenant e est remplacée par $ed_$, puis le nœud suivant contenant $d_$ est supprimé. Les liens entre les nœuds voisins sont mis à jour de la même manière que pour la première occurrence.

Une nouvelle paire $(f, ed_)$ est alors créée. Elle est ajoutée au dictionnaire des fréquences avec une fréquence égale à 1. Elle est également ajoutée au dictionnaire des positions avec comme valeur associée un ensemble contenant un couple de références vers les deux nœuds représentant cette occurrence, c'est-à-dire les nœuds contenant respectivement f et $ed_$.

Comme le voisin droit de $ed_$ est `endOfWord`, aucune nouvelle paire n'est créée à droite.

Une fois les deux occurrences traitées, la paire $(e, d_)$ n'existe plus dans le corpus sous sa forme initiale. Elle est donc supprimée du dictionnaire des fréquences ainsi que du dictionnaire des positions.

La figure 5.3 illustre l'état de la liste doublement chaînée.

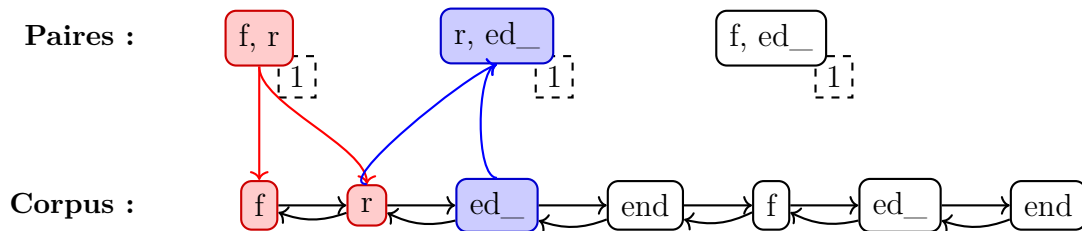


FIGURE 5.3 – État de la liste doublement chaînée après fusion de la paire (e, d_)

À ce stade, trois paires possèdent la fréquence maximale, égale à 1 : (f,r), (r,ed_) et (f,ed_). Supposons que la paire (f,r) soit sélectionnée pour la fusion. Les mêmes opérations que celles décrites précédemment sont alors effectuées : mise à jour des dictionnaires des fréquences et des positions, modification de la liste doublement chaînée, suppression des paires devenues inexistantes et ajout des nouvelles paires créées par la fusion.

Le processus est répété de manière itérative jusqu'à ce qu'il n'existe plus de paire pouvant être fusionnée ou jusqu'à ce que la taille de vocabulaire souhaitée soit atteinte.

5.1.2 Fonctionnement de l'optimisation du BPE

5.1.2.1 Entraînement

1. Comptage des occurrences mots

On commence par compter les occurrences de chaque mot du corpus afin d'éviter de répéter plusieurs fois les mêmes traitements sur des mots identiques. On obtient ainsi un dictionnaire associant chaque mot à sa fréquence d'apparition dans le corpus.

2. Représentation du corpus à l'aide d'une liste doublement chaînée

Chaque nœud de la liste doublement chaînée contient un couple (symbole, fréquence), où symbole représente un caractère ou un sous-mot et fréquence correspond à la fréquence du mot auquel appartient ce symbole.

Pour chaque association mot→fréquence du dictionnaire, on insère successivement un nœud pour chaque caractère du mot, contenant (caractère, fréquence). Une fois tous les caractères insérés, on ajoute un nœud contenant ("_", fréquence) afin de marquer la fin du mot. Enfin, un symbole spécial "endOfWord" est inséré dans la liste doublement chaînée afin de séparer les mots et d'éviter la création de paires entre symboles appartenant à des mots différents.

3. Construction des dictionnaires des fréquences et des positions

Deux dictionnaires sont construits : un dictionnaire des fréquences des paires et un dictionnaire des positions des paires. On parcourt tous les nœuds de la liste doublement chaînée pour construire ces deux dictionnaires.

Pour chaque nœud de la liste doublement chaînée, si le nœud suivant existe et ne contient pas le symbole "endOfWord", on construit la paire formée par les symboles stockés dans les deux nœuds.

Pour le dictionnaire des fréquences, si la paire n'existe pas encore, on l'ajoute avec comme valeur la fréquence stockée dans le nœud courant. Sinon, on incrémente sa fréquence dans le dictionnaire des fréquences de cette même valeur.

Pour le dictionnaire des positions, si la paire n'existe pas encore, on crée une association entre cette paire et un ensemble contenant un couple de références vers le nœud courant et le nœud suivant du nœud courant, correspondant à cette occurrence. Sinon, ce couple de références est ajouté à l'ensemble déjà associé à cette paire.

Ainsi, pour chaque paire, le dictionnaire des positions contient l'ensemble des couples de références vers les nœuds correspondant à ses occurrences dans la liste doublement chaînée.

4. Sélection de la paire la plus fréquente

On sélectionne la paire possédant la fréquence maximale dans le dictionnaire des fréquences.

5. Ajout du nouveau symbole au vocabulaire

Les deux symboles de la paire sélectionnée sont fusionnés pour former un nouveau symbole, qui est ajouté au vocabulaire.

6. Récupération des positions de la paire à fusionner

On récupère dans le dictionnaire des positions l'ensemble des couples de références associés à la paire sélectionnée. Chaque couple de références correspond à une occurrence de la paire et contient des références vers les deux nœuds contenant respectivement le premier et le second symbole.

7. Mise à jour des dictionnaires avant la fusion

Pour chaque couple de références $p = (p_1, p_2)$ de l'ensemble associé à la paire maximale dans le dictionnaire des positions :

- Si le nœud précédent du nœud référencé par p_1 existe et que sa valeur n'est pas `endOfWord`, la paire formée par le symbole contenu dans ce nœud précédent et le symbole contenu dans le nœud référencé par p_1 disparaîtra après la fusion. La fréquence de cette paire dans le dictionnaire des fréquences est donc décrétementée de la fréquence associée à cette occurrence. Si sa fréquence devient nulle, la paire est supprimée du dictionnaire des fréquences. Le couple de références formé par le nœud précédent du nœud référencé par p_1 et le nœud référencé par p_1 est retiré de l'ensemble associé à cette paire dans le dictionnaire des positions. Si cet ensemble devient vide, la paire est supprimée du dictionnaire des positions.
- Si le nœud suivant du nœud référencé par p_2 existe et que sa valeur n'est pas `endOfWord`, alors la paire formée par le symbole contenu dans le nœud référencé par p_2 et celui contenu dans ce nœud suivant disparaîtra après la fusion. La fréquence de cette paire dans le dictionnaire des fréquences est décrétementée de la fréquence associée à cette occurrence. Si sa fréquence devient nulle, elle est supprimée du dictionnaire des fréquences. Le couple de références formé par le nœud référencé par p_2

et le nœud suivant du nœud référencé par p_2 est retiré de l'ensemble associé à cette paire dans le dictionnaire des positions. Si cet ensemble devient vide, la paire est supprimée du dictionnaire des positions.

Remarque

La mise à jour des dictionnaires est effectuée avant la fusion. En effet, après la fusion, certains nœuds de la liste doublement chaînée — notamment ceux contenant le second symbole de la paire — sont supprimés ou modifiés. Il devient alors impossible de mettre à jour correctement le dictionnaire des positions des paires.

8. Modification de la liste doublement chaînée

- le symbole contenu dans le nœud référencé par p_1 est remplacé par le nouveau symbole obtenu par fusion ;
- le nœud référencé par p_2 , qui contient le second symbole de la paire, est supprimé ;
- les pointeurs de la liste doublement chaînée sont mis à jour afin de préserver la structure de la liste.

9. Ajout des nouvelles paires après la fusion

- Si le nœud précédent du nœud référencé par p_1 existe et ne contient pas le symbole `endOfWord`, on forme une nouvelle paire entre le symbole contenu dans ce nœud précédent et le nouveau symbole obtenu après la fusion. Cette paire est mise à jour dans le dictionnaire des fréquences : si elle existe déjà, sa fréquence est incrémentée de la fréquence associée à l'occurrence en cours de traitement ; sinon, elle est créée avec cette valeur. On construit un couple de références (q, p_1) représentant cette nouvelle occurrence, où q est la référence du nœud précédent du nœud référencé par p_1 . Si la paire est déjà présente dans le dictionnaire des positions, ce couple est ajouté à l'ensemble associé à cette paire. Sinon, un nouvel ensemble contenant uniquement ce couple est créé et associé à la paire dans le dictionnaire des positions.
- Si le nœud suivant du nœud référencé par p_1 existe et ne contient pas le symbole `endOfWord`, on forme une nouvelle paire entre le symbole contenu dans p_1 (après fusion) et le symbole contenu dans ce nœud suivant. Cette paire est ensuite mise à jour dans le dictionnaire des fréquences : si elle existe déjà, sa fréquence est incrémentée de la fréquence associée à l'occurrence en cours de traitement ; sinon, elle est créée avec cette valeur. on construit un couple de références (p_1, r) , où r est la référence du nœud suivant du nœud référencé par p_1 . Si la paire existe déjà dans le dictionnaire des positions, ce couple est ajouté à l'ensemble associé à cette paire. Sinon, un nouvel ensemble contenant ce couple est créé et associé à la paire dans le dictionnaire des positions.

10. Itération

Une fois toutes les occurrences de la paire maximale fusionnées et les dictionnaires mis à jour, on sélectionne à nouveau la paire la plus fréquente et on répète l'étape 5 jusqu'à l'étape 9.

Le processus est répété jusqu'à ce que la taille du vocabulaire souhaitée soit atteinte ou qu'il n'existe plus de paire pouvant être fusionnée.

5.1.2.2 Encodage

Après l'entraînement, les fusions apprises sont appliquées à un texte.

1. Prétraitement du texte

On commence par découper le texte en une liste de mots, en utilisant les espaces, tabulations et suites d'espaces comme séparateurs.

2. Construction de la liste doublement chaînée

Pour chaque mot de la liste précédente :

- chaque caractère du mot est inséré dans une liste doublement chaîné ;
- à la fin de chaque mot, on ajoute un symbole spécial "_" dans la liste doublement chaînée ;
- puis on ajoute le symbole `endOfWord` afin de marquer la fin du mot.

3. Construction du dictionnaire des positions

On parcourt ensuite la liste chaînée afin de construire le dictionnaire des positions, en partant du premier nœud.

Pour chaque nœud n de la liste tel que n existe et que sa valeur n'est pas `endOfWord` :

- on forme une paire de symboles entre le contenu du nœud n et celui du nœud suivant ;
- on forme une couple de références représentant une occurrence de cette paire de symboles et contenant la référence du nœud courant n et la référence du nœud suivant de n ;
- Si la paire n'existe pas encore dans le dictionnaire des positions, un nouvel ensemble contenant ce couple est créé. Sinon, le couple est ajouté à l'ensemble déjà associé à cette paire.

4. Application des fusions

Les fusions apprises lors de l'entraînement sont stockées dans une liste ordonnée. Chaque élément de cette liste est une association entre une paire de symboles et le symbole obtenu après leur fusion.

Les fusions doivent être appliquées dans le même ordre que celui utilisé lors de l'entraînement. Ce point est essentiel : appliquer les fusions dans un ordre différent peut conduire à une segmentation différente du texte.

Pour chaque paire de la liste des fusions :

- si la paire n'existe pas dans le dictionnaire des positions, aucune occurrence de cette paire n'est présente dans le texte et on passe à la paire suivante dans la liste ;
- sinon, on récupère l'ensemble des couples de références associés à cette paire dans le dictionnaire des positions. Chaque couple de références correspond à une occurrence de la paire et permet d'accéder directement aux deux nœuds concernés dans la liste chaînée.

Pour chaque couple de références (p_1, p_2) :

(a) **Mise à jour des paires affectées avant la fusion**

Avant d'effectuer la fusion, il est nécessaire de mettre à jour le dictionnaire des positions afin de supprimer les occurrences des paires qui vont disparaître à la suite de la fusion.

- Si le nœud précédent du nœud référencé par p_1 existe et ne contient pas le symbole `endOfWord`, alors la paire formée par le symbole contenu dans ce nœud précédent et le symbole contenu dans le nœud référencé par p_1 disparaîtra après la fusion.

On retire de la valeur associée à cette paire dans le dictionnaire des positions le couple de référence formé par la référence du nœud précédent du nœud référencé par p_1 et la référence p_1 .

Si l'ensemble associé à cette paire devient vide après cette suppression, la paire est retirée du dictionnaire des positions.

- Si le nœud suivant du nœud référencé par p_2 existe et ne contient pas le symbole `endOfWord`, alors la paire formée par le symbole contenu dans le nœud référencé par p_2 et le symbole contenu dans ce nœud suivant disparaîtra également après la fusion.

On retire de la valeur associée à cette paire dans le dictionnaire des positions le couple de référence formé par la référence p_2 et la référence du nœud suivant du nœud référencé par p_2 .

Si l'ensemble associé à cette paire devient vide après cette suppression, la paire est également retirée du dictionnaire des positions.

(b) **Fusion dans la liste chaînée**

- le symbole contenu dans p_1 est remplacé par le symbole fusionné ;
- le nœud référencé par p_2 est supprimé ;
- les pointeurs de la liste chaînée sont mis à jour pour maintenir la continuité.

(c) **Ajout des nouvelles paires après fusion**

Après la fusion, le symbole contenu dans le nœud référencé par p_1 a été remplacé par le symbole fusionné et le nœud référencé par p_2 a été supprimé. Cette modification peut faire apparaître de nouvelles paires dans la liste chaînée.

- Si le nœud précédent du nœud référencé par p_1 existe et ne contient pas le symbole `endOfWord`, une nouvelle paire est formée entre le symbole contenu dans ce nœud précédent et le symbole fusionné contenu dans le nœud référencé par p_1 .

On construit alors le couple de références représentant cette occurrence de la paire, formé par la référence du nœud précédent du nœud référencé par p_1 et la référence p_1 .

- Si le nœud suivant du nœud référencé par p_1 existe et ne contient pas le symbole `endOfWord`, une nouvelle paire est formée entre le symbole fusionné contenu dans

le nœud référencé par p_1 et le symbole contenu dans ce nœud suivant.

On construit alors le couple de références représentant cette occurrence de la paire, formé par la référence p_1 et la référence du nœud suivant du nœud référencé par p_1 .

Pour chacune de ces nouvelles paires :

- si la paire est déjà présente dans le dictionnaire des positions, le couple de références correspondant à l'occurrence est ajouté à l'ensemble déjà associé à cette paire ;
- sinon, un nouvel ensemble contenant ce couple est créé et associé à la paire dans le dictionnaire des positions.

Lorsque toutes les occurrences de la paire ont été fusionnées et que les nouvelles paires ont été ajoutées au dictionnaire des positions, la paire fusionnée est supprimée de ce dictionnaire. L'algorithme passe alors à l'élément suivant de la liste des fusions et répète les mêmes opérations jusqu'à ce que toutes les fusions apprises aient été appliquées.

5.2 Réalisation

Cette section présente l'implémentation dans Pharo de l'optimisation proposée pour la version naïve de l'algorithme BPE.

5.2.1 Création des classes

Cinq classes ont été développées pour réaliser cette optimisation : `BPETrain`, `BPEEncoder`, `Preprocessing`, `Token` et `EndOfWord`.

5.2.1.1 Classe `BPETrain`

La classe `BPETrain` est responsable de la construction de la représentation du corpus, de la création et de la mise à jour des dictionnaires utilisés par l'algorithme d'entraînement du BPE ainsi que de la réalisation des fusions successives.

Elle regroupe les variables d'instance et les méthodes nécessaires à l'entraînement du BPE. Les variables d'instance sont les suivantes :

- **vocabulary**, stocke le vocabulaire appris durant l'entraînement. Initialement, il contient l'ensemble des caractères présents dans le corpus ainsi qu'un symbole spécial '`</t>`' marquant la fin d'un mot. Au cours de l'entraînement, les nouveaux symboles obtenus par fusion sont progressivement ajoutés à ce vocabulaire.
- **corpus**, qui représente le texte sous la forme d'une liste doublement chaînée. Cette structure est modifiée après chaque fusion et constitue la représentation principale du corpus durant l'apprentissage.

- **pairFrequencies**, un dictionnaire associant chaque paire de symboles à sa fréquence d'apparition dans le corpus. Il permet d'identifier rapidement la paire la plus fréquente à fusionner.
- **pairPositions**, un dictionnaire associant chaque paire de symboles à l'ensemble de ses occurrences dans le corpus. Chaque occurrence est représentée par un couple de références vers les deux nœuds contenant respectivement le premier et le second symbole de la paire.
- **merges**, qui mémorise l'ensemble des fusions apprises durant l'entraînement. Ces fusions seront ensuite réutilisées lors de l'encodage.

La classe contient également les méthodes permettant de construire la représentation du corpus, d'initialiser les dictionnaires, de sélectionner la paire la plus fréquente, d'effectuer les fusions et de mettre à jour les dictionnaires et le corpus après chaque fusion.

```
Object << #BPETrain
  slots: { #corpus . #vocabulary . #merges . #pairFrequencies . #pairPositions };
  package: 'Tokenizer'
```

5.2.1.2 Classe BPEEncoder

La classe **BPEEncoder** est utilisée pour appliquer à un nouveau texte les fusions apprises lors de l'entraînement.

Elle hérite de **BPETrain** car les phases d'entraînement et d'encodage utilisent des structures de données similaires et partagent plusieurs traitements communs. Dans les deux cas, le texte est représenté à l'aide d'une liste doublement chaînée et les occurrences des paires sont stockées dans un dictionnaire permettant d'y accéder directement. Plusieurs opérations sont également communes aux deux phases, notamment la construction de ces structures, la recherche des occurrences d'une paire, la mise à jour des positions des paires et la réalisation des fusions. Plusieurs méthodes définies dans **BPETrain** sont donc réutilisées, tandis que d'autres sont redéfinies afin de prendre en compte les spécificités de l'encodage.

Les principales différences entre les deux phases sont les suivantes :

- **Prétraitement du corpus** : lors de l'entraînement, les occurrences de chaque mot sont comptabilisées afin d'éviter de traiter plusieurs fois des mots identiques. Lors de l'encodage, cette étape n'est pas réalisée car l'ordre exact des mots dans le texte doit être conservé.
- **Gestion des paires** : Durant l'entraînement, les dictionnaires **pairFrequencies** et **pairPositions** sont utilisés afin d'identifier la paire la plus fréquente et de localiser ses occurrences dans le corpus. Lors de l'encodage, les fusions à appliquer sont déjà connues puisqu'elles ont été apprises pendant l'entraînement. Il n'est donc plus nécessaire de calculer ou de conserver les fréquences des paires. Seul le dictionnaire **pairPositions** est utilisé afin de localiser les occurrences des paires auxquelles les fusions doivent être appliquées.

Cette classe ne possède aucune variable d'instance supplémentaire. Elle hérite des variables d'instance de `BPETrain` ainsi que des méthodes communes aux deux phases, tout en redéfinissant celles dont le comportement diffère lors de l'encodage.

```
BPETrain << #BPSEncoder
  slots: {};
  package: 'Tokenizer'
```

5.2.1.3 Classe Preprocessing

La classe `Preprocessing` regroupe les méthodes utilisées lors du prétraitement du texte.

Elle contient notamment les méthodes permettant de découper un texte ou une liste de textes en une liste de mots.

Cette classe ne possède aucune variable d'instance.

```
Object << #Preprocessing
  slots: {};
  package: 'Tokenizer'
```

5.2.1.4 Classe Token

La classe `Token` représente un symbole du corpus. Ce symbole peut être un caractère ou un sous-mot obtenu après plusieurs opérations de fusion.

Chaque instance de cette classe contient trois informations principales :

- **token** : le symbole représenté ;
- **occurrence** : la fréquence du mot auquel appartient ce symbole ;
- **position** : la référence vers le nœud de la liste doublement chaînée contenant ce symbole.

La classe `Token` contient les méthodes permettant d'accéder à ces informations et de les modifier. Elle définit également les opérations nécessaires à la construction de `pairFrequencies` et `pairPositions`, ainsi qu'à leur mise à jour lors des fusions réalisées par l'algorithme.

```
Object << #Token
  slots: { #token . #occurrence . #position };
  package: 'Tokenizer'
```

5.2.1.5 Classe EndOfWord

La classe `EndOfWord` est utilisée pour représenter explicitement les frontières entre les mots dans le corpus.

Des instances de cette classe sont insérées entre deux mots dans la liste doublement chaînée afin d'empêcher la création de paires reliant des symboles appartenant à des mots différents.

Aucune variable d'instance spécifique n'est définie dans cette classe.

L'utilisation d'une classe dédiée présente plusieurs avantages par rapport à l'utilisation d'une simple chaîne de caractères telle que "endOfWord". Tout d'abord, ce type de représentation pourrait apparaître dans le texte lui-même et entraîner des ambiguïtés. Ensuite, elle permet d'exploiter le polymorphisme pour simplifier l'implémentation.

Les mêmes messages peuvent être envoyés aux objets de type **Token** et **EndOfWord**, mais leur comportement diffère. Les méthodes définies dans **Token** réalisent effectivement les opérations de mise à jour nécessaires à l'algorithme, tandis que celles définies dans **EndOfWord** n'effectuent aucune action. Cette approche permet d'éviter la multiplication de tests conditionnels dans le code et rend l'implémentation plus modulaire.

Remarque : Les éléments stockés dans les nœuds de la liste doublement chaînée **corpus** sont des instances des classes **Token** ou **EndOfWord**. Ainsi, chaque nœud contient soit un symbole du corpus, soit un marqueur de fin de mot.

5.2.2 Structures de données utilisées

1. Liste doublement chaînée

La liste doublement chaînée permet de représenter le texte. Chaque nœud contient soit une instance de la classe **Token**, soit une instance de la classe **EndOfWord**. Les instances de **Token** représentent les symboles composant les mots, tandis que les instances de **EndOfWord** marquent les frontières entre les mots.

Cette structure est particulièrement adaptée car les mots étant des séquences ordonnées de caractères, elle permet de conserver naturellement cet ordre tout en autorisant des modifications locales lors des opérations de fusion.

Elle offre également un accès direct au nœud précédent et au nœud suivant d'un nœud donné, ce qui permet d'identifier les symboles voisins d'une paire et de déterminer les nouvelles paires créées ou les anciennes paires affectées par une fusion.

En combinant cet accès local avec les références vers les occurrences des paires, les modifications peuvent être appliquées uniquement aux nœuds concernés, sans parcourir l'ensemble de la structure. Cette approche permet également de mettre à jour efficacement les dictionnaires **PairFrequencies** et **PairPositions**.

2. Bag

Cette structure de données est utilisée pour stocker les paires de symboles présentes dans le corpus ainsi que leur nombre d'occurrences. Contrairement à un ensemble, une même paire peut être ajoutée plusieurs fois dans un Bag. Celui-ci maintient alors un compteur associé à chaque paire, correspondant au nombre de fois où elle a été ajoutée.

Cette structure permet ainsi d'obtenir rapidement le nombre d'occurrences d'une paire donnée sans parcourir l'ensemble du corpus. Elle facilite également la mise à jour des fréquences : lorsqu'une nouvelle occurrence d'une paire est rencontrée, il suffit d'ajouter cette paire dans le Bag, ce qui incrémente son compteur. De même, lorsqu'une occurrence

disparaît à la suite d'une fusion, le compteur associé à cette paire peut être décrémenté en supprimant une occurrence de cette paire du Bag.

3. Dictionnaire

Cette structure de données est utilisée pour associer chaque paire de symboles à l'ensemble de ses occurrences dans le corpus. Pour chaque paire, la valeur associée dans le dictionnaire est un ensemble de couples de références vers les deux nœuds consécutifs de la liste doublement chaînée contenant cette paire.

Le principal avantage de cette structure est de permettre un accès direct aux occurrences d'une paire donnée. En utilisant la paire comme clé, il est possible d'obtenir immédiatement l'ensemble des couples de références associés, sans avoir à parcourir l'intégralité du corpus.

Cette structure joue un rôle essentiel lors des opérations de fusion, car elle permet de localiser rapidement les occurrences de la paire à fusionner ainsi que les paires voisines susceptibles d'être affectées par cette fusion. Les mises à jour peuvent ainsi être limitées aux seules parties du corpus concernées, ce qui évite de reparcourir l'ensemble de la liste doublement chaînée à chaque itération de l'algorithme et améliore son efficacité.

4. Liste

Une liste est utilisée pour stocker les fusions apprises lors de la phase d'entraînement, ainsi que leur ordre d'apprentissage. Cet ordre est essentiel lors de la phase d'encodage, car les fusions doivent être appliquées dans le même ordre que celui dans lequel elles ont été apprises. Toute modification de cet ordre conduirait à une segmentation différente du texte.

Chaque élément de la liste représente une fusion, généralement sous la forme d'une paire de symboles associée au nouveau symbole obtenu. L'ordre de la liste définit donc la séquence exacte des transformations à appliquer sur le corpus.

Ainsi, les premières fusions apprises sont appliquées en premier lors de l'encodage.

5. Ensemble

Un ensemble est utilisé pour stocker les symboles appris lors de la phase d'entraînement, ainsi que les caractères initiaux présents dans le texte avant l'entraînement. Cette structure ne conserve pas l'ordre des éléments et ne stocke pas d'informations de fréquence.

L'utilisation d'un ensemble est donc adaptée lorsque seule la présence ou l'absence d'un symbole est importante. Elle permet notamment d'éviter les doublons et de garantir que chaque symbole n'est stocké qu'une seule fois.

Dans ce contexte, l'ensemble est principalement utilisé pour représenter le vocabulaire du modèle.

5.2.3 Implémentation du prétraitement du texte

Le prétraitement du texte consiste à transformer un texte brut en une liste ordonnée de mots. Cette opération est réalisée par la méthode de classe preprocess, définie dans la classe

Preprocessing.

Cette méthode reçoit en entrée soit un texte, soit une liste de textes. Son rôle est de créer une instance de la classe **Preprocessing**, puis de lui envoyer le message **preprocess:** avec l'objet reçu en argument. Elle retourne ensuite le résultat produit par cette méthode.

```
preprocess: anObject
```

```
|p|
```

```
p := self new.
```

```
^ (p preprocess: anObject)
```

La méthode d'instance **preprocess:** détermine ensuite le traitement à effectuer en fonction du type de l'objet reçu. Si l'objet est une chaîne de caractères, elle lui applique le prétraitement correspondant à un texte unique. Si l'objet est une collection de textes, elle applique le prétraitement à chacun des textes de cette collection.

```
preprocess: anObject
```

```
anObject isArray ifTrue: [ ^ self preprocessArrayOfSentences: anObject ].
```

```
anObject isString ifTrue: [ ^ self preprocessText: anObject ].
```

Lorsque l'entrée est un texte, la méthode **preprocessText:** est utilisée.

Cette méthode reçoit un texte en entrée et le découpe en mots en utilisant les espaces, les tabulations et les retours à la ligne comme séparateurs. Elle retourne la liste des mots obtenus.

```
preprocessText:aText
```

```
|rawWords|
```

```
rawWords :=(aText substrings: {Character space . Character cr . Character tab}).
```

```
^rawWords
```

Lorsque l'entrée est une liste de textes, la méthode **preprocessArrayOfSentences:** est utilisée.

Cette méthode applique d'abord le prétraitement à chaque texte de la collection à l'aide de la méthode **preprocessText:**. Le résultat est alors une collection de listes de mots. **flatCollect:** est ensuite utilisée pour aplatir cette structure et obtenir une unique liste contenant tous les mots extraits des textes de la collection.

```
preprocessArrayOfSentences: anArray
```

```
  ^ (anArray collect: [ :each | self preprocessText: each ])
    flatCollect: [ :each | each ].
```

Ainsi, que l'entrée soit un texte unique ou une liste de textes, le résultat final est toujours une liste ordonnée de mots.

Le prétraitement a été implémenté dans une classe distincte des classes `BPETrain` et `BPEEncoder` afin de séparer les responsabilités des différents composants du système. Cette conception facilite la maintenance du code ainsi que son évolution. Elle permet notamment de réutiliser les mêmes opérations de prétraitement pour plusieurs tokenizers et d'ajouter ultérieurement de nouvelles stratégies de prétraitement sans modifier les classes responsables de l'entraînement et de l'encodage du BPE.

5.2.4 Initialisation des variables d'instance de `BPETrain`

`corpus` est initialisée avec une instance de la classe `DoubleLinkedList`, qui permet de représenter le corpus sous la forme d'une liste doublement chaînée ;

`vocabulary` est initialisée avec une instance de la classe `Set`, utilisée pour stocker les symboles du vocabulaire sans doublons ;

`pairFrequencies` est initialisée avec une instance de la classe `Bag`, utilisée pour mémoriser les paires de symboles ainsi que leurs fréquences d'occurrence ;

`pairPositions` est initialisée avec une instance de la classe `Dictionary`, qui associe chaque paire de symboles à l'ensemble de ses occurrences dans le corpus ;

`merges` est initialisée avec une instance de la classe `OrderedCollection`, utilisée pour conserver les fusions apprises ainsi que leur ordre d'apprentissage.

L'initialisation de ces variables est réalisée dans la méthode `initialize` de la classe `BPETrain`.

La méthode `initialize` est définie comme suit :

```
initialize
```

```
  super initialize.
```

```
  corpus := DoubleLinkedList new.
```

```
  vocabulary := Set new.
```

```
  pairFrequencies := Bag new .
```

```
  pairPositions := Dictionary new.
```

```
merges:= OrderedCollection new.
```

5.2.5 Construction de la représentation du corpus dans BPETrain

Le corpus est représenté à l'aide d'une liste doublement chaînée stockée dans la variable d'instance `corpus`. Chaque nœud de cette liste contient soit une instance de la classe `Token`, représentant un symbole du corpus, soit une instance de la classe `EndOfWord`, utilisée pour marquer les frontières entre les mots.

La construction de cette représentation est réalisée par la méthode `corpus` : définie dans la classe `BPETrain`.

Cette méthode reçoit en entrée un texte ou une collection de textes. Cette entrée est ensuite transmise à la méthode de classe `preprocess` : de la classe `Preprocessing`, qui retourne une liste de mots.

Les mots obtenus sont ensuite ajoutés dans une instance de la classe `Bag`. Cette structure permet de compter automatiquement le nombre d'occurrences de chaque mot dans le corpus. Ainsi, un mot apparaissant plusieurs fois n'est représenté qu'une seule fois dans la liste doublement chaînée. Sa fréquence d'apparition est conservée dans les instances de `Token` qui représentent ses symboles. Cette approche réduit la taille de la représentation du corpus et évite d'effectuer plusieurs fois les mêmes traitements sur des mots identiques lors de l'entraînement.

La méthode parcourt ensuite les associations du `Bag`. Chaque association est constituée d'un mot et de sa fréquence d'apparition dans le corpus.

Pour chaque caractère du mot, les opérations suivantes sont effectuées :

1. Une instance de la classe `Token` est créée.
2. La variable d'instance `token` reçoit le caractère courant.
3. La variable d'instance `occurrence` reçoit la fréquence du mot dans le corpus.
4. L'instance est ajoutée à la liste doublement chaînée `corpus`.
5. La variable d'instance `position` de cette instance est initialisée avec une référence vers le nœud de la liste contenant le symbole ajouté.

Une fois tous les caractères du mot insérés dans la liste, un symbole spécial '`</t>`' est ajouté :

1. Une nouvelle instance de `Token` est créée.
2. La variable d'instance `token` reçoit la valeur '`</t>`'.
3. La variable d'instance `occurrence` reçoit la fréquence du mot.
4. L'instance est ajoutée à la liste doublement chaînée.
5. Sa variable d'instance `position` est initialisée avec une référence vers le nœud contenant ce symbole.

Enfin, une instance de la classe **EndOfWord** est ajoutée à la liste doublement chaînée.

Le symbole '**</t>**' fait partie du vocabulaire manipulé par l'algorithme BPE et permet de représenter explicitement la fin d'un mot lors des opérations de fusion. L'objet **EndOfWord**, quant à lui, joue un rôle purement technique : il sert à séparer deux mots consécutifs dans la liste doublement chaînée et empêche la création de paires entre des symboles appartenant à des mots différents.

Cette représentation conserve l'ordre des symboles composant les mots tout en permettant des modifications locales efficaces lors des opérations de fusion. Grâce aux références stockées dans les instances de **Token**, il est également possible d'accéder directement aux nœuds concernés par une fusion sans parcourir l'ensemble du corpus.

```
corpus: anObject
```

```
| wordFrequencies |
```

```
wordFrequencies := Bag new.
```

```
wordFrequencies addAll: (Preprocessing preprocess: anObject) .
```

```
wordFrequencies associationsDo: [ :each|
```

```
    each key do: [ :char |
```

```
        corpus add: (
```

```
            Token
```

```
                token: char asString
```

```
                occurence: each value) .
```

```
corpus lastLink value position: corpus lastLink ].
```

```
corpus add: (
```

```
    Token
```

```
        token: '</t>'
```

```
        occurence: each value).
```

```
corpus lastLink value position: corpus lastLink .
```

```
corpus add: EndOfWord new ].
```

```
^ corpus
```

5.2.6 Construction de vocabulary dans BPETrain

La méthode `computeVocabulary` initialise le vocabulaire avec tous les symboles présents dans le corpus avant les opérations de fusion. Ces symboles correspondent aux caractères des mots ainsi qu'au symbole spécial '`</t>`' marquant la fin d'un mot.

La méthode parcourt la liste doublement chaînée `corpus` en commençant par son premier nœud et continue jusqu'à ce que tous les nœuds aient été visités.

Comme expliqué précédemment, la valeur contenue dans chaque nœud est soit une instance de `Token`, soit une instance de `EndOfWord`.

Lorsque la valeur du nœud courant est une instance de `Token`, la méthode récupère la valeur de sa variable d'instance `token` et l'ajoute au vocabulaire en envoyant le message `addToVocabulary:` à l'instance courante de `BPETrain`.

La méthode `addToVocabulary:` est responsable de l'ajout d'un symbole dans le vocabulaire. Celui-ci étant représenté par une instance de `Set`, les doublons sont automatiquement ignorés. Ainsi, chaque symbole n'apparaît qu'une seule fois dans le vocabulaire, même s'il est présent plusieurs fois dans le corpus.

L'implémentation de `computeVocabulary` est la suivante :

```
computeVocabulary
```

```
|i| i:=corpus firstLink .
```

```
[ i notNil ] whileTrue: [
```

```
    [ i value class ~= EndOfWord whileTrue: [
```

```
        self addToVocabulary: (i value token ).
```

```
        i:= i nextLink ].
```

```
i:= i nextLink ]
```

5.2.7 Construction des dictionnaires `pairFrequencies` et `pairPositions` dans BPETrain

La construction des dictionnaires `pairFrequencies` et `pairPositions` est réalisée par la méthode `computePairs`, définie dans la classe `BPETrain`.

```
computePairs
```

```
|current|
```

```

current := self corpus firstLink .

[current nextLink notNil ] whileTrue: [

current nextLink value
  tryToCreatePairWith: current value
  in: self.

current := current nextLink ].

```

Cette méthode parcourt la liste doublement chaînée, `corpus`, représentant le corpus afin d'identifier toutes les paires de symboles consécutifs. Le parcours commence au premier nœud de la liste et se poursuit jusqu'à ce qu'il n'existe plus de nœud suivant.

Pour chaque nœud courant, la méthode tente de construire une paire de symboles à partir de la valeur contenue dans ce nœud et de celle contenue dans le nœud suivant. Les valeurs stockées dans les nœuds étant soit des instances de la classe `Token`, soit des instances de la classe `EndOfWord`, la création effective d'une paire de symboles repose sur le polymorphisme.

La méthode `tryToCreatePairWith:in:` est définie dans les classes `Token` et `EndOfWord`.

Dans la classe `EndOfWord`, cette méthode ne réalise aucune opération :

```

tryToCreatePairWith: anObject in: aBPE

^ self

```

Ainsi, lorsqu'un des deux symboles correspond à une frontière de mot, aucune paire n'est créée.

Dans la classe `Token`, la méthode vérifie d'abord que l'objet passé en paramètre n'est pas nul. Si ce n'est pas le cas, elle délègue la création effective de la paire à cet objet en lui envoyant le message `createPairWith:in:`.

```

tryToCreatePairWith: anObject in: aBPE

anObject ifNil: [ ^self ].

anObject
  createPairWith: self
  in: aBPE

```

Là encore, le comportement dépend du type de l'objet recevant le message.

Dans la classe `EndOfWord`, la méthode `createPairWith:in:` ne réalise aucune action :

```
createPairWith: anObject in: aBPE
```

```
^ self
```

En revanche, dans la classe **Token**, cette méthode demande à l'instance de **BPETrain** de créer et d'enregistrer la paire formée par les valeurs des variables d'instance **token** des deux objets **Token** concernés :

```
createPairWith: aToken in: aBPE
```

```
aBPE
```

```
addNewPairFrom: self
```

```
and: aToken
```

La création effective de la paire est réalisée par la méthode **addNewPairFrom:and:**, définie dans la classe **BPETrain**.

Cette méthode reçoit deux instances de **Token** correspondant aux deux symboles de la paire.

Elle construit tout d'abord la paire formée par les valeurs des variables d'instance **token** de ces deux objets.

Cette paire est ensuite ajoutée dans **pairFrequencies**. La fréquence ajoutée correspond à la valeur de la variable d'instance **occurrence** de l'un des deux objets **Token**. Les deux objets possèdent en effet la même fréquence puisqu'ils appartiennent au même mot.

La méthode met également à jour **pairPositions**. Pour chaque occurrence de la paire, un couple de références vers les deux nœuds correspondants de la liste doublement chaînée est enregistré. Si la paire n'est pas encore présente dans le dictionnaire, un nouvel ensemble est créé. Sinon, le couple de références est ajouté à l'ensemble déjà associé à cette paire.

```
addNewPairFrom: aToken and: aToken1
```

```
pairFrequencies
```

```
add: { aToken token . aToken1 token }
```

```
withOccurrences: aToken occurrence .
```

```
pairPositions
```

```
at: { aToken token . aToken1 token }
```

```
update: [:set|
```

```
set add: { aToken position . aToken1 position } ;
```

```
yourself ]
```

```
initial: (Set with: { aToken position . aToken1 position })
```

Ainsi, `pairFrequencies` contient le nombre d'occurrences de chaque paire de symboles dans le corpus, tandis que `pairPositions` associe à chaque paire l'ensemble des références vers ses occurrences dans la liste doublement chaînée `corpus`.

L'utilisation du polymorphisme entre les classes `Token` et `EndOfWord` permet d'éviter la présence de tests conditionnels explicites lors de la construction des dictionnaires. Les paires impliquant une frontière de mot sont naturellement ignorées grâce au comportement défini dans la classe `EndOfWord`.

En résumé, La méthode `computePairs` parcourt l'ensemble des nœuds de la liste doublement chaînée représentant le corpus. Pour chaque paire de nœuds consécutifs, elle tente de construire une paire de symboles. Une paire n'est effectivement créée que lorsque les deux valeurs des nœuds concernées sont des instances de la classe `Token`. Les informations relatives à cette paire sont alors ajoutées dans les structures `pairFrequencies` et `pairPositions`, qui seront utilisées durant les différentes étapes de l'entraînement du BPE.

5.2.8 Réalisation d'une étape de fusion

La méthode `mergesOneStep` réalise une itération de l'algorithme d'entraînement du BPE. Une itération correspond à l'apprentissage d'une nouvelle fusion.

Son implémentation dans la classe `BPETrain` est la suivante :

```
mergesOneStep

| pair newVocabulary |

pair := self findPairToMerge.

newVocabulary := self createNewVocabulary: pair .

self addToMergesPair: pair
    mergedInto: newVocabulary .

self addToVocabulary: newVocabulary .

self update: pair
```

La première étape consiste à sélectionner la paire de symboles ayant la fréquence maximale dans `pairFrequencies`. Cette opération est réalisée par la méthode `findPairToMerge`. La paire obtenue correspond à la paire qui sera fusionnée lors de cette itération.

Les deux symboles composant cette paire sont ensuite fusionnés afin de former un nouveau symbole. Cette opération est réalisée par la méthode `createNewVocabulary:` qui construit la représentation du symbole obtenu après fusion.

La paire sélectionnée ainsi que le nouveau symbole créé sont ensuite ajoutés à la variable d'instance `merges` grâce à la méthode `addToMergesPair:mergedInto:`. Cette structure mémorise toutes les fusions apprises ainsi que leur ordre d'apprentissage, ordre qui devra être respecté lors de la phase d'encodage.

Le nouveau symbole est également ajouté au vocabulaire à l'aide de la méthode `addToVocabulary:`. Le vocabulaire est ainsi enrichi progressivement au fur et à mesure de l'apprentissage.

Enfin, la méthode `update:` est appelée afin d'appliquer la fusion dans `corpus`. Cette méthode remplace chaque occurrence de la paire sélectionnée par le nouveau symbole fusionné et met à jour les structures de données utilisées par l'algorithme.

En effet, la fusion modifie localement `corpus` : certaines paires de symboles consecutives disparaissent, tandis que de nouvelles paires de symboles consecutives sont créées à proximité des occurrences fusionnées. Les dictionnaires `pairFrequencies` et `pairPositions` doivent donc être mis à jour afin de refléter ces modifications. Cette mise à jour est réalisée uniquement sur les paires affectées par la fusion, ce qui constitue le principe de l'optimisation proposée.

5.2.8.1 Mise à jour du corpus du corpus, de `pairFrequencies` et de `pairPositions` dans `BPETrain` après une fusion

La mise à jour de la liste doublement chaînée `corpus` ainsi que des dictionnaires `pairFrequencies` et `pairPositions` après la fusion d'une paire de symboles est réalisée par la méthode `update:` définie dans la classe `BPETrain`.

```
update: pair

| position |

position := pairPositions at: pair.

position do: [ :each |

    each first value
    tryToUpdatePairWith: each first previousLink value
    in: self.

    each second nextLink value
    tryToUpdatePairWith: each second value
    in: self.

    self fusion: each.

    each first value
    tryToCreatePairWith: each first previousLink value
```

```

in: self.

each first nextLink value
tryToCreatePairWith: each first value
in: self ].

pairFrequencies
    removeKey: pair
    ifAbsent: [ ].
pairPositions
    removeKey: pair
    ifAbsent: [ ].

```

Cette méthode reçoit en paramètre la paire à fusionner. Elle récupère tout d'abord dans `pairPositions` l'ensemble des occurrences de cette paire présentes dans le corpus. Chaque occurrence est représentée par un couple de références vers deux nœuds consécutifs de la liste doublement chaînée. Ces références permettent d'accéder directement aux nœuds concernés sans avoir à parcourir l'ensemble du corpus pour retrouver chaque occurrence de la paire.

La méthode traite ensuite successivement chacune de ces occurrences. La fusion remplace les deux symboles formant la paire par un nouveau symbole représentant leur concaténation. Cette opération modifie localement la structure du `corpus` et entraîne la disparition de certaines paires de symboles adjacents ainsi que l'apparition de nouvelles paires. Afin de maintenir la cohérence des dictionnaires `pairFrequencies` et `pairPositions`, il est donc nécessaire de mettre à jour les informations associées aux paires affectées par cette modification.

Pour une occurrence donnée notée $p = (p_1, p_2)$, seules les paires situées dans le voisinage immédiat de l'occurrence peuvent être modifiées. Plus précisément, il s'agit de la paire formée par les valeurs du nœud précédant du nœud référencé par p_1 et du nœud référencé par p_1 , ainsi que de la paire formée par les valeurs du nœud référencé par p_2 et du nœud qui le suit. Après la fusion, la première paire disparaît car la valeur de la variable d'instance `token` de la valeur contenue dans p_1 est remplacée par le nouveau symbole obtenu après la fusion. La seconde paire disparaît également car le nœud p_2 est supprimé de la liste doublement chaînée `corpus`. Les informations associées à ces deux paires doivent donc être retirées de `pairFrequencies` et de `pairPositions`.

Avant d'effectuer cette suppression, il est nécessaire de vérifier que la paire formée par les valeurs de deux nœuds consécutifs forment une paire valide. Une paire est considérée comme valide lorsque les valeurs contenues dans les deux nœuds sont des instances de `Token`. En effet, les nœuds du corpus peuvent également contenir une instance de `EndOfWord`, utilisée pour marquer la fin d'un mot, ou ne pas exister lorsqu'un nœud se trouve à l'une des extrémités de la liste doublement chaînée. Afin d'éviter la multiplication de tests conditionnels dans le code, cette vérification repose sur le polymorphisme.

La méthode `tryToUpdatePairWith:in:` est utilisée pour déterminer si une paire doit effectivement être supprimée de `pairFrequencies` et `pairPositions`. Elle reçoit en paramètre un objet correspondant à la valeur du nœud référencé par p_1 (respectivement la valeur du nœud suivant du nœud référencé par p_2) ainsi que l'instance courante de `BPETrain`. Elle est envoyée à la valeur du nœud précédant du nœud référencé par p_1 (resp la valeur du nœud référencé par p_2). Son comportement dépend de la classe de l'objet qui reçoit le message.

Lorsque l'objet receveur est une instance de `EndOfWord`, aucune opération n'est effectuée. En effet, une fin de mot ne peut pas participer à une paire valide et aucune mise à jour n'est nécessaire dans ce cas.

```
tryToUpdatePairWith: anObject in: aBPE
```

```
  ^ self
```

Lorsque l'objet receveur est une instance de `Token`, la méthode vérifie tout d'abord que le son premier parametre existe. Si ce paramètre est `nil`, cela signifie qu'aucune paire valide ne peut être formée et aucune opération n'est réalisée. Dans le cas contraire, le traitement est délégué à ce paramètre au moyen de la méthode `updatePairWith:in:`.

```
tryToUpdatePairWith: anObject in: aBPE
```

```
  anObject ifNil: [ ^ self ].
```

```
  anObject
```

```
    updatePairWith: self
```

```
    in: aBPE
```

Cette délégation permet de poursuivre le traitement uniquement lorsque les deux éléments susceptibles de former une paire existent. La méthode `updatePairWith:in:` est alors responsable de déterminer si la mise à jour peut effectivement être réalisée.

Comme précédemment, son comportement dépend de la classe de l'objet qui reçoit le message. Lorsqu'il s'agit d'une instance de `EndOfWord`, aucune opération n'est effectuée puisque la présence d'une fin de mot empêche la formation d'une paire valide.

```
updatePairWith: aToken in: aBPE
```

```
  ^ self
```

Lorsque l'objet receveur est une instance de `Token`, les deux éléments considérés comme susceptibles de former une paire sont nécessairement des tokens valides. La mise à jour de la

paire peut alors être effectuée. La responsabilité de cette mise à jour est déléguée à son deuxième paramètre qui est une instance de `BPETrain`, qui possède les structures `pairFrequencies` et `pairPositions`.

```
updatePairWith: aToken in: aBPE
```

```
aBPE
```

```
    updatePairFrom: self  
    and: aToken
```

La méthode `updatePairFrom:and:` constitue le point central de la suppression d'une paire affectée par une fusion. Les filtrages réalisés par les méthodes précédentes garantissent que les deux paramètres reçus sont des instances de `Token`. La méthode peut donc effectuer directement les mises à jour nécessaires sans réaliser de vérifications supplémentaires.

```
updatePairFrom: aToken and: aToken1
```

```
pairFrequencies
```

```
    add: {aToken token . aToken1 token }  
    withOccurrences: aToken occurrence negated .
```

```
(pairFrequencies occurrencesOf: {aToken token . aToken1 token }) = 0 ifTrue: [  
    pairFrequencies  
        removeKey: {aToken token . aToken1 token }  
        ifAbsent: [].
```

```
pairPositions
```

```
    at: {aToken token . aToken1 token }  
    update: [:set|  
        set  
            remove: {aToken position . aToken1 position }  
            ifAbsent: [];  
            yourself ].
```

```
pairPositions
```

```
    at: {aToken token . aToken1 token }) size = 0 ifTrue: [  
        pairPositions removeKey: {aToken token . aToken1 token }]
```

Elle commence par construire la paire à partir des valeurs de l'attribut `token` des deux objets reçus en paramètre. Cette paire correspond à celle qui disparaît du corpus à la suite de la fusion.

La fréquence associée à cette paire dans `pairFrequencies` est décrémentée de la valeur de l'attribut `occurrence` de l'un des deux objets reçus en paramètre, et non de 1, car `corpus` ne stocke pas les occurrences des mots sous forme répétée.

Le texte original est représenté par une unique liste doublement chaînée de nœuds, dans laquelle les mots sont délimités par des nœuds `EndOfWord`. Chaque mot correspond ainsi à un segment de cette liste.

Tous les nœuds appartenant à un même segment contiennent des valeurs de type `Token`, partageant la même valeur de l'attribut `occurrence`, qui correspond au nombre d'apparitions du mot dans le texte original.

Ainsi, lorsqu'une paire de symboles est supprimée à la suite d'une fusion, cette suppression est effectuée au niveau du segment correspondant au mot concerné. Bien que l'opération soit réalisée sur une seule représentation du mot dans la liste doublement chaînée, elle est pondérée par sa fréquence afin de refléter toutes ses occurrences dans le texte original. C'est pour cette raison que la mise à jour de la valeur associée à la paire dans `pairFrequencies` doit tenir compte de la valeur de `occurrence`.

Les deux objets reçus en paramètre possèdent nécessairement la même valeur pour l'attribut `occurrence`, car ils appartiennent au même segment de mot délimité par des nœuds `EndOfWord`.

La méthode met également à jour la valeur associée à la paire dans `pairPositions` en supprimant le couple formé par les valeurs de l'attribut `position` des deux objets reçus en paramètre, afin de refléter la disparition de cette occurrence de la paire dans la liste doublement chaînée après la fusion.

Après avoir mis à jour les structures `pairFrequencies` et `pairPositions` pour un couple de référence $p = (p_1, p_2)$ représentant une occurrence de la paire à fusionner dans la liste doublement chaînée `corpus`, où p_1 et p_2 désignent des références vers deux nœuds consécutifs dont les valeurs sont des instances de `Token` représentant les symboles de la paire, la méthode procède au remplacement de cette occurrence par le résultat de la fusion.

Cette opération est réalisée par la méthode `fusion`, définie dans la classe `BPETrain`, qui reçoit en paramètre le couple (p_1, p_2) .

La fusion consiste à remplacer la valeur de l'attribut `token` de la valeur contenu dans le nœud référencé par p_1 par la concaténation des valeurs des attributs `token` des valeurs contenus dans les nœuds référencé par p_1 et p_2 . Le nœud référencé par p_2 est ensuite supprimé de la liste doublement chaînée `corpus`, ce qui réduit la représentation de la paire à un seul symbole.

```
fusion: anArray
```

```
anArray first value token: (anArray first value token , anArray second value token ) .
```

```
corpus removeLink: anArray second .
```

La fusion entraîne également l'apparition de nouvelles paires dans le voisinage immédiat du symbole obtenu. Ces nouvelles paires sont formées, d'une part, par les valeurs contenues dans

le nœud précédant le nœud référencé par p_1 et dans le nœud référencé par p_1 , et, d'autre part, par les valeurs contenues dans le nœud référencé par p_1 et dans son nœud suivant. Ces paires correspondent aux nouveaux couples de symboles adjacents créés par la fusion.

Comme ces paires n'étaient pas nécessairement présentes avant la fusion, leurs informations doivent être ajoutées dans les dictionnaires `pairFrequencies` et `pairPositions`. Toutefois, avant d'effectuer cet ajout, il est nécessaire de vérifier que les deux éléments formant chaque paire constituent une paire valide.

Une paire est considérée comme valide lorsque les valeurs contenues dans les deux nœuds concernés sont des instances de `Token`. En effet, les valeurs des nœuds de `corpus` peuvent être une instance de `Token`, une instance de `EndOfWord` ou être inexistantes (`nil`). Dans les deux derniers cas, aucune paire valide ne peut être formée et aucune mise à jour ne doit être réalisée.

Afin d'éviter l'utilisation répétée de tests conditionnels, l'ajout des nouvelles paires repose également sur le polymorphisme. La méthode `tryToCreatePairWith:in:` est utilisée à cet effet. Elle est envoyée à la valeur du nœud précédant le nœud référencé par p_1 (respectivement à la valeur du nœud référencé par p_1) et reçoit en paramètre la valeur du nœud référencé par p_1 (respectivement la valeur de son nœud suivant), ainsi que l'instance courante de `BPETrain`. Le comportement de la méthode dépend de la classe de l'objet qui reçoit le message. Selon que cet objet est une instance de `Token`, de `EndOfWord` ou qu'il n'existe pas, l'ajout de la paire sera effectué ou ignoré. Le fonctionnement détaillé de l'ajout des nouvelles paires au moyen de la méthode `tryToCreatePairWith:in:` est présenté dans la Section 5.2.7.

5.2.9 Application des fusions apprises avec `BPEEncoder`

Après avoir présenté les différentes étapes de l'entraînement de BPE dans la classe `BPETrain`, nous nous intéressons maintenant à l'application des fusions apprises sur un nouveau texte. Cette tâche est assurée par la classe `BPEEncoder`, qui contient les méthodes nécessaires à l'encodage d'un texte à partir des règles de fusion produites lors de l'entraînement.

La classe `BPEEncoder` diffère de `BPETrain` sur plusieurs points. Tout d'abord, lors de l'encodage, il n'est pas nécessaire de regrouper les mots identiques ni de calculer leur fréquence, car l'ordre des mots du texte doit être conservé. De plus, l'objectif n'est plus d'apprendre de nouvelles fusions, mais d'appliquer celles qui ont déjà été apprises.

Alors que `BPETrain` cherche à déterminer les fusions les plus fréquentes dans un corpus d'entraînement, `BPEEncoder` applique simplement une séquence de fusions déjà apprise. Aucune décision n'est donc prise durant cette phase : les règles sont appliquées dans l'ordre où elles ont été apprises lors de l'entraînement.

Par conséquent, les fréquences des paires ne sont plus utilisées durant cette phase. Seul le dictionnaire `pairPositions` est construit, car il permet de localiser les occurrences des paires auxquelles les règles de fusion doivent être appliquées. Après chaque fusion, seules les structures `corpus` et `pairPositions` sont mises à jour.

La classe `BPEEncoder` est une sous-classe de `BPETrain`. Elle hérite donc de ses variables d'instance ainsi que de la majorité de ses méthodes. Certaines méthodes sont toutefois redéfinies

afin de tenir compte des spécificités de la phase d'encodage.

La méthode `corpus:` est modifiée de manière à ne plus calculer la fréquence des mots du texte. Chaque occurrence d'un mot est conservée dans le corpus afin de préserver l'ordre du texte original.

```
corpus: anObject

| wordSequences |
wordSequences := Preprocessing preprocess: anObject .

wordSequences do: [ :word|
    word do: [ :char |

        corpus add: (Token token: char asString) .

        corpus lastLink value position: corpus lastLink ].

    corpus add: (Token token: '</t>').

    corpus lastLink value position: corpus lastLink .

    corpus add: EndOfWord new ].

^ corpus
```

La méthode `addNewPairFrom:and:` est également redéfinie. Dans `BPETrain`, elle ajoute une nouvelle paire à la fois dans `pairFrequencies` et dans `pairPositions`. Dans `BPEEncoder`, la mise à jour concerne uniquement `pairPositions`, puisque les fréquences des paires ne sont pas utilisées pendant l'encodage.

```
addNewPairFrom: aToken and: aToken1

pairPositions
    at: { aToken token . aToken1 token}
    update: [:set|
        set add: { aToken position . aToken1 position };
        yourself]
    initial: (Set with: { aToken position . aToken1 position })
```

De même, la méthode `updateNewPairFrom:` ne met à jour que `pairPositions`.

```
updatePairFrom: aToken and: aToken1
```

```
pairPositions at: { aToken token . aToken1 token }  
  update: [:set|  
    set remove: { aToken position . aToken1 position }  
    ifAbsent: [] ;  
    yourself].
```

```
(pairPositions at: {aToken token . aToken1 token }) size = 0 ifTrue: [  
  pairPositions removeKey: { aToken token . aToken1 token } ]
```

Les autres méthodes héritées de `BPETrain` sont conservées sans modification, leur comportement restant valide pour l'application des règles de fusion.

Enfin, la classe `BPEEncoder` introduit la méthode `applyRulesTo:from:`, qui permet d'appliquer à un texte l'ensemble des règles de fusion contenues dans un fichier.

```
applyRulesTo: inputText from: filePath
```

```
|tokens|  
self corpus: inputText .  
self computePairs .  
self loadMergesSTON: filePath .  
tokens := OrderedCollection new .  
  
merges do: [ :each |  
  |pair|  
  pair := each key .  
  ( pairPositions includesKey: pair ) ifTrue: [ self update: pair ] ].  
  
corpus linksDo: [ :link |  
  ( link value class ~= EndOfWord ) ifTrue: [ tokens add: link value token ] ].  
  
^ tokens
```

Cette méthode reçoit en paramètre le texte à encoder ainsi que le chemin du fichier contenant les règles de fusion apprises.

Elle commence par construire la variable d'instance `corpus` à partir du texte fourni en entrée. Contrairement à la phase d'entraînement, les mots ne sont pas regroupés selon leurs fréquences afin de préserver l'ordre du texte original.

La méthode construit ensuite le dictionnaire `pairPositions` à l'aide de la méthode `computePairs`. Ce dictionnaire permet de localiser les occurrences des paires présentes dans le corpus et susceptibles d'être fusionnées.

Une liste vide est ensuite initialisée. Elle servira à stocker les tokens obtenus après l'application de l'ensemble des règles de fusion.

La méthode charge alors le fichier contenant les fusions apprises et initialise la variable `merges` avec la liste des règles ainsi récupérées. Les éléments de cette liste sont parcourus dans l'ordre où ils ont été appris durant l'entraînement. Cet ordre est important, car l'application des règles de fusion doit reproduire exactement le processus ayant conduit à la construction du vocabulaire BPE.

Pour chaque fusion de `merges`, la méthode vérifie si la paire correspondante est présente dans `pairPositions`. Si c'est le cas, les occurrences de cette paire sont fusionnées dans le `corpus`. Cette opération est réalisée par la méthode `update`, qui met à jour la liste doublement chaînée `corpus` ainsi que le dictionnaire `pairPositions` afin de refléter la disparition des anciennes paires et l'apparition des nouvelles paires créées par la fusion. L'utilisation de `pairPositions` permet d'accéder directement aux occurrences de la paire à fusionner sans avoir à parcourir l'ensemble du corpus pour les retrouver. De plus, seules les paires situées dans le voisinage immédiat des occurrences fusionnées sont mises à jour. Cette approche évite de reconstruire entièrement les structures de données après chaque fusion et réduit ainsi le coût de l'encodage.

Une fois toutes les règles appliquées, la méthode parcourt les nœuds de `corpus`. Pour chaque nœud dont la valeur est une instance de `Token`, la valeur de son attribut `token` est ajoutée à la liste précédemment initialisée. Les nœuds contenant une instance de `EndOfWord` sont ignorés, puisqu'ils servent uniquement à délimiter les mots dans la représentation interne du corpus.

La méthode renvoie finalement cette liste de tokens, qui correspond à la représentation BPE du texte après l'application de l'ensemble des règles de fusion apprises.

5.3 Optimisation de BPE par fusion de paires en lots

5.3.1 Idée sur l'optimisation de BPE par fusion de paires en lots

Une autre piste d'optimisation de BPE consiste à effectuer plusieurs fusions simultanément à chaque étape de l'entraînement. Cette approche, appelée fusion par lots (batch merging), vise à réduire le temps d'exécution de l'algorithme. En effet, dans BPE classique, une seule paire est fusionnée à chaque itération. Il est alors nécessaire de rechercher à nouveau la paire la plus fréquente après chaque fusion.

L'idée est de sélectionner, à chaque itération, un ensemble de paires à fusionner simultanément. Dans notre approche, les paires candidates sont choisies parmi celles possédant la fréquence maximale dans `pairFrequencies`. La fréquence maximale est d'abord déterminée, puis toutes les paires possédant cette fréquence sont considérées pour une éventuelle inclusion dans le lot.

Sélection de fusions sûres

Il n'est cependant pas possible de fusionner systématiquement toutes les paires de fréquence maximale. En effet, certaines paires peuvent interférer entre elles. Lorsqu'une paire est fusionnée, elle modifie le corpus et peut entraîner la disparition ou la transformation d'occurrences d'autres paires. Deux fusions réalisées simultanément doivent donc être compatibles.

Afin de garantir la cohérence du processus, seules des fusions sûres sont autorisées dans un même lot. Deux paires sont considérées comme compatibles si leur fusion simultanée ne peut pas empêcher l'application de l'une ou de l'autre.

Pour vérifier cette propriété, lorsqu'une nouvelle paire est examinée, il faut s'assurer que :

- son premier élément n'apparaît pas comme second élément d'une paire déjà retenue dans le lot ;
- son second élément n'apparaît pas comme premier élément d'une paire déjà retenue dans le lot.

Si l'une de ces conditions n'est pas respectée, la paire est rejetée et ne peut pas être ajoutée au lot.

Respect de l'ordre des fusions

La sélection de fusions sûres introduit toutefois une difficulté supplémentaire. Dans BPE classique, lorsque plusieurs paires possèdent la même fréquence maximale, elles sont fusionnées selon un ordre déterminé par leur apparition dans la structure contenant les fréquences. Lors de la construction d'un lot, certaines de ces paires peuvent être rejetées en raison d'un conflit avec des paires déjà sélectionnées.

Or, une paire candidate qui n'entre pas en conflit avec les éléments actuellement présents dans le lot pourrait néanmoins entrer en conflit avec une paire plus prioritaire ayant été rejetée précédemment. Si cette contrainte n'est pas prise en compte, l'ordre des fusions obtenu pourrait différer de celui produit par BPE classique.

Pour limiter ce problème, les paires rejetées sont également mémorisées lors de la construction du lot. Lorsqu'une nouvelle paire est examinée, il faut vérifier qu'elle n'entre pas seulement en conflit avec les paires déjà sélectionnées, mais également avec les paires plus fréquentes ou plus prioritaires qui ont été rejetées auparavant. Cette stratégie permet de mieux préserver l'ordre des fusions qui aurait été obtenu dans l'algorithme séquentiel.

Limites de l'approche

L'entraînement par lots reste néanmoins une approximation du comportement de BPE classique.

En effet, lorsqu'une paire est fusionnée, elle crée de nouveaux symboles et donc de nouvelles paires. Certaines de ces nouvelles paires peuvent devenir très fréquentes immédiatement après leur création. Dans BPE classique, elles peuvent parfois être sélectionnées dès les itérations suivantes. En revanche, dans une approche par lots, les décisions de fusion sont prises avant que ces nouvelles fréquences ne soient connues.

De plus, les paires rejetées lors de la construction du lot continuent à influencer la sélection des autres paires afin de préserver l'ordre des fusions. Or, après l'application effective des fusions retenues, la fréquence de ces paires rejetées peut diminuer significativement, voire elles peuvent

disparaître complètement. Elles continuent pourtant à contraindre la composition du lot alors qu'elles ne constituent plus nécessairement des candidates pertinentes à la fusion.

Ces phénomènes peuvent conduire à un vocabulaire différent de celui obtenu par BPE classique. Toutefois, ils permettent généralement d'obtenir un gain de temps important tout en conservant une qualité de tokenisation comparable.

Paramétrage de la taille des lots

La taille du lot influence directement le compromis entre rapidité d'exécution et fidélité au comportement de BPE classique. Des lots de grande taille permettent d'effectuer davantage de fusions à chaque itération et réduisent ainsi le temps d'entraînement. En revanche, ils augmentent le risque de s'éloigner de l'ordre des fusions produit par l'algorithme séquentiel.

Pour cette raison, la taille du lot est laissée configurable par l'utilisateur. Celui-ci peut ainsi choisir un compromis adapté à ses besoins entre vitesse d'entraînement et proximité avec le comportement du BPE classique.

5.3.2 Fonctionnement

1. L'algorithme commence par découper le texte en une liste de mots. Les espaces, les tabulations et les retours à la ligne sont utilisés comme séparateurs.
2. La fréquence de chaque mot est ensuite calculée.
3. Une représentation du corpus à l'aide d'une liste doublement chaînée est alors construite. Les détails sur la construction de cette représentation sont présentés dans la section **5.1.2.1**.
4. À partir de cette représentation, l'algorithme construit :
 - le vocabulaire initial ;
 - le dictionnaire des fréquences des paires, qui associe à chaque paire sa fréquence dans le corpus ;
 - le dictionnaire des positions, qui associe à chaque paire l'ensemble des références vers les nœuds représentant ses occurrences dans la liste doublement chaînée.

La construction de ces structures est également décrite dans la section **5.1.2.1**.

5. L'entraînement peut alors commencer.

Afin de constituer un lot de fusions sûres, une paire candidate ne doit pas entrer en conflit avec les paires déjà examinées, qu'elles aient été retenues ou rejetées. Pour effectuer cette vérification efficacement, deux ensembles sont utilisés :

- un ensemble contenant les premiers éléments des paires déjà examinées ;
- un ensemble contenant les seconds éléments des paires déjà examinées.

Lorsqu'une paire est examinée, son premier élément ne doit pas apparaître dans l'ensemble des seconds éléments déjà vus et son second élément ne doit pas apparaître dans l'ensemble des premiers éléments déjà vus. Si ces deux conditions sont satisfaites, la paire est considérée comme sûre et peut être fusionnée.

Quelle que soit l'issue du test, les deux éléments de la paire examinée sont ajoutés aux

ensembles correspondants. Cette stratégie permet de garantir que les paires retenues ne sont pas en conflit avec les paires plus prioritaires déjà examinées, y compris celles qui ont été rejetées.

Dans cette implémentation, aucun lot explicite n'est construit. Dès qu'une paire est jugée sûre, sa fusion est immédiatement appliquée. Cette approche évite de parcourir une seconde fois les paires sélectionnées afin d'exécuter les fusions.

L'entraînement se poursuit tant que la taille cible du vocabulaire n'est pas atteinte et que le dictionnaire des fréquences n'est pas vide.

À chaque itération, les opérations suivantes sont réalisées :

- (a) Deux ensembles vides sont créés : l'ensemble des premiers éléments des paires examinées et l'ensemble des seconds éléments des paires examinées.
- (b) Les paires présentes dans le dictionnaire des fréquences des paires sont triées par ordre décroissant de fréquence. Le résultat est stocké dans la variable *pairesTrieesParFrequence*.
- (c) La fréquence maximale est obtenue à partir du premier élément de *pairesTrieesParFrequence* et stockée dans la variable *frequenceMax*.
- (d) Les paires de *pairesTrieesParFrequence* dont la fréquence est égale à *frequenceMax* sont ensuite examinées successivement.

Pour chaque paire examinée, si son premier élément n'appartient pas à l'ensemble des seconds éléments déjà vus et si son second élément n'appartient pas à l'ensemble des premiers éléments déjà vus alors la paire est fusionnée.

La fusion consiste à :

- créer un nouveau symbole représentant la paire ;
- ajouter ce symbole au vocabulaire ;
- enregistrer l'association entre la paire et son symbole dans la structure contenant les règles de fusion ;
- remplacer dans la liste doublement chaînée toutes les occurrences de la paire par le nouveau symbole ;
- mettre à jour les dictionnaires des fréquences des paires et des positions des paires afin de refléter les modifications apportées au corpus.

Le détail de la mise à jour de ces structures est présenté dans la section **5.1.2.1**.

Que la paire soit fusionnée ou rejetée, ses deux éléments sont ajoutés aux ensembles des premiers et des seconds éléments déjà vus.

Le même traitement est ensuite appliqué à la paire suivante de fréquence maximale.

L'examen des paires se poursuit tant que :

- la fréquence de la paire courante est égale à *frequenceMax* ;
- la taille cible du vocabulaire n'est pas atteinte ;
- le nombre maximal de paires à examiner pour le lot courant n'est pas atteint ;
- et qu'il reste des éléments dans *pairesTrieesParFrequence*.

5.3.3 Implémentation

La méthode de classe `trainOn: vocabularySize: batchSize:` implémentée dans `BPETrain` permet de réaliser l'entraînement de BPE par fusion par lots. Elle prend en entrée un texte d'entraînement, une taille cible de vocabulaire et un paramètre *batchSize* correspondant au nombre maximal de paires de fréquence maximale examinées lors de chaque itération. Parmi ces paires, seules celles qui ne sont pas en conflit avec les paires déjà examinées sont effectivement fusionnées. La méthode retourne une instance de `BPETrain` contenant le vocabulaire appris et les règles de fusion produites durant l'entraînement.

Son implémentation est présentée ci-dessous.

```
trainOn: aText vocabularySize: n batchSize: k

|bpe| bpe := self new.
bpe fromText: aText.

[ bpe vocabulary size < n
and: [ bpe pairFrequencies notEmpty ] ] whileTrue: [
|seenFirstTokens seenSecondTokens sortedPairsByFrequency maxFrequency i|

seenFirstTokens := Set new.
seenSecondTokens := Set new.
sortedPairsByFrequency := bpe pairFrequencies sortedCounts.
maxFrequency := sortedPairsByFrequency first key.
i := 1.

[ ( i <= ( sortedPairsByFrequency size min: k ))
and: [ (sortedPairsByFrequency at: i)key = maxFrequency
and: [ bpe vocabulary size < n ] ] ] whileTrue: [ |pair|
pair := (sortedPairsByFrequency at: i) value.
( (seenFirstTokens includes: pair second)
or: [seenSecondTokens includes: pair first] ) ifFalse: [
bpe mergePairAndUpdateStructures: pair ].
seenFirstTokens add: pair first.
seenSecondTokens add: pair second.
i:= i+1 ].].
^bpe
```

Les paires sont stockées dans `sortedPairsByFrequency`, une collection contenant les éléments de `pairFrequencies` triés par ordre décroissant de fréquence. Lors du parcours de cette collection, seules les premières paires sont examinées.

Le nombre maximal de paires à examiner est déterminé par le paramètre `batchSize`. Cependant, lorsque le nombre de paires présentes dans `sortedPairsByFrequency` est inférieur à cette valeur, il n'est pas possible d'examiner davantage de paires que celles effectivement disponibles. Pour cette raison, la borne supérieure de l'indice est définie comme le minimum entre `batchSize` et la taille de `sortedPairsByFrequency`.

Cette précaution permet d'éviter toute tentative d'accès à un élément inexistant de la collection, ce qui provoquerait une erreur d'exécution.

L'indice est initialisé à 1 car les collections de Pharo sont indexées à partir de 1. De plus, comme les paires sont triées par ordre décroissant de fréquence, les premiers éléments de `sortedPairsByFrequency` correspondent aux paires les plus fréquentes et sont donc examinés en priorité.

5.4 Benchmarking

Cette section présente le benchmark des différentes implémentations du BPE (Byte Pair Encoding) développées dans ce travail.

5.4.1 Entraînement

Chaque implémentation a été exécutée 100 fois pour chaque taille de vocabulaire n afin de réduire l'impact des fluctuations liées au système d'exploitation et au garbage collector.

Les temps d'exécution obtenus ont ensuite été moyennés (division par 100) afin d'obtenir des valeurs représentatives, reportées dans les tableaux et figures ci-dessous.

Les expérimentations ont été réalisées sur deux types de données issues du système Pharo : les noms de classes et les noms de méthodes.

Le tableau 5.1 et la figure 5.4 présentent les temps d'exécution des différentes implémentations du BPE sur les noms de classes.

n	BytePairEncoder	BPETrain($k = 1$)	BPETrain($k = 3$)	BPETrain($k = 6$)	BPETrain($k = 10$)	BPETrain($k = 20$)
50	29.42 ms	21.54 ms	18.8 ms	20.43 ms	19.55 ms	19.3 ms
100	213.89 ms	53.11 ms	48.13 ms	49.33 ms	48.65 ms	50.44 ms
200	482.72 ms	88.73 ms	66.45 ms	62.57 ms	59.37 ms	60.27 ms
400	846.82 ms	131.46 ms	87.69 ms	77.29 ms	72.23 ms	73.89 ms
600	1 074.85 ms	159.5 ms	95.26 ms	79.86 ms	75.44 ms	73.6 ms
807	1 226.99ms	169.63 ms	101.24 ms	87.75 ms	77.81 ms	74.75 ms

TABLE 5.1 – Benchmark des temps d'entraînement des implémentations du BPE sur les noms de classes de Pharo en fonction de la taille du vocabulaire n .

Benchmark des temps d'entraînement des implémentations du BPE sur les noms de classes de Pharo

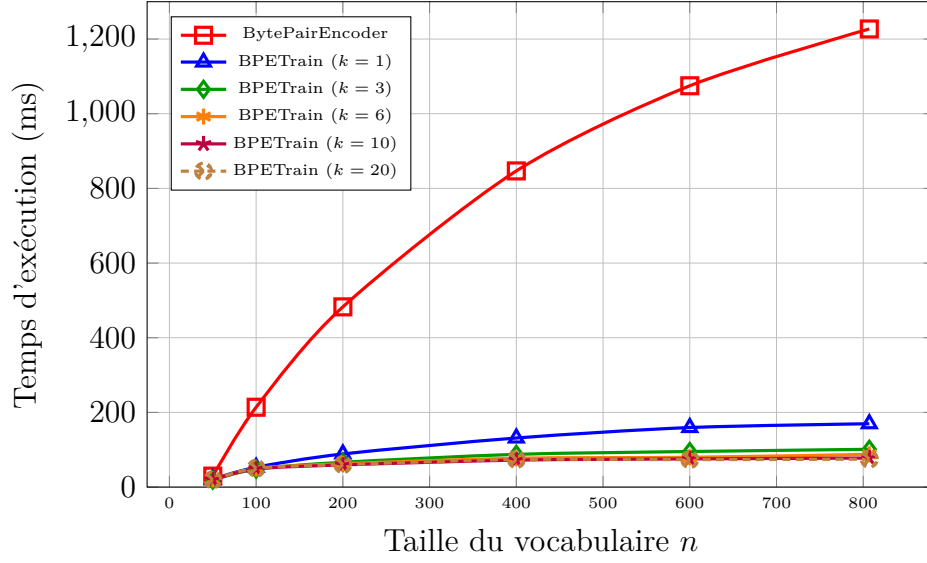


FIGURE 5.4 – Benchmark des temps d'entraînement des implémentations du BPE sur les noms de classes de Pharo en fonction de la taille du vocabulaire n .

On observe que les implémentations optimisées de **BPETrain** sont systématiquement plus rapides que l'implémentation naïve **BytePairEncoder**.

Le temps d'exécution augmente avec la taille du vocabulaire n pour toutes les implémentations. Cependant, les versions optimisées présentent une croissance beaucoup plus modérée. Les gains sont particulièrement visibles pour les grandes valeurs de n .

On remarque également que les variantes de **BPETrain** avec différents paramètres k offrent des performances proches, avec une légère amélioration pour certaines valeurs intermédiaires de k .

Le tableau 5.2 et la figure 5.5 présentent les temps d'exécution des différentes implémentations du BPE sur les noms de classes.

n	BytePairEncoder	BPETrain($k=1$)	BPETrain($k=3$)	BPETrain($k=6$)	BPETrain($k=10$)	BPETrain($k=20$)
50	31.72 ms	57.90 ms	63.48 ms	64.06 ms	58.67 ms	59.56 ms
100	1 159.5 ms	158.73 ms	162.87 ms	153.61 ms	151.42 ms	149.75 ms
200	2 976.07 ms	298.28 ms	258.13 ms	238.27 ms	224.13 ms	221.18 ms
400	5 773.95 ms	576.63 ms	390.35 ms	323.72 ms	298.48 ms	290.78 ms
600	8 168.77 ms	945.35 ms	477.84 ms	390.02 ms	343.29 ms	315.59 ms
800	10 694.04 ms	1 183.29 ms	586.68 ms	438.3 ms	373.34 ms	353.67 ms
1 000	13 100.72 ms	1 311.63 ms	668.6 ms	496.60 ms	402.44 ms	368.52 ms
1 500	16 472.68 ms	1 726.72 ms	812.56 ms	572.06 ms	456.5 ms	398.43 ms
2 000	20 925.20 ms	2 008.64 ms	939.84 ms	643.25 ms	487.48 ms	414.61 ms
2 500	21 979.98 ms	2 505.10 ms	1 035.93 ms	648.88 ms	538.79 ms	451.86 ms
3 000	23 927.86 ms	2 696.62 ms	1 119.42 ms	672.30 ms	539.92 ms	476.45 ms
3 519	25 254.48 ms	2 688.1 ms	1 120.46 ms	716.25 ms	541.03 ms	469.71 ms

TABLE 5.2 – Benchmark des temps d'entraînement des implémentations du BPE sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n .

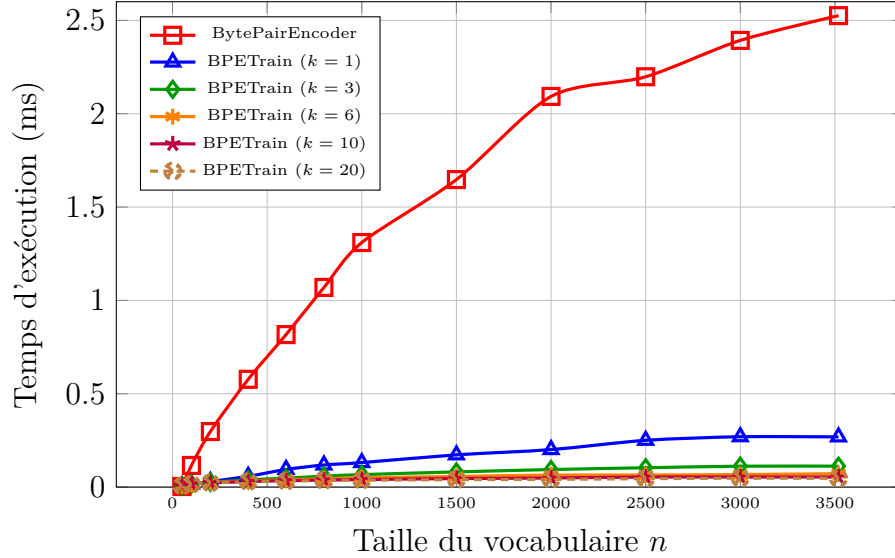


FIGURE 5.5 – Benchmark des temps d’entraînement des implémentations du BPE sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n .

Les mêmes tendances sont observées sur les noms de méthodes, mais avec des temps d’exécution globalement plus élevés, ce qui s’explique par une taille de corpus plus importante donc un nombre de paires candidates plus élevé.

Les implémentations optimisées restent significativement plus efficaces que l’algorithme naïf.

5.4.2 Encodage

Dans cette section, nous mesurons le temps d’exécution de l’étape d’encodage du Byte Pair Encoding (BPE) en comparant trois implémentations : une version naïve (**BytePairEncoder**), une version optimisée (**BPEEncoder**), et une seconde version optimisée intégrant une structure de priorité (**BPEEncoder (order)**).

La première version optimisée (**BPEEncoder**) applique directement les règles de fusion apprises lors de l’entraînement sur le texte à encoder. La seconde version (**BPEEncoder (order)**) associe à chaque paire du texte un ordre de priorité correspondant à la séquence des fusions apprises. À chaque itération, la paire ayant la priorité la plus élevée est sélectionnée et fusionnée. Cette version utilise une structure de type tas binaire (heap) afin de gérer efficacement les priorités des paires.

Le tableau 5.3 présente les temps d’exécution de l’encodage appliqué sur les noms de méthodes du système Pharo. La variable n représente le nombre de méthodes encodées.

n	BytePairEncoder	BPETrain	BPETrain(order)
20	10 ms	12 ms	12 ms
100	32 ms	19 ms	20 ms
500	174 ms	56 ms	61 ms
1083	447 ms	118 ms	113 ms

TABLE 5.3 – Comparaison des temps d’exécution de l’encodage BPE pour différentes implémentations sur les noms de méthodes de Pharo en fonction de la taille du corpus n .

On observe que les deux versions optimisées présentent des performances proches pour les petites tailles de corpus. Cependant, un écart apparaît à partir d’environ 500 éléments encodés.

Pour analyser plus finement ce comportement, nous avons décomposé le temps d’exécution :

— Pour **BPEEncoder** :

— Prétraitement : 42 ms

— Encodage : 51 ms

— Pour **BPEEncoder** (order) :

— Prétraitement : 57 ms

— Encodage : 68 ms

La méthode **BPEEncoder** (order) est plus coûteuse en raison de l’utilisation d’un tas binaire (heap), qui implique des opérations supplémentaires d’insertion et de suppression pour maintenir l’ordre de priorité des paires. Pour les très petites tailles de corpus, **BytePairEncoder** peut être plus rapide ou équivalent aux versions optimisées. Cela s’explique par le coût fixe du prétraitement et de la structure de données utilisée dans les implémentations optimisées, qui n’est pas amorti sur de petites entrées. En revanche, lorsque la taille du corpus augmente, les versions optimisées deviennent plus efficaces grâce à une meilleure gestion des opérations d’encodage.

Chapitre 6

Optimisation de WordPiece

Ce chapitre présente les optimisations apportées à notre implémentation de WordPiece. Deux aspects sont considérés : l'optimisation de l'entraînement et celle de l'encodage. L'optimisation de l'encodage est inspirée de l'article Fast WordPiece Tokenization, tandis que l'optimisation de l'entraînement repose sur le même principe que celui présenté pour BPE.

6.1 Optimisation de l'entraînement de WordPiece

6.1.1 Problème

À chaque itération de l'entraînement de WordPiece, la paire de symboles ayant le meilleur score est fusionnée afin de former un nouveau symbole.

Après chaque fusion, le corpus doit être mis à jour en remplaçant toutes les occurrences de la paire fusionnée par le nouveau symbole obtenu. Dans une implémentation naïve, cette mise à jour nécessite de parcourir tous les mots du corpus, y compris ceux qui ne contiennent aucune occurrence de la paire fusionnée.

La fusion modifie également les statistiques utilisées pour sélectionner la prochaine paire à fusionner. En effet, certaines paires disparaissent, d'autres voient leur fréquence diminuer, tandis que de nouvelles paires apparaissent. De même, la fréquence des deux symboles composant la paire fusionnée diminue, alors qu'un nouveau symbole apparaît avec une fréquence égale au nombre d'occurrences de la paire fusionnée.

Ainsi, trois traitements coûteux sont effectués à chaque itération : un premier parcours complet du corpus pour remplacer toutes les occurrences de la paire fusionnée, un deuxième parcours complet du corpus pour mettre à jour les fréquences des symboles et des paires, puis un parcours de l'ensemble des paires afin de recalculer leurs scores, y compris ceux des paires qui ne sont pas affectées par la fusion. Lorsque le corpus est de grande taille, ces traitements répétés augmentent considérablement le temps d'entraînement.

6.1.2 Solution

Afin d'éviter ces parcours complets du corpus après chaque fusion, nous proposons une représentation permettant de limiter les mises à jour aux seuls éléments affectés.

Dans un premier temps, les mots identiques du corpus sont regroupés à l'aide d'une instance de la classe **Bag**, ce qui permet de connaître la fréquence de chaque mot. Ainsi, un mot n'est représenté qu'une seule fois, accompagné de sa fréquence, évitant de répéter les mêmes traitements sur plusieurs occurrences identiques.

Chaque mot est ensuite représenté par un segment d'une **liste doublement chaînée**. Chaque nœud de cette liste contient un symbole du mot ainsi que la fréquence du mot auquel il appartient. Après le nœud contenant le dernier symbole de chaque mot, un nœud contenant une instance de la classe **EndOfWord** est inséré afin de délimiter les mots et d'empêcher toute fusion entre des symboles appartenant à deux mots différents. Cette représentation conserve l'ordre des symboles tout en permettant un accès direct aux voisins gauche et droit de chaque symbole.

Grâce à cette liste doublement chaînée, lorsqu'une paire est fusionnée, il est possible d'identifier immédiatement les paires dont la fréquence est modifiée. Il s'agit uniquement :

- de la paire formée par le voisin gauche du premier symbole et ce premier symbole, si ce voisin existe et n'est pas un nœud **EndOfWord** ;
- de la paire formée par le second symbole et son voisin droit, si ce voisin existe et n'est pas un nœud **EndOfWord**.

La liste doublement chaînée facilite également la fusion elle-même, puisqu'il suffit de modifier localement les liens entre les nœuds concernés sans parcourir l'ensemble du corpus.

Afin d'accéder directement aux occurrences de chaque paire, un dictionnaire associe chaque paire de symboles à un ensemble de couples de références vers les deux nœuds consécutifs contenant les symboles de chacune de ses occurrences dans la liste doublement chaînée. Cette structure permet de retrouver immédiatement toutes les occurrences d'une paire, d'effectuer leur fusion sans parcourir le corpus, puis de mettre à jour uniquement les paires dont la fréquence est effectivement modifiée.

Les fréquences des symboles et des paires sont stockées dans des instances de la classe **Bag**, ce qui permet de les mettre à jour efficacement après chaque fusion.

Cependant, la mise à jour des scores nécessite une attention particulière. En effet, le score d'une paire dépend non seulement de la fréquence de cette paire, mais également de la fréquence des deux symboles qui la composent. Après une fusion, la fréquence des deux symboles fusionnés diminue, tandis qu'un nouveau symbole apparaît. Par conséquent, le score de toutes les paires contenant l'un des deux symboles fusionnés peut être modifié, même si leur fréquence reste inchangée. Pour identifier efficacement ces paires, un second dictionnaire associe chaque symbole à l'ensemble des références vers les nœuds contenant ses occurrences dans la liste doublement chaînée. À partir des occurrences des deux symboles fusionnés, il est alors possible de retrouver toutes les paires contenant l'un de ces symboles en examinant uniquement leurs voisins gauche et droit. Les paires ainsi identifiées sont regroupées dans une instance de la classe **Set**, ce

qui évite les doublons et permet de mettre à jour uniquement les scores des paires réellement affectées par la fusion.

6.1.2.1 Exemple sur l'optimisation de l'entraînement de WordPiece

Afin d'illustrer le fonctionnement de l'optimisation de l'entraînement de WordPiece, nous considérons l'exemple suivant. Dans les figures qui suivent, la paire ayant le score maximal est représentée en rouge. Les symboles et les paires dont la fréquence diminue après la fusion de cette paire sont représentés en bleu. Les paires dont le score est modifié à la suite de la mise à jour des fréquences des symboles sont également représentées en bleu.

Le corpus utilisé pour cet exemple est : « fred fed ted bread and ted fed fred bread ». Dans un premier temps, la fréquence de chaque mot du corpus est calculée. Ensuite, pour chaque mot distinct, chacun des caractères qui le composent est inséré successivement dans une liste doublement chaînée, accompagné de la fréquence du mot auquel il appartient. Après l'insertion du dernier caractère, un symbole spécial EOW (End Of Word) est ajouté afin de marquer la fin du mot et d'empêcher la formation de paires entre des symboles appartenant à deux mots différents. Cette représentation permet ensuite d'accéder efficacement aux symboles voisins lors des opérations de fusion, sans avoir à reconstruire le corpus à chaque itération.

La Figure 6.1 présente la représentation de ce corpus sous la forme d’une liste doublement chaînée. Les fréquences des symboles, les fréquences des paires ainsi que les scores des paires calculés à partir de ces fréquences sont également indiqués.

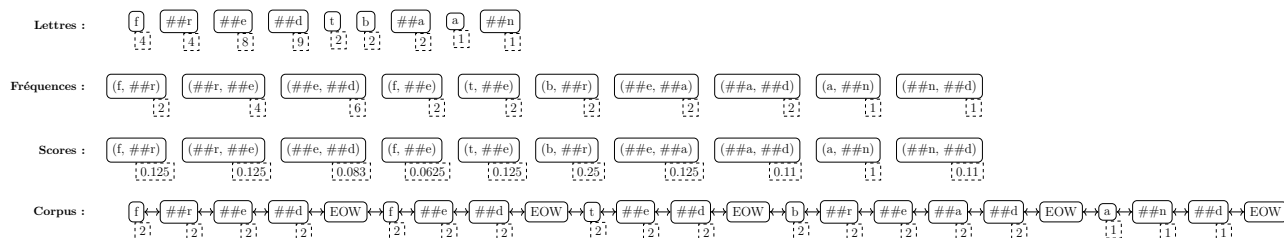


FIGURE 6.1 – Representation du corpus « fred fed ted bread and ted fed fred bread » en utilisant une liste doublement chainee

La paire ayant le score maximal est (a, ##n). Elle est donc sélectionnée pour être fusionnée afin de créer un nouveau symbole an.

Grâce au dictionnaire associant chaque paire existante à un ensemble contenant les références vers les nœuds de la liste doublement chaînée où cette paire apparaît, il est possible d'accéder directement aux occurrences de (a, ##n) sans parcourir l'ensemble du corpus. Pour chaque occurrence trouvée, seules les paires affectées par cette modification doivent être mises à jour.

En effet, après le remplacement de (a, ##n) par le nouveau symbole an, certaines paires qui existaient avant la fusion ne correspondent plus à deux symboles consécutifs dans la liste doublement chaînée. Dans cet exemple, la paire concernée est (##n, ##d). Sa fréquence est décrémentée de la fréquence du mot contenant cette occurrence. Comme la paire (a, ##n) apparaît uniquement dans le mot and, dont la fréquence est égale à 1, la fréquence de (##n, ##d) est diminuée de 1.

La paire (EOW, a) n'est pas prise en compte, car EOW représente la fin d'un mot. Elle empêche la création de paires entre des symboles appartenant à deux mots différents.

La fréquence des symboles a et ##n est également mise à jour, car certaines de leurs occurrences sont remplacées par le nouveau symbole an après la fusion. De ce fait, les paires contenant ces symboles peuvent voir leur score modifié. Dans cet exemple, la mise à jour concerne notamment la paire (##n, ##d).

Les paires dont la fréquence devient nulle après ces mises à jour sont supprimées des dictionnaires des fréquences et des positions. Enfin, la paire fusionnée (a, ##n) est supprimée, puisqu'elle n'existe plus dans la représentation du corpus après son remplacement par le nouveau symbole an.

La Figure 6.2 met en évidence les éléments qui seront affectés lors de la fusion de la paire (a, ##n). Les éléments en rouge correspondent à la paire fusionnée, tandis que les éléments en bleu représentent les symboles et les paires dont les fréquences ou les scores seront mis à jour.

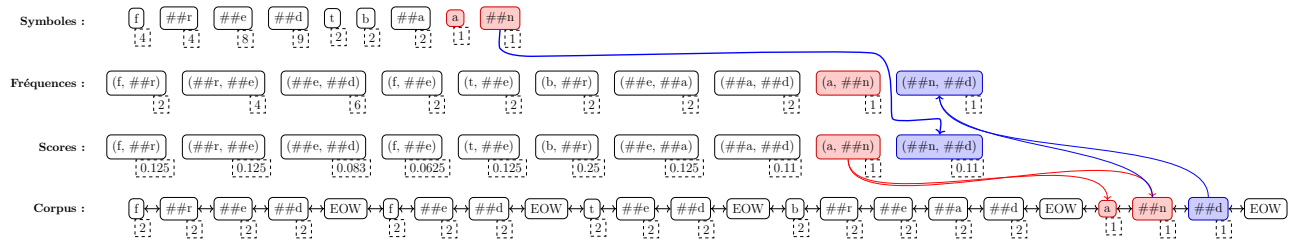


FIGURE 6.2 – Représentation du corpus 'fred fed ted bread, and ted fed fred bread' en utilisant une liste doublement chaînée

La Figure 6.3 illustre l'état de la liste doublement chaînée après la fusion de la paire (a, ##n).

Après cette fusion, la paire ayant le score maximal est (b, ##r), dont le score est égal à 0,25. Cette paire est donc sélectionnée pour être fusionnée afin de créer le nouveau symbole br.

Grâce au dictionnaire qui associe chaque paire à l'ensemble des références vers les nœuds contenant ses occurrences dans la liste doublement chaînée, il est possible d'accéder directement aux occurrences de (b, ##r) sans parcourir l'ensemble du corpus et identifier les paires qui disparaissent à la suite de la fusion de (b, ##r), c'est-à-dire celles qui ne correspondent plus à deux symboles consécutifs après le remplacement de cette paire par le nouveau symbole br. La paire (b, ##r) apparaît uniquement dans le mot BREAD, lequel est présent deux fois dans le corpus. La fréquence de la paire (##r, ##e) est donc décrétementée de 2. La paire (EOW, b) n'est pas prise en compte, car EOW marque la fin d'un mot et empêche la formation de paires entre deux mots différents.

Les fréquences des symboles b et ##r sont ensuite mises à jour afin de tenir compte des occurrences remplacées par le nouveau symbole br. Dans cet exemple, le mot bread apparaît deux fois dans le corpus. Les deux occurrences de b et les deux occurrences de ##r appartenant à ce mot sont donc remplacées par br. La fréquence de b diminue ainsi de 2 et celle de ##r diminue également de 2.

Cette mise à jour entraîne une modification des scores des paires contenant ces symboles.

Dans cet exemple, il s'agit des paires (f,##r) et (##r,##e).

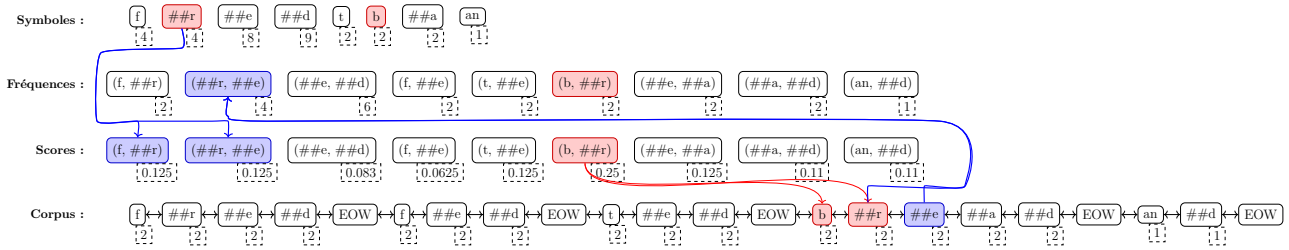


FIGURE 6.3 – État de la liste doublement chaînée après fusion de la paire (a, ##n)

La Figure 6.4 présente l'état de la liste doublement chaînée ainsi que les fréquences des symboles, les fréquences des paires et les scores après la fusion de (b,##r).

La paire possédant désormais le score maximal est (f,##r). L'algorithme applique alors exactement les mêmes opérations : récupération des occurrences de cette paire à partir du dictionnaire des positions, mise à jour locale de la liste doublement chaînée, décrémentation des fréquences des symboles et des paires affectés, recalcul des scores correspondants, puis insertion du nouveau symbole obtenu par fusion. Ce processus est répété jusqu'à atteindre la taille de vocabulaire souhaitée.

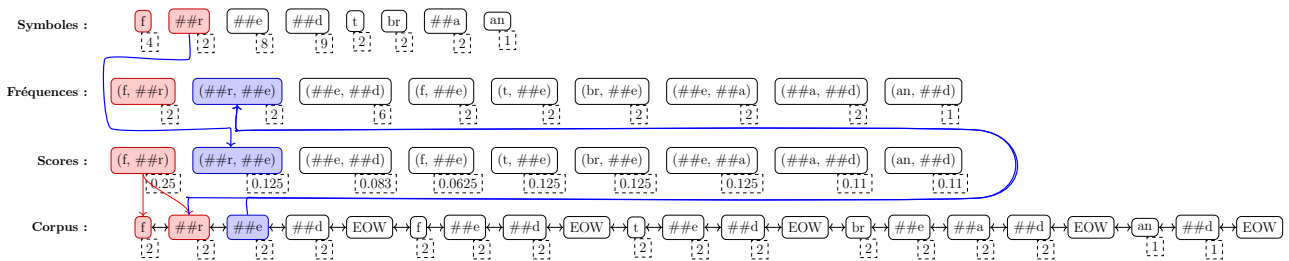


FIGURE 6.4 – État de la liste doublement chaînée après fusion de la paire (b, ##r)

6.1.3 Fonctionnement

1. Calcul de la fréquence des mots

Le corpus est d'abord découpé en mots, les espaces, les espaces multiples et les tabulations étant considérés comme des séparateurs. La fréquence d'apparition de chaque mot est ensuite calculée.

2. Représentation du corpus à l'aide d'une liste doublement chaînée

Pour chaque mot unique du corpus, on insère successivement chacun des caractères qui le composent dans une unique liste doublement chaînée représentant l'ensemble du corpus. Chaque nœud contient le symbole ainsi que la fréquence du mot auquel il appartient. Conformément au principe de WordPiece, le premier caractère d'un mot est conservé tel quel, tandis que les caractères suivants sont préfixés par ##. Après l'insertion du dernier caractère d'un mot, un symbole spécial EndOfWord est ajouté dans la liste doublement chaînée afin de marquer la frontière entre deux mots et d'empêcher la fusion de symboles

appartenant à des mots différents. Cette opération est répétée jusqu'à ce que tous les mots uniques du corpus aient été insérés dans la liste.

3. Construction du vocabulaire, du dictionnaire des fréquences des symboles et du dictionnaire des positions des symboles

On parcourt la liste doublement chaînée afin de construire le vocabulaire, le dictionnaire des fréquences des symboles et le dictionnaire des positions des symboles.

Pour chaque nœud de la liste dont le symbole est différent de EndOfWord, les opérations suivantes sont effectuées :

- le symbole contenu dans le nœud courant est ajouté au vocabulaire s'il n'y figure pas déjà ;
- si ce symbole n'est pas encore présent dans le dictionnaire des fréquences des symboles, il y est ajouté avec comme valeur la fréquence contenue dans le nœud courant. Dans le cas contraire, cette valeur est incrémentée de la fréquence contenue dans le nœud courant. Ainsi, le dictionnaire des fréquences des symboles associe à chaque symbole son nombre total d'occurrences dans le corpus ;
- si ce symbole n'est pas encore présent dans le dictionnaire des positions des symboles, il y est ajouté avec comme valeur un ensemble contenant une référence vers le nœud courant. Dans le cas contraire, une référence vers ce nœud est ajoutée à l'ensemble déjà associé à ce symbole. Le dictionnaire des positions des symboles permet ainsi d'accéder directement à toutes les occurrences d'un symbole dans la liste doublement chaînée, sans avoir à parcourir de nouveau le corpus.

4. Construction du dictionnaire des fréquences des paires et du dictionnaire des positions des paires

On parcourt chaque nœud de la liste doublement chaînée dont le nœud suivant est différent de NIL. Pour chaque couple de nœuds consécutifs, les opérations suivantes sont effectuées :

- on forme une paire constituée des symboles contenus dans le nœud courant et dans son nœud suivant. Si l'un des deux symboles est EndOfWord, cette paire n'est pas prise en compte ;
- si cette paire n'est pas encore présente dans le dictionnaire des fréquences des paires, elle y est ajoutée avec comme valeur associée la fréquence contenue dans le nœud courant. Dans le cas contraire, sa fréquence est incrémentée de la fréquence contenue dans le nœud courant. Le dictionnaire des fréquences des paires associe ainsi à chaque paire son nombre total d'occurrences dans le corpus ;
- si cette paire n'est pas encore présente dans le dictionnaire des positions des paires, elle y est ajoutée avec comme valeur un ensemble contenant un couple de références, dont la première pointe vers le nœud courant et la seconde vers le nœud suivant du nœud courant. Dans le cas contraire, un couple de références, dont la première pointe vers le nœud courant et la seconde vers son nœud suivant, est ajouté à l'ensemble associé à cette paire. Le dictionnaire des positions des paires permet d'accéder directement à toutes les occurrences d'une paire dans la liste doublement chaînée, sans avoir à

parcourir de nouveau le corpus.

5. Construction du dictionnaire des scores

Le dictionnaire des scores associe à chaque paire de symboles consécutives son score, calculé selon la formule utilisée par WordPiece :

$$score(x, y) = \frac{fréquence(x, y)}{fréquence(x) * fréquence(y)}$$

Pour construire ce dictionnaire, on parcourt toutes les paires présentes dans le dictionnaire des fréquences des paires. Pour chaque paire, on calcule son score à l'aide de sa fréquence ainsi que des fréquences de chacun des deux symboles qui la composent, puis on l'ajoute au dictionnaire des scores avec la valeur obtenue.

La fréquence de chaque paire est récupérée à partir du dictionnaire des fréquences des paires, tandis que les fréquences des deux symboles sont obtenues à partir du dictionnaire des fréquences des symboles.

6. Sélection de la paire de symboles ayant le score maximal et création d'un nouveau symbole

On sélectionne dans le dictionnaire des scores la paire de symboles ayant le score maximal, notée p_{max} .

À partir des deux éléments composant p_{max} , on crée un nouveau symbole correspondant au résultat de leur fusion. Lorsque le deuxième élément de la paire possède le préfixe « ## », celui-ci est supprimé avant la concaténation des deux éléments. Par exemple, la fusion de la paire (a, ##n) produit le symbole an.

Le nouveau symbole obtenu est ensuite ajouté au vocabulaire.

7. Mise à jour de la liste doublement chaînée et des dictionnaires

Après la sélection de la paire p_{max} , la liste doublement chaînée ainsi que les différents dictionnaires doivent être mis à jour afin de refléter la fusion des occurrences de cette paire.

a) Récupération des occurrences de p_{max}

On récupère la valeur associée à p_{max} dans le dictionnaire des positions des paires, que l'on note P_1 . Cette valeur est un ensemble de couples de références. Pour chaque couple $p_1 \in P_1$, la première référence pointe vers le nœud contenant le premier symbole de p_{max} , tandis que la seconde référence pointe vers le nœud suivant, contenant le second symbole de p_{max} . Ainsi, P_1 permet d'accéder directement à toutes les occurrences de p_{max} dans la liste doublement chaînée, sans avoir à parcourir l'ensemble du corpus.

b) Récupération des occurrences des symboles composant p_{max}

On récupère également les valeurs associées au premier et au second symbole de p_{max} dans le dictionnaire des positions des symboles, notées respectivement P_2 et P_3 . Les éléments de P_2 (respectivement P_3) sont des références vers les nœuds contenant les occurrences du premier (respectivement du second) symbole de p_{max} .

c) **Identification des paires dont le score sera modifié**

Les seuls scores susceptibles d'être modifiés après la fusion de p_{max} sont ceux des paires contenant le premier ou le second symbole de p_{max} . En effet, la fusion entraîne une diminution de la fréquence de ces deux symboles, ce qui modifie le dénominateur de la formule de calcul du score des paires qui les contiennent.

Pour identifier ces paires, on utilise les ensembles P_2 et P_3 , qui contiennent respectivement les références vers les nœuds contenant les occurrences du premier et du second symbole de p_{max} .

Pour chaque référence $p_2 \in P_2$, on accède au nœud qu'elle référence, lequel contient une occurrence du premier symbole de p_{max} . Deux paires de symboles consécutifs sont alors considérées :

- la paire formée par le symbole contenu dans le nœud précédent du nœud référencé par p_2 et celui contenu dans le nœud référencé par p_2 ;
- la paire formée par le symbole contenu dans le nœud référencé par p_2 et celui contenu dans le nœud suivant du nœud référencé par p_2 .

Pour chacune de ces deux paires, on vérifie qu'il s'agit d'une paire valide, c'est-à-dire que les deux nœuds contiennent des instances de **Token**. On vérifie également que cette paire est différente de p_{max} . Si ces conditions sont satisfaites, la paire est ajoutée à l'ensemble P_4 .

De même, pour chaque référence $p_3 \in P_3$, on accède au nœud référencé ainsi qu'à son nœud suivant, puis on forme la paire constituée du symbole contenu dans le nœud référencé et du symbole contenu dans le nœud suivant. Si cette paire est valide, elle est également ajoutée à P_4 .

L'ensemble P_4 contient ainsi toutes les paires dont le score devra être recalculé après la mise à jour des fréquences des symboles et des fréquences des paires.

L'utilisation d'un ensemble garantit qu'une même paire n'est enregistrée qu'une seule fois, même si elle est rencontrée plusieurs fois lors de la construction de P_4 . Ainsi, le score de chaque paire n'est recalculé qu'une seule fois.

d) **Mise à jour des dictionnaires des symboles**

Après la fusion, certaines occurrences des deux symboles composant p_{max} disparaissent puisqu'elles sont remplacées par le nouveau symbole.

À l'aide des informations contenues dans P_1 , on met donc à jour le dictionnaire des fréquences des symboles ainsi que le dictionnaire des positions des symboles.

Pour chaque élément p_1 de P_1 :

- on décrémente la fréquence du premier symbole de p_{max} de la fréquence contenue dans le nœud référencé par le premier élément de p_1 ;
- on décrémente de la même manière la fréquence du second symbole de p_{max} ;
- on supprime la première référence de p_1 de l'ensemble des références associé au premier symbole de p_{max} dans le dictionnaire des positions des symboles, ainsi

que la seconde référence de p_1 de l'ensemble des références associé au second symbole de p_{max} .

e) **Mises à jour de la liste doublement chaînée, du dictionnaire des fréquences des paires et du dictionnaire des positions des paires**

Après la fusion de p_{max} , toutes ses occurrences sont remplacées par le nouveau symbole dans la liste doublement chaînée. Cette modification entraîne deux conséquences :

- certaines paires de symboles consécutifs n'existent plus après la fusion ; leurs fréquences et leurs positions doivent donc être mises à jour dans les dictionnaires des fréquences et des positions des paires ;
- de nouvelles paires de symboles consécutifs apparaissent ; elles doivent être ajoutées dans ces mêmes dictionnaires.

Les informations contenues dans P_1 permettent d'accéder directement à toutes les occurrences de p_{max} dans la liste doublement chaînée. Les mises à jour peuvent ainsi être effectuées localement, sans parcourir de nouveau l'ensemble du corpus.

Pour chaque couple de références $p_1 \in P_1$, on procède comme suit.

— **Mise à jour du dictionnaire des fréquences des paires et du dictionnaire des positions des paires**

On considère les deux paires qui disparaissent après la fusion :

- la paire formée par le symbole contenu dans le nœud précédent du nœud référencé par le premier élément de p_1 et le symbole contenu dans le nœud référencé par le premier élément de p_1 ;
- la paire formée par le symbole contenu dans le nœud référencé par le second élément de p_1 et le symbole contenu dans le nœud suivant du nœud référencé par le second élément de p_1 .

Pour chacune de ces deux paires, si elle est valide (c'est-à-dire qu'aucun de ses deux symboles n'est EndOfWord), sa fréquence est décrémentée de la fréquence contenue dans le nœud référencé par le premier élément de p_1 . On supprime également, dans le dictionnaire des positions des paires, le couple de références correspondant à cette occurrence de la paire. Pour la première paire, ce couple est constitué de la référence vers le nœud précédent du nœud référencé par le premier élément de p_1 et du premier élément de p_1 . Pour la seconde paire, le couple est constitué du second élément de p_1 et de la référence vers le nœud suivant du nœud référencé par le second élément de p_1 .

Si, après cette décrémentation, la fréquence d'une paire devient nulle, cette paire est supprimée des dictionnaires des fréquences des paires et des positions des paires.

— **Mise à jour de la liste doublement chaînée**

Le symbole contenu dans le nœud référencé par le premier élément de p_1 est rem-

placé par le nouveau symbole obtenu par la fusion des deux symboles composant p_{max} . La fréquence stockée dans ce nœud reste inchangée.

Le nœud référencé par le second élément de p_1 est supprimé de la liste doublement chaînée, car son symbole est fusionné avec celui du nœud référencé par le premier élément de p_1 . Avant sa suppression, on met à jour les liens entre les nœuds voisins : le pointeur suivant du nœud référencé par le premier élément de p_1 est remplacé par le nœud qui suivait le nœud référencé par le second élément de p_1 . Réciproquement, le pointeur précédent de ce nœud est remplacé par le nœud référencé par le premier élément de p_1 .

— **Ajout des nouvelles paires dans le dictionnaire des fréquences des paires et le dictionnaire des positions des paires**

Après la mise à jour de la liste doublement chaînée, deux nouvelles paires peuvent apparaître :

- la paire de symboles formée par le symbole contenu dans le nœud précédent du nœud référencé par le premier élément de p_1 et le nouveau symbole contenu dans le nœud référencé par le premier élément de p_1 ;
- la paire formée par le nouveau symbole contenu dans le nœud référencé par le premier élément de p_1 et le symbole contenu dans le nœud suivant du nœud référencé par le premier élément de p_1 .

Pour chacune de ces paires, si elle est valide (c'est-à-dire qu'aucun de ses deux symboles n'est EndOfWord), on met à jour les dictionnaires :

- si la paire n'existe pas encore dans le dictionnaire des fréquences des paires, elle y est ajoutée avec comme valeur la fréquence contenue dans le nœud référencé par le premier élément de p_1 ; sinon, cette fréquence est incrémentée de cette même valeur ;
- si la paire n'existe pas encore dans le dictionnaire des positions des paires, elle y est ajoutée avec comme valeur un ensemble contenant un couple de références. Pour la première paire, ce couple de références est constitué de la référence vers le nœud précédent du nœud référencé par le premier élément de p_1 et du premier élément de p_1 . Pour la seconde paire, il est constitué du premier élément de p_1 et de la référence vers le nœud suivant du nœud référencé par le premier élément de p_1 . Si la paire est déjà présente dans le dictionnaire, ce couple de références est ajouté dans la valeur associée à la paire dans ce dictionnaire.

f) **Mise à jour du dictionnaire des scores**

Après la fusion des éléments de p_{max} , les scores des paires susceptibles d'être modifiés sont les paires contenant l'un des deux symboles composant p_{max} . En effet, la fusion entraîne une modification des fréquences de ces symboles ainsi que des fréquences de certaines paires qui les contiennent.

Les paires concernées ont été identifiées précédemment et sont stockées dans l'en-

semble P_4 . Pour chaque paire $p_4 \in P_4$, on met à jour sa valeur dans le dictionnaire des scores en recalculant son score :

$$score(p_4) = \frac{fréquence(p_4)}{fréquence(p_4[1]) \times fréquence(p_4[2])}$$

où :

- $fréquence(p_4)$ est obtenue à partir du dictionnaire des fréquences des paires ;
- $fréquence(p_4[1])$ et $fréquence(p_4[2])$ correspondent aux fréquences des deux symboles composant la paire, obtenues à partir du dictionnaire des fréquences des symboles.

Si une paire n'existe plus dans le dictionnaire des fréquences des paires après les mises à jour précédentes, elle est supprimée du dictionnaire des scores. Ainsi, le dictionnaire des scores contient uniquement les paires encore présentes dans le corpus avec leurs scores actualisés.

g) **Ajout du nouveau symbole dans le dictionnaire des fréquences des symboles et le dictionnaire des positions des symboles**

Après la fusion des éléments de p_{max} , un nouveau symbole est créé. Ce symbole remplace uniquement les occurrences de p_{max} , c'est-à-dire les occurrences où le premier élément et le second élément de p_{max} sont contenus dans deux nœuds successifs de la liste doublement chaînée, le nœud contenant le premier élément étant immédiatement suivi par le nœud contenant le second élément.

Les éléments de P_1 permettent d'accéder directement aux nœuds contenant les occurrences de ce nouveau symbole après la fusion. Ainsi, il est possible de mettre à jour les dictionnaires des fréquences et des positions des symboles sans parcourir l'ensemble de la liste doublement chaînée.

Pour chaque couple de références $p_1 \in P_1$, le premier élément de p_1 pointe désormais vers un nœud contenant une occurrence du nouveau symbole. On effectue alors les opérations suivantes :

- Si le nouveau symbole n'existe pas encore dans le dictionnaire des fréquences des symboles, il y est ajouté avec comme valeur associée la fréquence contenue dans le nœud référencé par le premier élément de p_1 . Sinon, cette valeur est incrémentée de cette même fréquence.
- Si le nouveau symbole n'existe pas encore dans le dictionnaire des positions des symboles, il y est ajouté avec comme valeur associée un ensemble contenant le premier élément de p_1 . Sinon, le premier élément de p_1 est ajoutée à l'ensemble déjà associé à ce symbole dans le dictionnaire.

Ainsi, après le traitement de toutes les occurrences de p_{max} , les dictionnaires des fréquences et des positions des symboles contiennent les informations relatives au nouveau symbole créé par la fusion.

h) **Ajout des nouvelles paires de symboles dans le dictionnaire des scores**

Après le remplacement des occurrences dep_{max} par le nouveau symbole résultant de la fusion de ses éléments dans la liste doublement chaînée, de nouvelles paires de symboles consécutifs peuvent apparaître. Ces paires doivent être ajoutées dans le dictionnaire des scores afin que celui-ci reflète l'état actuel du corpus.

Les éléments contenus dans P_1 permettent d'identifier directement ces nouvelles paires en accédant directement aux nœuds contenant les occurrences du nouveau symbole, sans avoir à parcourir l'ensemble de la liste doublement chaînée.

Pour chaque couple de références $p_1 \in P_1$, on construit au maximum deux nouvelles paires :

- une paire formée par le symbole contenu dans le nœud précédent du nœud référencé par le premier élément de p_1 et le symbole contenu dans le nœud référencé par le premier élément de p_1 ;
- une paire formée par le symbole contenu dans le nœud référencé par le premier élément de p_1 et le symbole contenu dans le nœud suivant du nœud référencé par le premier élément de p_1 .

Si une paire obtenue est valide, c'est-à-dire si aucun de ses deux symboles n'est égal à EndOfWord, et si elle n'existe pas encore dans le dictionnaire des scores, elle est ajoutée avec comme valeur associée son score calculé selon la formule :

$$score(x, y) = \frac{fréquence(x, y)}{fréquence(x) \times fréquence(y)}$$

Les informations nécessaires au calcul du score sont récupérées à partir du dictionnaire des fréquences des paires et du dictionnaire des fréquences des symboles mis à jour précédemment.

8. Répétition de l'étape 6 à 7 jusqu'à atteindre la taille du vocabulaire souhaitée

6.1.4 Réalisation

Dans cette partie, nous présentons une implémentation de l'optimisation de l'entraînement de WordPiece réalisée dans Pharo.

6.1.4.1 Création d'une classe

WordPiece et BPE partagent de nombreux points communs dans les étapes de l'entraînement, notamment :

- la représentation du corpus ;
- le calcul des fréquences des paires de symboles ;
- le stockage des positions des paires de symboles dans une liste doublement chaînée ;
- les opérations de fusion et de mise à jour des structures de données.

Cependant, ils diffèrent principalement sur les points suivants :

- Prétraitement : dans WordPiece, les symboles situés au milieu d'un mot sont préfixés par `##`, tandis que dans BPE un marqueur de fin de mot, tel que `_` ou `'</t>'`, est généralement ajouté à la fin des mots.
- Sélection de la paire à fusionner : WordPiece sélectionne la paire de symboles ayant le score maximal, alors que BPE sélectionne la paire de symboles ayant la fréquence maximale.
- Fréquence des symboles : dans WordPiece, la fréquence de chaque symbole doit être calculée car elle intervient dans le calcul du score des paires.
- Calcul du score des paires : chaque paire de symboles consécutifs possède un score calculé à partir de sa fréquence et des fréquences des symboles qui la composent.

Étant donné que WordPiece réutilise une grande partie de la logique de BPE, nous avons défini `WordPieceTrain` comme une sous-classe de `BPETrain`.

```
BPETrain << #WordPieceTrain
  slots: { #tokenFrequencies . #tokenPositions . #pairScores . #affectedPairSet }.
  package: 'Tokenizer'
```

Les variables d'instance spécifiques à `WordPieceTrain` sont les suivantes :

- `tokenFrequencies` : stocke les symboles et leurs fréquences ;
- `tokenPositions` : stocke les symboles et les références vers les nœuds contenant leurs occurrences dans la liste doublement chaînée ;
- `pairScores` : stocke les paires de symboles consécutifs et leurs scores ;
- `affectedPairSet` : stocke les paires dont le score doit être recalculé après une fusion.

La classe `WordPieceTrain` hérite également de plusieurs variables d'instance définies dans `BPETrain`, notamment :

- `corpus` : représentation du corpus ;
- `pairFrequencies` : fréquences des paires de symboles consécutifs ;
- `pairPositions` : positions des occurrences des paires.
- `vocabulary` : l'ensemble des vocabulaires appris durant l'entraînement.

Certaines méthodes sont héritées sans modification, notamment : `fromText:`, `mergeOneStep`, `addNewPairFrom:And:`, `addNewPairFrom:And:` et `updatePairFrom:And:`.

D'autres méthodes sont redéfinies afin de prendre en compte les spécificités de WordPiece : `computePairs`, `computeVocabulary`, `corpus:`, `createNewVocabulary`, `update:`, `findPairToMerge` et `fusion`.

Enfin, des méthodes spécifiques à WordPiece ont été ajoutées : `addNewPairScoresFrom:And:`, `addToAffectedPairSet:`, `addToTokenFrequenciesAndTokenPositions`, `pairsAffectedByMerging:`, `updatePairScoresFrom:` et `updateTokenFrequenciesAndTokenPositions:`.

Pour rappel, chaque symbole du corpus est représenté par une instance de la classe `Token`, qui stocke :

- le symbole(`token`) ;

- la fréquence du mot auquel il appartient(**occurrence**);
- la référence vers le nœud qui le contient dans la liste doublement chaînée**position**.

Une instance de la classe **EndOfWord** est utilisée pour séparer deux mots dans la liste doublement chaînée et empêcher la fusion de symboles appartenant à des mots différents.

Les classes **Token** et **EndOfWord** sont présentées en détail dans le chapitre consacré à l'optimisation de BPE.

6.1.4.2 Structures de données utilisées

- **Bag** : La classe **Bag** est utilisée pour compter les occurrences des mots du corpus, des symboles et des paires de symboles. L'ajout d'un élément dans une instance de **Bag** incrémente automatiquement son nombre d'occurrences. **pairFrequencies** et **tokenFrequencies** sont des instances de **Bag**.
- **DoubleLinkedList** : Le corpus est représenté par une liste doublement chaînée. Chaque nœud de la liste doublement chaînée contient soit une instance de la classe **EndOfWord** soit une instance de la classe **Token** contenant un symbole du corpus, la fréquence du mot auquel appartient ce symbole et une référence vers sa position dans la liste doublement chaînée. Cette structure conserve l'ordre des symboles dans les mots, permet des modifications locales lors des fusions et évite de parcourir de nouveau tout le corpus lorsqu'on connaît déjà les références des nœuds concernés. La variable **corpus** est une instance de **DoubleLinkedList**.
- **Dictionary** : La classe **Dictionary** est utilisée pour accéder rapidement aux positions des symboles, aux positions des paires et aux scores des paires. **pairPositions**, **pairScores** et **tokenPositions** sont des instances de **Dictionary**.
- **Set** : La classe **Set** est utilisée pour stocker des éléments sans doublons. **vocabulary** et **affectedPairSet** sont des instances de **Set**.

6.1.4.3 Méthodes

Certaines méthodes héritées de **BPETrain** (**fromText**:, **mergeOneStep**, **addNewPairFrom:And**:, **addToVocabulary**:, **updatePairFrom:And**:) ne sont pas redéfinies dans **WordPieceTrain**. Leur fonctionnement a été présenté dans le chapitre sur l'optimisation de BPE.

1. Initialisation des variables d'instance

```
initialize
  super initialize .
  pairScores := Dictionary new.
  tokenFrequencies := Bag new.
  tokenPositions := Dictionary new.
  affectedPairSet := Set new .
```

2. Construction de la représentation du corpus

La méthode `corpus` : construit la représentation du corpus sous forme de liste doublement chaînée stockée dans la variable d'instance `corpus`. Chaque nœud de cette liste contient soit une instance de `Token`, représentant un symbole du corpus, soit une instance de `EndOfWord`, représentant une frontière entre deux mots.

```
corpus: anObject
| wordFrequencies |
wordFrequencies := Bag new.
wordFrequencies addAll: (Preprocessing preprocess: anObject) .
wordFrequencies associationsDo: [ :each||word|
    word := each key.
    1 to: word size do: [ :i ||char|
        char := word at: i.
        i = 1
            ifTrue:
                [ corpus add: (Token token: char asString occurrence: each value) ]
            ifFalse:
                [corpus add: (Token token: ('##', char asString) occurrence: each value)].
        corpus lastLink value position: corpus lastLink ].
    corpus add: EndOfWord new ].
^ corpus
```

La méthode `corpus` : reçoit en entrée un texte ou une collection de textes. Cette entrée est prétraitée par la méthode `preprocess` : de la classe `preprocessing` afin de produire une liste de mots en utilisant les espaces, les espaces multiples et les tabulations comme séparateurs.

Les mots obtenus sont ensuite ajoutés dans une instance de `Bag` afin de calculer leurs fréquences. Cette étape permet de traiter chaque mot une seule fois, ce qui évite de répéter les mêmes opérations pour plusieurs occurrences identiques du même mot.

Pour chaque mot distinct du corpus, une instance de la classe `Token` est créée pour chacun des caractères qui le composent. La variable d'instance `token` contient le caractère correspondant. Si ce caractère est le premier du mot, il est conservé tel quel ; sinon, il est préfixé par `##`, conformément au principe de `WordPiece`. La variable d'instance `occurrence` contient la fréquence du mot auquel appartient ce caractère. Cette instance de `Token` est ensuite ajoutée à la fin de la liste doublement chaînée, puis la référence vers le nœud nouvellement créé est enregistrée dans sa variable d'instance `position`. Cette opération est répétée pour chacun des caractères du mot jusqu'à ce que l'instance de la classe `Token` représentant le dernier caractère soit insérée dans la liste doublement chaînée. Après l'insertion de cette dernière, une instance de la classe `EndOfWord` est ajoutée à la

liste doublement chaînée afin de marquer la fin de ce mot. Ce symbole sert de séparateur entre deux mots et empêche la création de paires de symboles appartenant à des mots différents lors des opérations de fusion. Les mêmes opérations sont ensuite répétées pour chacun des autres mots distincts du corpus.

Cette représentation permet de conserver l'ordre des symboles composant les mots du corpus. Elle facilite également les opérations de fusion, puisque celles-ci ne nécessitent que des modifications locales de la liste doublement chaînée. Enfin, lorsqu'une référence vers un nœud est connue, elle permet d'accéder directement à ce nœud sans avoir à parcourir l'ensemble de la liste doublement chaînée.

3. Construction de **vocabulary**, **tokenFrequencies** et **tokenPositions**

La méthode `computeVocabulary` construit simultanément les variables d'instance **vocabulary**, **tokenFrequencies** et **tokenPositions**. Contrairement à `BPETrain`, où cette méthode construit uniquement le vocabulaire, `WordPieceTrain` calcule également les fréquences et les positions des symboles. Ces deux structures sont nécessaires au calcul du score des paires de symboles lors de l'entraînement de `WordPiece`.

Ces trois structures pouvant être construites au cours d'un même parcours de la liste doublement chaînée, leur initialisation est regroupée dans une seule méthode. Cela évite de parcourir plusieurs fois la liste et réduit ainsi le temps d'exécution.

vocabulary est une instance de la classe `Set`. Elle contient l'ensemble des symboles appartenant au vocabulaire appris. Au début de l'entraînement, elle contient tous les symboles présents dans le corpus, c'est-à-dire le premier caractère de chaque mot ainsi que les caractères situés au milieu ou à la fin des mots, préfixés par `##`.

tokenFrequencies est une instance de la classe `Bag`. Elle associe à chaque symbole du corpus son nombre total d'occurrences. Au cours de l'entraînement, son contenu est mis à jour après chaque fusion afin de refléter les fréquences des symboles existants.

tokenPositions est une instance de la classe `Dictionary`. À chaque symbole est associé un ensemble contenant les références vers les nœuds de la liste doublement chaînée dans lesquels apparaissent les occurrences de ce symbole. Comme **tokenFrequencies**, cette structure est mise à jour après chaque fusion.

L'initialisation de ces structures est réalisée par la méthode `computeVocabulary`, qui parcourt la liste doublement chaînée à partir de son premier nœud jusqu'au dernier.

Pour chaque nœud dont la valeur est une instance de la classe `Token`, la méthode appelle `addToTokenFrequenciesAndTokenPositions`. Cette méthode ajoute le symbole contenu dans la variable d'instance **token** de l'objet `Token` dans **tokenFrequencies**, avec comme nombre d'occurrences la valeur de sa variable d'instance **occurrence**.

Elle met également à jour **tokenPositions**. Si le symbole n'est pas encore présent dans ce dictionnaire, il est ajouté avec comme valeur un ensemble contenant la référence stockée dans la variable d'instance **position** de l'objet `Token`. Dans le cas contraire, cette référence est ajoutée à l'ensemble déjà associé à ce symbole.

Les mêmes opérations sont répétées jusqu'à ce que tous les nœuds de la liste doublement chaînée aient été parcourus.

Une fois ce parcours terminé, le vocabulaire est obtenu en convertissant `tokenFrequencies` en un ensemble, ce qui supprime automatiquement les doublons. Le résultat est ensuite affecté à la variable d'instance `vocabulary`.

La méthode `addToTokenFrequenciesAndTokenPositions` est implémentée comme suit :

```
addToTokenFrequenciesAndTokenPositions: aToken
    tokenFrequencies
    add: aToken token
    withOccurrences: aToken occurrence .
    tokenPositions
    at: aToken token
    update: [:set|
        set add: aToken position ;
        yourself ]
    initial: (Set with: aToken position)
```

La méthode `computeVocabulary` est quant à elle implémentée de la manière suivante :

```
computeVocabulary
|i|
i:=corpus firstLink .
[ i notNil ]
whileTrue: [
    [ i value class ~= EndOfWord ]
    whileTrue: [
        self addToTokenFrequenciesAndTokenPositions: (i value ) .
        i:= i nextLink].
    i:= i nextLink].
vocabulary := tokenFrequencies asSet .
```

4. Construction de `pairFrequencies`, `pairPositions` et `pairScores`

La construction de `pairFrequencies`, `pairPositions` et `pairScores` est réalisée par la méthode `computePairs`.

Dans `BPETrain`, cette méthode construit uniquement `pairFrequencies` et `pairPositions`. Dans `WordPieceTrain`, elle construit également `pairScores`, qui est utilisé pour sélectionner, à chaque itération de l'entraînement, la paire de symboles ayant le score maximal. `pairFrequencies` est une instance de la classe `Bag`. Elle contient toutes les paires de symboles consécutifs présentes dans le corpus ainsi que leurs fréquences. Après chaque

fusion, cette structure est mise à jour afin de refléter les modifications apportées au corpus : certaines paires disparaissent parce qu'elles ne sont plus constituées de deux symboles consécutifs, tandis que de nouvelles paires apparaissent.

`pairPositions` est une instance de la classe `Dictionary`. Elle associe à chaque paire de symboles l'ensemble des couples de références vers les deux nœuds consécutifs de la liste doublement chaînée contenant chacune de ses occurrences. Cette structure permet, après la sélection de la paire ayant le score maximal, d'accéder directement à toutes ses occurrences sans parcourir de nouveau l'ensemble du corpus. Elle facilite ainsi la mise à jour de la liste doublement chaînée, des fréquences des paires et des positions des paires. Elle est également mise à jour après chaque fusion.

`pairScores` est aussi une instance de la classe `Dictionary`. Elle associe à chaque paire de symboles son score, calculé à partir de la fréquence de cette paire et des fréquences des deux symboles qui la composent. Cette structure permet d'identifier rapidement la paire à fusionner lors de chaque itération de l'entraînement. Comme les fréquences des paires et des symboles évoluent après chaque fusion, `pairScores` est également mis à jour après chaque itération.

La méthode `computePairs` de `WordPieceTrain` commence par invoquer la méthode `computePairs` de `BPETrain`, qui construit `pairFrequencies` et `pairPositions`. L'implémentation de cette partie est présentée dans le chapitre consacré à l'optimisation de BPE.

Une fois `pairFrequencies` construit, la méthode calcule le score de chaque paire de symboles. Pour chaque paire présente dans `pairFrequencies`, elle calcule son score à l'aide de la formule suivante :

$$score(x, y) = \frac{fréquence(x, y)}{fréquence(x) * fréquence(y)}$$

Les fréquences des symboles sont obtenues à partir de `tokenFrequencies`.

L'implémentation de cette méthode dans Pharo est la suivante :

```
computePairs
  super computePairs .
  pairFrequencies associationsDo: [ :each|
    pairScores
      at: each key
      put: each value
      / ((tokenFrequencies occurrencesOf: each key first)
        * (tokenFrequencies occurrencesOf: each key second))asFloat ]
```

Cette implémentation évite de parcourir une seconde fois la liste doublement chaînée pour retrouver les paires de symboles. Elle réutilise directement les informations déjà construites dans `pairFrequencies` ainsi que les fréquences des symboles stockées dans `tokenFrequencies` pour calculer efficacement le score de chaque paire.

5. Sélection de la paire à fusionner

Dans WordPiece, la paire à fusionner à chaque itération de l'entraînement est la paire de symboles ayant le score maximal parmi les paires de symboles présentes dans le dictionnaire `pairScores`.

La méthode `findPairToMerge` réalise cette opération en parcourant les associations contenues dans `pairScores` et en sélectionnant celle dont la valeur associée, correspondant au score de la paire, est maximale. Elle retourne ensuite la paire de symboles correspondante.

L'implémentation de cette méthode dans Pharo est la suivante :

```
findPairToMerge
  ^ (pairScores associations detectMax: [:assoc | assoc value]) key
```

6. Création d'un nouveau symbole à partir d'une paire de symboles

Après avoir sélectionné la paire de symboles ayant le score maximal dans `pairScores`, un nouveau symbole est créé par fusion des deux symboles qui la composent.

Contrairement à BPE, où les deux éléments de la paire sélectionnée sont directement concaténés, WordPiece prend en compte le préfixe `##` utilisé pour représenter les symboles qui apparaissent en milieu ou en fin de mot.

Ainsi, si le second symbole de la paire sélectionnée possède le préfixe `##`, celui-ci est d'abord supprimé avant la fusion. Les deux symboles sont ensuite concaténés afin de former un nouveau symbole. Ce nouveau symbole représente l'unité de sous-mot créée par la fusion et il est ajouté au vocabulaire.

Ce nouveau symbole sera ensuite utilisé pour remplacer les occurrences de cette paire dans la liste doublement chaînée lors de l'étape de mise à jour du corpus.

L'implémentation dans Pharo est la suivante :

```
createNewVocabulary: pair
  |newVocabulary|
  newVocabulary := pair first, ( pair second allButFirst: 2 ) .
  ^ newVocabulary
```

7. Mise à jour de corpus, tokenFrequencies, tokenPositions, pairFrequencies, pairPositions et pairScores

Nous avons maintenant sélectionné la paire de symboles ayant le score maximal, que nous noterons p_{max} , puis fusionné ses deux éléments afin de créer un nouveau symbole.

L'étape suivante consiste à remplacer toutes les occurrences de p_{max} dans la liste doublement chaînée par ce nouveau symbole. Grâce aux informations stockées dans `pairPositions`, il est possible d'accéder directement aux nœuds contenant les occurrences de p_{max} , sans avoir à parcourir l'ensemble de la liste doublement chaînée.

La fusion des éléments de p_{max} modifie la représentation du corpus et entraîne plusieurs conséquences :

- Certaines occurrences des deux symboles composant p_{max} disparaissent. Les fréquences de ces symboles dans `tokenFrequencies` diminuent et les références vers les nœuds correspondants doivent être supprimées de `tokenPositions`.
- Certaines occurrences de paires de symboles ne forment plus des paires de symboles consécutifs. Pour chacune de ces occurrences, la fréquence de la paire correspondante est décrémentée dans `pairFrequencies` et le couple de références pointant vers les deux nœuds qui contenaient cette occurrence est supprimé de l'ensemble associé à cette paire dans `pairPositions`.
- De nouvelles paires de symboles consécutifs apparaissent. Elles doivent être ajoutées dans `pairFrequencies`, `pairPositions` et `pairScores`.
- Enfin, la modification des fréquences de certains symboles et de certaines paires entraîne une modification du score de plusieurs paires de symboles. Ces scores doivent donc être recalculés afin que `pairScores` reste cohérent avec l'état courant du corpus.

Il est donc nécessaire de mettre à jour `tokenFrequencies`, `tokenPositions`, `pairFrequencies`, `pairPositions` et `pairScores` afin que ces structures de données reflètent les modifications apportées à `corpus` après chaque fusion.

Une mise à jour consistant à reparcourir l'ensemble de la liste doublement chaînée après chaque itération serait coûteuse. Afin d'éviter ce parcours complet, l'algorithme exploite les informations déjà stockées dans `tokenPositions` et `pairPositions`, qui permettent d'accéder directement aux nœuds concernés par la fusion et identifier ainsi les symboles et les paires dont les informations doivent être mises à jour.

Ces différentes opérations sont réalisées par la méthode `update:` de la classe `WordPieceTrain`, dont l'implémentation est la suivante :

```
update:aPair
| position|
affectedPairSet removeAll .
self pairsAffectedByMerging: aPair .
position := pairPositions at: aPair .
position do: [ :each |
    self updateTokenFrequenciesAndFrequenciesPositions: each first value.
    self updateTokenFrequenciesAndFrequenciesPositions: each second value].
super update: aPair .
affectedPairSet do: [ :each |
    self updatePairsScoresFrom: each].
position do: [ :each |
    self addToTokenFrequenciesAndTokenPositions: each first value ].
position do: [ :each|
```

```

each first value
tryAddPairsScoreWith: each first previousLink value
in: self.
each first nextLink value
tryAddPairsScoreWith: each first value
in: self].
pairScores removeKey: aPair.

```

La méthode `update:` reçoit en paramètre la paire de symboles p_{max} sélectionnée pour la fusion. Elle met ensuite à jour, dans un ordre précis, les différentes structures de données utilisées par l'algorithme afin qu'elles restent cohérentes avec le contenu de la liste doublement chaînée après la fusion des occurrences de p_{max} .

a) **Initialisation de `affectedPairSet`**

À chaque itération, la méthode `update:` commence par vider `affectedPairSet`, car son contenu dépend de la paire de symboles fusionnée à l'itération courante. Pour rappel, `affectedPairSet` est un ensemble contenant les paires de symboles dont le score est modifié et devra être recalculé après la fusion de p_{max} .

b) **Identification des paires de symboles dont le score est modifié**

La méthode `update:` appelle ensuite la méthode `pairsAffectedByMerging:` en lui passant en paramètre p_{max} . Cette méthode identifie les paires de symboles dont le score sera modifié après la fusion des éléments de p_{max} et les ajoute dans `affectedPairSet`. Son implémentation est la suivante :

```

pairsAffectedByMerging: aPair
|pair|
pair:= {
    (Token new token: aPair first) .
    (Token new token: aPair second)
}.

(tokenPositions at: aPair first ) do: [ :each |
    ({ each previousLink value . each value } ~= pair )
    ifTrue: [
        each value
        tryAddPairAffectedByMergeWith: each previousLink value
        in: self ].
    ({ each value . each nextLink value } ~= pair )
    ifTrue: [
        each nextLink value
        tryAddPairAffectedByMergeWith: each value

```

```

in: self ] ].

(tokenPositions at: aPair second ) do: [ :each |
  ({ each previousLink value . each value } ~= pair )
  ifTrue: [
    each value
    tryAddPairAffectedByMergeWith: each previousLink value
    in: self ].
  ({ each value . each nextLink value } ~= pair )
  ifTrue: [
    each nextLink value
    tryAddPairAffectedByMergeWith: each value
    in: self ]].

```

Pour rappel, les paires dont le score est susceptible d'être modifié sont les paires contenant l'un des deux symboles composant p_{max} . En effet, après la fusion, certaines occurrences de ces deux symboles disparaissent, ce qui modifie leur fréquence et peut donc modifier le score des paires qui les contiennent.

Afin d'identifier ces paires sans parcourir l'ensemble du corpus, `pairsAffectedByMerging:` exploite les informations contenues dans `tokenPositions`. Pour chacun des deux symboles composant p_{max} , la méthode récupère la valeur qui lui est associée dans `tokenPositions`. Cette valeur est un ensemble de références vers les nœuds de la liste doublement chaînée contenant les occurrences de ce symbole. On va noter P cet ensemble.

Pour chaque référence p de P , qui pointe vers un nœud contenant une occurrence du symbole considéré dans la liste doublement chaînée, la méthode tente d'identifier les deux paires de symboles consécutifs contenant cette occurrence du symbole :

- la paire formée par le symbole contenu dans le nœud précédent du nœud référencé par p et le symbole contenu dans le nœud référencé par p ;
- la paire formée par le symbole contenu dans le nœud référencé par p et le symbole contenu dans le nœud suivant du nœud référencé par p .

Pour identifier la première paire, la méthode envoie le message `tryAddPairAffectedByMergeWith:in:` à l'objet contenu dans le nœud référencé par p . Elle lui passe en premier paramètre l'objet contenu dans le nœud précédent du nœud référencé par p , et en second paramètre l'instance courante de `WordPieceTrain`.

Pour identifier la seconde paire, elle envoie de nouveau le message `tryAddPairAffectedByMergeWith:in:` à l'objet contenu dans le nœud suivant du nœud référencé par p . Cette fois, elle lui passe en premier paramètre l'objet contenu dans le nœud référencé par p , et en second paramètre l'instance courante de `WordPieceTrain`.

Chaque nœud de la liste doublement chaînée contient soit une instance de `Token`, soit une instance de `EndOfWord`. La méthode `tryAddPairAffectedByMergeWith:in:` s'appuie sur le polymorphisme pour adapter son comportement selon la classe de l'objet qui reçoit le message.

Lorsque le receveur est une instance de `EndOfWord`, la méthode ne réalise aucune opération :

```
tryAddPairAffectedByMergeWith: anObject in: aBPE
    ^ self
```

Lorsque le receveur est une instance de `Token`, la méthode vérifie d'abord que son paramètre n'est pas `NIL`. Si cette condition est satisfaite, elle délègue l'opération en envoyant le message `addPairAffectedByMergeWith:in:` à cet objet.

```
tryAddPairAffectedByMergeWith: anObject in: aBPE
    anObject ifNil: [ ^ self ].
    anObject addPairAffectedByMergeWith: self in: aBPE
```

La méthode `addPairAffectedByMergeWith:in:` s'appuie également sur le polymorphisme.

Si le receveur est une instance de `EndOfWord`, aucune opération n'est effectuée.

```
addPairAffectedByMergeWith: aToken in: aBPE
    ^ self
```

Si le receveur est une instance de `Token`, la méthode construit une paire de symboles à partir de la valeur de l'attribut `token` du receveur et de la valeur de l'attribut `token` de l'objet `Token` reçu en premier paramètre. Elle transmet ensuite cette paire à la méthode `addToAffectedPairSet:`, qui l'ajoute dans `affectedPairSet`.

```
addPairAffectedByMergeWith: aToken in: aBPE
    aBPE addToAffectedPairSet: { self token . aToken token }
```

La méthode `pairsAffectedByMerging:` répète ces opérations pour chacune des références contenues dans les ensembles associés aux deux symboles composant p_{max} dans `tokenPositions`.

Une paire n'est ajoutée dans `affectedPairSet` que si :

- elle est formée de deux symboles consécutifs dans la liste doublement chaînée ;
- aucun de ses deux éléments n'est une instance de `EndOfWord` ;

- elle contient l'un des deux symboles composant p_{max} ;
- elle est différente de p_{max} .

L'utilisation de `tokenPositions` permet ainsi d'identifier directement toutes les paires dont le score devra être recalculé, sans parcourir l'ensemble de la liste doublement chaînée après chaque fusion. Le recours au polymorphisme permet quant à lui d'éviter l'utilisation répétée de tests conditionnels (`ifTrue : / ifFalse :`) pour distinguer les objets `Token` et `EndOfWord`.

c) **Mise à jour des valeurs associées aux deux symboles composant p_{max} dans `tokenFrequencies` et `tokenPositions`**

Après avoir identifié les paires de symboles consécutifs dont le score sera modifié après la fusion, la méthode `update:` met à jour les informations associées aux deux symboles composant p_{max} dans `tokenFrequencies` et `tokenPositions`.

Cette mise à jour est nécessaire, car les scores des paires identifiées dépendent de la fréquence des symboles qui les composent. Or, après la fusion, les occurrences des deux symboles participant à une occurrence de p_{max} n'existent plus en tant que symboles distincts dans le corpus : elles sont remplacées par le nouveau symbole obtenu par la fusion. La fréquence de chacun de ces deux symboles diminue donc, ce qui entraîne une modification du score des paires contenant l'un d'eux.

Les occurrences des deux symboles composant p_{max} qui vont disparaître après la fusion sont obtenues directement à partir de `P`, la valeur associée à p_{max} dans `pairPositions`.

Pour rappel, `P` est un ensemble de couples de références. Chaque couple $p \in P$ représente une occurrence de p_{max} dans la liste doublement chaînée : son premier élément référence le nœud contenant le premier symbole de p_{max} , tandis que son second élément référence le nœud contenant le second symbole.

Ainsi, chaque couple de références permet d'accéder directement aux deux nœuds contenant les occurrences des deux symboles composant p_{max} qui seront remplacés par le nouveau symbole issu de la fusion.

La méthode `update:` parcourt chaque couple de références `p` de `P`. Elle appelle d'abord la méthode `updateTokenFrequenciesAndFrequenciesPositions:` en lui passant comme paramètre l'objet `Token` contenu dans le nœud référencé par le premier élément de `p`. Elle appelle ensuite une seconde fois cette même méthode en lui passant l'objet `Token` contenu dans le nœud référencé par le second élément de `p`. Ce processus est répété pour tous les couples de références de `P`.

L'implémentation de `updateTokenFrequenciesAndFrequenciesPositions:` est la suivante :

```
updateTokenFrequenciesAndFrequenciesPositions: aToken
    tokenFrequencies
    add: aToken token
```

```

withOccurrences: aToken occurrence negated .

(tokenFrequencies occurrencesOf: aToken token ) = 0
ifTrue: [
    tokenFrequencies
    removeKey: aToken token
    ifAbsent: [].
tokenPositions
at: aToken token
update: [:set|
    set
    remove: aToken position
    ifAbsent:[] ;
yourself ].

(tokenPositions at: aToken token ) size = 0
ifTrue: [ tokenPositions removeKey: aToken token ]

```

Cette méthode reçoit en paramètre un objet de la classe **Token**, correspondant à l'une des occurrences d'un symbole composant p_{max} qui doit disparaître après la fusion.

Elle récupère tout d'abord le symbole stocké dans la variable d'instance **token** de cet objet. Elle décrémente ensuite dans **tokenFrequencies** la fréquence associée à ce symbole de la valeur stockée dans la variable d'instance **occurrence** de l'objet **Token**. Cette diminution correspond à la suppression de cette occurrence du symbole dans le corpus après la fusion.

La méthode supprime également, de la valeur associée au symbole dans **tokenPositions**, la référence stockée dans la variable d'instance **position** de l'objet **Token**. Cette référence correspond au nœud de la liste doublement chaînée contenant cette occurrence du symbole.

d) Mise à jour du corpus, de **pairFrequencies** et de **pairPositions**

Cette opération consiste à remplacer chaque occurrence de p_{max} , c'est-à-dire chaque occurrence où les deux symboles composant p_{max} apparaissent dans deux nœuds consécutifs de la liste doublement chaînée, par le nouveau symbole résultant de la combinaison de ces deux symboles.

Les occurrences de p_{max} à traiter sont obtenues directement à partir de **pairPositions**. La valeur associée à p_{max} dans ce dictionnaire contient les couples de références vers les deux nœuds consécutifs de la liste doublement chaînée qui contiennent chaque occurrence de cette paire.

Ces références permettent d'accéder directement aux emplacements où p_{max} apparaît dans la liste doublement chaînée afin d'effectuer les modifications nécessaires sans avoir à parcourir l'ensemble des nœuds du **corpus**.

La modification du **corpus** entraîne une mise à jour des paires de symboles consécutifs présentes. Lors du remplacement d'une occurrence de p_{max} , le nœud contenant le premier symbole de p_{max} est conservé dans la liste doublement chaînée mais sa valeur est remplacée par le nouveau symbole issu de la combinaison des deux symboles de p_{max} . Le nœud contenant le second symbole est ensuite retiré de la liste doublement chaînée, et les liens entre les nœuds voisins sont mis à jour.

Cette modification peut entraîner la disparition de certaines occurrences de paires de symboles consécutifs qui existaient avant la mise à jour du **corpus**. Leurs fréquences doivent donc être diminuées dans **pairFrequencies**, et les couples de références correspondants doivent être supprimés de **pairPositions**.

D'autre part, de nouvelles paires de symboles consécutifs apparaissent à cause de l'apparition du nouveau symbole. Ces nouvelles paires correspondent principalement aux paires formées entre le nouveau symbole et ses voisins dans la liste doublement chaînée. Elles doivent être ajoutées dans **pairFrequencies** et **pairPositions**.

Pour effectuer ces mises à jour efficacement, il n'est pas nécessaire de parcourir tous les nœuds de la liste doublement chaînée. Les informations stockées dans **pairPositions** permettent d'accéder directement aux nœuds contenant les occurrences de p_{max} . À partir de ces nœuds, il est possible d'effectuer la fusion, d'identifier les paires dont la fréquence diminue et de déterminer les nouvelles paires créées après la fusion.

Dans **WordPieceTrain**, la méthode **update** : réalise ces mises à jour en appelant la méthode **update** : héritée de la classe **BPETrain**. Cette dernière effectue la modification de la liste doublement chaînée ainsi que la mise à jour de **pairFrequencies** et **pairPositions**. La description détaillée de cette méthode est présentée dans le chapitre consacré à l'optimisation de BPE.

e) **Mise à jour des scores des paires de symboles contenues dans affectedPairSet**

À cette étape, **tokenFrequencies** et **pairFrequencies** ont déjà été mises à jour. Les informations nécessaires au calcul des scores sont donc disponibles. Il reste alors à mettre à jour les scores des paires de symboles contenues dans **affectedPairSet**, c'est-à-dire les paires dont le score peut avoir été modifié à la suite de la fusion de p_{max} .

Pour cela, la méthode **update** : parcourt chaque paire de symboles de **affectedPairSet** et appelle la méthode **updatePairsScoresFrom** : en lui passant cette paire en paramètre.

La méthode **updatePairsScoresFrom** : vérifie d'abord si la paire de symboles existe encore dans **pairFrequencies**. Si sa fréquence est devenue nulle, cela signifie que

cette paire n'existe plus dans le corpus ; cette paire est alors supprimée de **pairScores**. Dans le cas contraire, son score est recalculé à partir de sa fréquence mise à jour dans **pairFrequencies** ainsi que des fréquences mises à jour des deux symboles qui la composent dans **tokenFrequencies**. La valeur associée à cette paire dans **pairScores** est ensuite remplacée par ce nouveau score.

Son implémentation dans Pharo est la suivante :

```
updatePairsScoresFrom: pair
  (pairFrequencies occurrencesOf: pair) = 0
    ifTrue: [
      pairScores
        removeKey: pair
        ifAbsent: [] ]
    ifFalse: [
      pairScores
        at: pair
        put: (pairFrequencies occurrencesOf: pair )
          /(((tokenFrequencies occurrencesOf: pair first)
            * (tokenFrequencies occurrencesOf: pair second )) asFloat] .
```

f) Ajout du nouveau symbole dans **tokenFrequencies** et **tokenPositions**

À l'issue de la mise à jour du **corpus**, les occurrences de p_{max} ont été remplacées par le nouveau symbole obtenu après la fusion de ses deux éléments. Cependant, ce nouveau symbole n'a pas encore été pris en compte dans **tokenFrequencies** et **tokenPositions**. Ces deux structures doivent pourtant contenir, à chaque itération de l'entraînement, l'ensemble des symboles présents dans le corpus ainsi que leurs fréquences et les positions de leurs occurrences.

Pour effectuer cette mise à jour sans parcourir l'ensemble de la liste doublement chaînée, la méthode **update:** réutilise P, la valeur associée à p_{max} dans **pairPositions**, récupérée avant la mise à jour du **corpus**. Pour rappel, P est un ensemble de couples de références vers les noeuds contenant les occurrences de p_{max} . Après l'exécution de **super update:**, le nœud référencé par le premier élément de chaque couple est conservé dans la liste doublement chaînée et contient désormais le nouveau symbole. Les références contenues dans P permettent donc d'accéder directement aux occurrences du nouveau symbole.

La méthode **update:** parcourt ensuite chaque couple de références p de P et appelle la méthode **addToTokenFrequenciesAndTokenPositions:** en lui passant comme paramètre la valeur du nœud référencé par le premier élément de p, qui est une instance de **Token** contenant le nouveau symbole.

Son implémentation en Pharo est la suivante :

```

addToTokenFrequenciesAndTokenPositions: aToken
  tokenFrequencies
  add: aToken token
  withOccurrences: aToken occurrence .
  tokenPositions
  at: aToken token
  update: [:set|
    set
    add: aToken position ;
    yourself ]
  initial: (Set with: aToken position)

```

La méthode `addToTokenFrequenciesAndTokenPositions:` reçoit en paramètre une instance de `Token`. Elle récupère le symbole contenu dans son attribut `token` et ajoute le symbole à `tokenFrequencies` avec un nombre d'occurrences égal à la valeur de l'attribut `occurrence` de l'objet `Token`.

La méthode met ensuite à jour `tokenPositions`. Si le symbole est déjà présent dans ce dictionnaire, la référence contenue dans l'attribut `position` de l'objet `Token` est ajoutée à l'ensemble des références qui lui est associé. Dans le cas contraire, la méthode ajoute le symbole dans `tokenPositions` et lui associe un ensemble contenant la référence stockée dans l'attribut `position` de l'objet `Token`.

À l'issue de cette étape, `tokenFrequencies` et `tokenPositions` reflètent de nouveau l'état courant du corpus en prenant en compte les occurrences du nouveau symbole créées par la fusion.

g) Ajout des nouvelles paires de symboles consécutifs dans `pairScores`

À ce stade de l'algorithme, `pairScores` ne contient pas encore les nouvelles paires de symboles consécutifs apparues après la fusion de p_{max} . Il est donc nécessaire de les ajouter afin que `pairScores` reflète l'état courant du corpus.

Pour effectuer cette mise à jour sans parcourir l'ensemble de la liste doublement chaînée, la méthode `update:` réutilise les informations contenues dans `P`, la valeur associée à p_{max} dans `pairPositions`, récupérée avant la mise à jour du corpus. Après l'exécution de `super update:`, le nœud référencé par le premier élément de chaque couple de références de `P` contient désormais le nouveau symbole obtenu par la fusion. Les références contenues dans `P` permettent ainsi d'accéder directement aux nœuds contenant les occurrences de ce nouveau symbole et d'identifier les nouvelles paires de symboles consécutifs le contenant.

Pour chaque couple de références $p \in P$, la méthode `update:` tente de créer deux paires de symboles contenant le nouveau symbole :

— elle envoie le message `tryAddPairsScoreWith:in:` à la valeur du nœud référencé

par le premier élément de `p`, en lui passant comme premier paramètre la valeur du nœud précédent du nœud référencé par le premier élément de `p` et comme second paramètre l'instance courante de `WordPieceTrain`;

- elle envoie ensuite le même message à la valeur du nœud suivant du nœud référencé par le premier élément de `p`, en lui passant comme premier paramètre la valeur du nœud référencé par le premier élément de `p` et comme second paramètre l'instance courante de `WordPieceTrain` .

Chaque nœud de la liste doublement chaînée contient soit une instance de `Token`, soit une instance de `EndOfWord`. La méthode `tryAddPairsScoreWith:in:` utilise le polymorphisme pour adapter son comportement selon le type de l'objet qui reçoit le message.

Si le receveur est une instance de `EndOfWord`, la méthode ne réalise aucune opération :

```
tryAddPairsScoreWith: anObject in: aWordPiece
    ^ self
```

Si le receveur est une instance de `Token`, la méthode vérifie d'abord que son premier paramètre n'est pas `NIL`. Si cette condition est satisfaite, elle envoie le message `addPairsScoreWith:in:` à ce premier paramètre en lui passant comme arguments l'objet `Token` qui a reçu `tryAddPairsScoreWith:in:` ainsi que son second paramètre, une instance de `WordPieceTrain`.

```
tryAddPairsScoreWith: anObject in: aWordPiece
    anObject ifNil: [ ^ self ].
    anObject addPairsScoreWith: self in: aWordPiece
```

La méthode `addPairsScoreWith:in:` utilise également le polymorphisme.

Si le receveur est une instance de `EndOfWord`, elle ne réalise aucune opération :

```
addPairsScoreWith: anObject in: aWordPiece
    ^ self
```

Si le receveur est une instance de `Token`, elle délègue l'ajout de la nouvelle paire à la méthode `addNewPairScoresFrom:and::` :

```
addPairsScoreWith: anObject in: aWordPiece
    aWordPiece addNewPairScoresFrom: self and: anObject
```

La méthode `addNewPairScoresFrom:and:` reçoit deux instances de `Token`. Elle construit la paire de symboles à partir des valeurs contenues dans leurs attributs `token`. Si cette

paire existe dans `pairFrequencies`, elle calcule son score à l'aide de sa fréquence dans `pairFrequencies` ainsi que des fréquences de chacun de ses deux symboles dans `tokenFrequencies`, puis ajoute cette paire dans `pairScores` en lui associant ce score.

Grâce aux références contenues dans `P`, seules les nouvelles paires impliquant le symbole créé par la fusion sont examinées. Il n'est donc pas nécessaire de parcourir l'ensemble de la liste doublement chaînée pour mettre à jour `pairScores`.

h) update : supprime p_{max} de `pairScores`.

6.2 Optimisation de l'encodage de WordPiece

6.2.1 Problème

L'algorithme WordPiece original utilise le principe du maximum matching (MaxMatch) pour encoder un mot. À chaque étape, il recherche le plus long préfixe du mot restant qui appartient au vocabulaire. Une fois ce préfixe trouvé, il l'ajoute au résultat, puis répète ce processus jusqu'à ce que le mot soit entièrement tokenisé.

Le principal inconvénient de cette approche est que certains caractères du mot peuvent être parcourus plusieurs fois au cours de la tokenisation. En effet, lorsqu'un préfixe est extrait, la recherche recommence depuis le début du suffixe restant, ce qui conduit à réexaminer des caractères déjà traités. Ces parcours répétés engendrent des traitements redondants et augmentent le temps d'exécution de l'algorithme.

Afin d'illustrer ce problème, nous reprenons l'exemple présenté dans l'article Fast WordPiece Tokenization. Le vocabulaire utilisé est $V = \{a, \#\#b, \#\#c, \#\#cdy, \#\#dz, abcdx\}$ et le mot à tokeniser est `abcdz`.

Les figures suivantes représentent ce vocabulaire sous la forme d'un trie. Les contours des nœuds correspondant à des chaînes appartenant au vocabulaire sont colorés en bleu, tandis que les nœuds colorés en rouge représentent les caractères parcourus pendant la recherche du plus long préfixe.

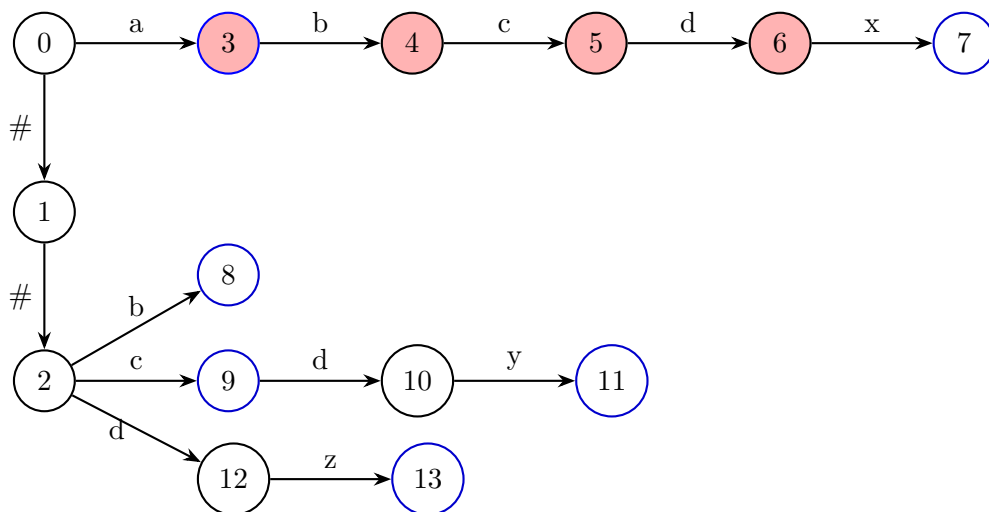


FIGURE 6.5 – Trie construit à partir du vocabulaire. La correspondance de la chaîne "abcdz" débute au nœud 0.

Dans la figure 6.5, la recherche commence au nœud 0. L'algorithme suit successivement les arêtes étiquetées a, b, c puis d, ce qui le conduit jusqu'au nœud 6. Il tente ensuite de suivre une arête étiquetée z, mais une telle arête n'existe pas.

Bien que les caractères a, b, c et d aient été parcourus, la chaîne abcd n'appartient pas au vocabulaire. Le dernier nœud correspondant à un token du vocabulaire rencontré durant cette recherche est donc le nœud représentant le token a. Le plus long préfixe reconnu est ainsi a, qui est ajouté au résultat de la tokenisation.

Il reste alors à tokeniser la chaîne bcdz. La tokenisation de cette chaîne est illustrée par la figure 6.6. Comme cette sous-chaîne se situe au milieu du mot, WordPiece recherche des tokens préfixés par `##`. La recherche reprend donc à partir du nœud représentant `##`, le nœud 2, puis suit l'arête étiquetée b, ce qui conduit au nœud 8.

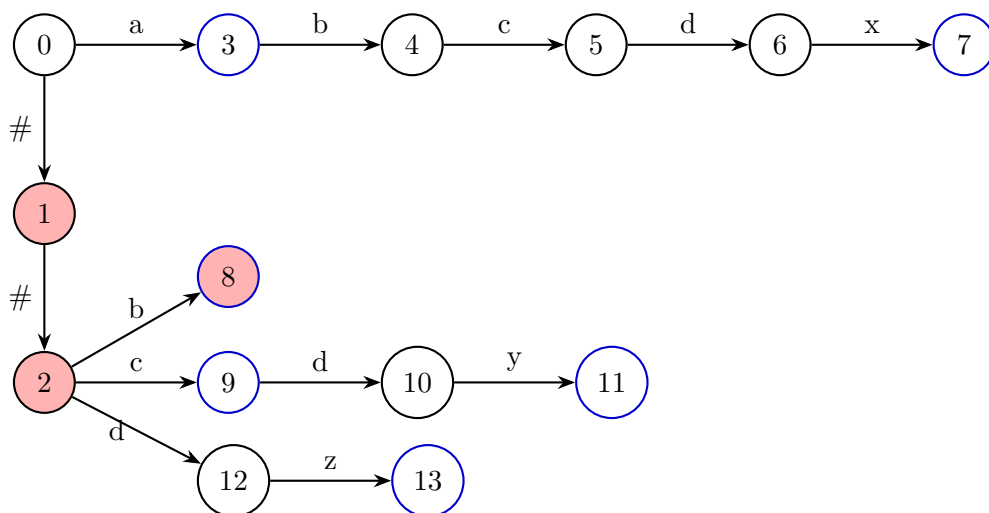


FIGURE 6.6 – Trie construit à partir du vocabulaire. La correspondance de la chaîne "bcdz" débute au nœud 2.

Dans la figure 6.6, à partir du nœud 8, l'algorithme tente de suivre une arête étiquetée c.

Aucune arête de ce type n'existe, la recherche s'arrête donc. Le plus long préfixe appartenant au vocabulaire est `##b`, qui est ajouté au résultat. Le résultat de la tokenisation contient désormais les tokens `a` et `##b`.

L'algorithme poursuit ensuite la tokenisation du suffixe restant `cdz`, illustrée par la figure 6.7.

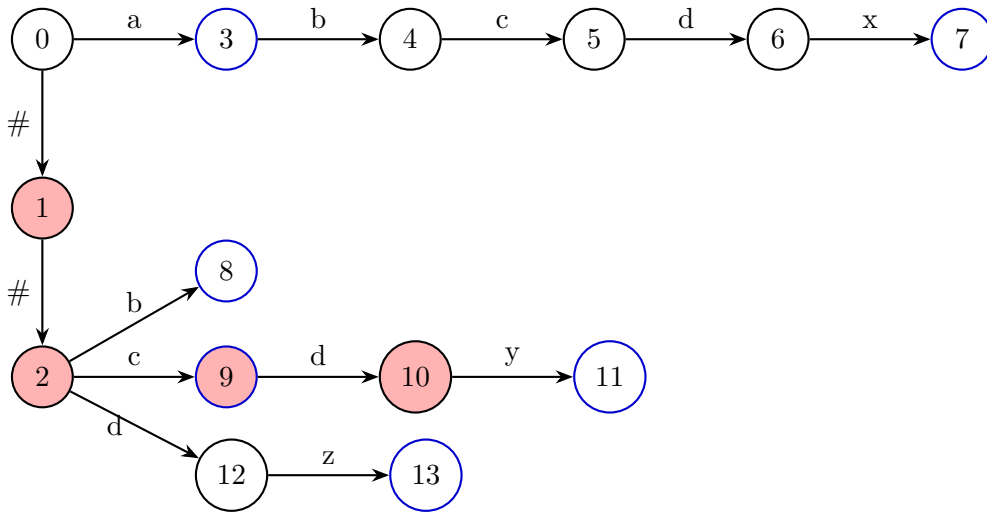


FIGURE 6.7 – Trie construit à partir du vocabulaire. La correspondance de la chaîne "cdz" débute au nœud 2.

La recherche repart du nœud représentant `##`, le nœud 2, puis suit l'arête étiquetée `c`, ce qui conduit au nœud 9. Ensuite, l'arête étiquetée `d` mène au nœud 10. L'algorithme tente alors de suivre une arête étiquetée `z`, mais celle-ci n'existe pas. Le plus long préfixe reconnu est donc `##c`, qui est ajouté au résultat.

Le résultat de la tokenisation est maintenant constitué des tokens `a`, `##b` et `##c`. Il reste alors à tokeniser le suffixe `dz`. La tokenisation de cette chaîne est illustrée par la figure 6.8.

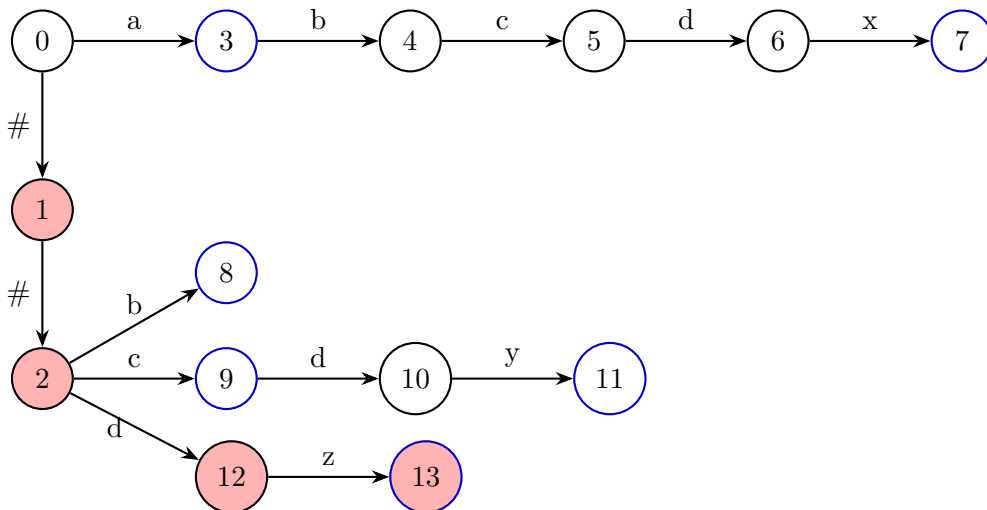


FIGURE 6.8 – Trie construit à partir du vocabulaire. La correspondance de la chaîne "dz" débute au nœud 2.

La recherche reprend une nouvelle fois à partir du nœud représentant `##`, puis suit l'arête étiquetée `d`, ce qui conduit au nœud 12. L'arête étiquetée `z` mène ensuite au nœud 13. Tous les

caractères de la chaîne `##dz` ont été reconnus et cette chaîne appartient au vocabulaire. Le token `##dz` est alors ajouté au résultat.

Le mot est finalement tokenisé sous la forme : `[a, ##b, ##c, ##dz]`

Dans notre exemple, le caractère `b` est parcouru deux fois : lors du traitement des chaînes `abcdz` et `##bcdz`. Le caractère `c` est parcouru trois fois : lors du traitement de `abcdz`, `##bcdz` et `##cdz`. De même, les caractères `d` et `z` sont chacun parcourus trois fois. Ces traitements répétés augmentent inutilement le temps de tokenisation, en particulier lorsque les mots sont longs.

Cet exemple met ainsi en évidence les limites de l'algorithme `MaxMatch`. L'idée principale de `LinMaxMatch` consiste précisément à éviter ces retraitements en utilisant des informations pré-calculées, appelées **failure links** et **failure pops**, qui permettent de poursuivre directement la recherche sans revenir sur les caractères déjà analysés.

6.2.2 Solution

Une solution consiste à pré-calculer, après la construction du trie à partir du vocabulaire, deux informations associées à chaque nœud : le **failure link** et le **failure pop**.

- **Failure link**. Lorsque la recherche du plus long préfixe d'une chaîne ne peut plus se poursuivre à partir d'un nœud $\mathbf{v}$, c'est-à-dire qu'il n'existe pas d'arête sortante de $\mathbf{v}$ étiquetée par le caractère courant, le failure link permet de poursuivre la recherche à partir d'un autre nœud du trie, sans avoir à reparcourir les caractères déjà traités.
- **Failure pop**. Lorsque la recherche ne peut plus se poursuivre à partir d'un nœud $\mathbf{v}$, le failure pop permet de récupérer les jetons qui auraient été produits par l'algorithme `MaxMatch` pour une partie de la chaîne représentée par ce nœud, avant de poursuivre la recherche à partir du nœud indiqué par le failure link.

Ces deux informations permettent d'éviter de reparcourir des caractères déjà traités lorsqu'un échec est rencontré au cours de la recherche du plus long préfixe appartenant au vocabulaire. Elles sont particulièrement utiles lorsque le plus long préfixe reconnu se trouve loin du nœud où la recherche s'est arrêtée, comme dans l'exemple de la tokenisation du mot `abcdz` présenté précédemment.

Les failure links et les failure pops sont calculés une seule fois après la construction du trie à partir du vocabulaire. Ils dépendent uniquement de ce vocabulaire et sont indépendants du texte qui sera ensuite tokenisé.

6.2.2.1 Définition

Soit $\mathbf{v}$ un nœud du trie représentant la chaîne χ_v .

Le failure pop de $\mathbf{v}$ est la plus courte liste non vide de jetons correspondant aux plus longs préfixes pouvant être successivement extraits du début de χ_v , jusqu'à ce que le suffixe restant soit représenté par un autre nœud $\mathbf{v}^*$ du trie.

Le failure link de $\mathbf{v}$ est ce nœud $\mathbf{v}^*$.

S'il n'existe aucune telle liste non vide, alors le failure pop de v est vide et le failure link de v est également vide.

Après la construction du trie à partir du vocabulaire, tous les failure links et tous les failure pops sont initialement vides. Ils sont ensuite calculés pour chacun des nœuds du trie en appliquant la définition précédente.

6.2.2.2 Exemple sur le calcul des failures link et des failures pop

La Figure 6.9 illustre les failure links et les failure pops associés à chaque nœud du trie construit à partir du vocabulaire $V = \{a, \#\#b, \#\#c, \#\#cdy, \#\#dz, abcdx\}$ de l'exemple précédent. Les chaînes représentées par les nœuds dont le contour est coloré en bleu appartiennent au vocabulaire. Pour chaque nœud, le rectangle de même couleur indique son failure pop, tandis que la flèche de même couleur représente son failure link.

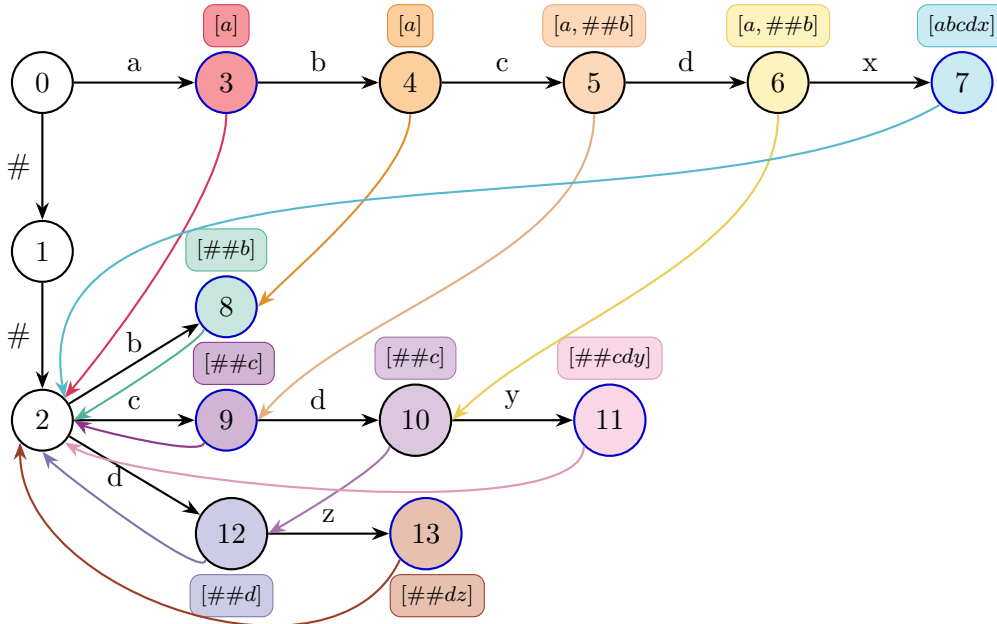


FIGURE 6.9 – Trie construit à partir du vocabulaire $V = \{a, \#\#b, \#\#c, \#\#cdy, \#\#dz, abcdx\}$, avec les failure links et les failure pops associés à chaque nœud.

Les failure links et les failure pops des nœuds 0, 1 et 2 sont vides.

Calcul du failure pop et du failure link du nœud 3

Le nœud 3 représente la chaîne a . Pour calculer son failure pop, on applique la définition précédente. Comme a appartient au vocabulaire, MaxMatch produit directement le jeton a . Après avoir retiré ce jeton, il ne reste plus aucun caractère. Comme ce suffixe vide ne correspond pas au début d'un mot, il est considéré comme précédé du préfixe $\#\#$, représenté par le nœud 2 du trie. Ainsi, le failure pop du nœud 3 est $[a]$ et son failure link pointe vers le nœud 2.

Calcul du failure pop et du failure link du nœud 4

Le nœud 4 représente la chaîne ab . Le mot ab n'appartenant pas au vocabulaire, MaxMatch produit le jeton a . Après son retrait, le suffixe restant est b . Comme il ne se trouve plus au

début d'un mot, il devient $##b$, lequel est représenté par le nœud 8 du trie. Le failure pop du nœud 4 est donc $[a]$ et son failure link pointe vers le nœud 8.

Calcul du failure pop et du failure link du nœud 5

Le nœud 5 représente la chaîne abc . MaxMatch produit d'abord le jeton a . Le suffixe restant est alors $##bc$. Cette chaîne n'étant représentée par aucun nœud du trie, MaxMatch est appliqué de nouveau et produit le jeton $##b$. Le suffixe restant devient $##c$, représenté par le nœud 9. Le failure pop du nœud 5 est donc $[a, ##b]$ et son failure link pointe vers le nœud 9.

Calcul du failure pop et du failure link du nœud 6

Le nœud 6 représente la chaîne $abcd$. MaxMatch produit d'abord le jeton a . Le suffixe restant est $##bcd$, qui n'est représenté par aucun nœud du trie. MaxMatch produit alors le jeton $##b$. Le suffixe restant devient $##cd$, représenté par le nœud 10. Le calcul s'arrête à ce niveau. Le failure pop du nœud 6 est $[a, ##b]$ et son failure link pointe vers le nœud 10.

Calcul du failure pop et du failure link du nœud 7

Le nœud 7 représente la chaîne $abcdx$. Cette chaîne appartient au vocabulaire; MaxMatch produit donc directement le jeton $abcdx$. Après son retrait, il ne reste plus aucun caractère. Le suffixe vide est représenté par le nœud 2. Le failure pop du nœud 7 est donc $[abcdx]$ et son failure link pointe vers le nœud 2.

Calcul du failure pop et du failure link du nœud 8

Le nœud 8 représente la chaîne $##b$. Cette chaîne appartient au vocabulaire; MaxMatch produit donc directement le jeton $##b$. Après son retrait, il ne reste plus aucun caractère. Le suffixe vide est représenté par le nœud 2. Le failure pop du nœud 8 est $[##b]$ et son failure link pointe vers le nœud 2.

Calcul du failure pop et du failure link du nœud 9

Le nœud 9 représente la chaîne $##c$. Cette chaîne appartient au vocabulaire; MaxMatch produit donc directement le jeton $##c$. Après son retrait, le suffixe restant est vide et correspond au nœud 2. Le failure pop du nœud 9 est $[##c]$ et son failure link pointe vers le nœud 2.

Calcul du failure pop et du failure link du nœud 10

Le nœud 10 représente la chaîne $##cd$. Cette chaîne n'appartient pas au vocabulaire. Son plus long préfixe appartenant au vocabulaire est $##c$, qui est produit par MaxMatch. Après son retrait, le suffixe restant est $##d$, représenté par le nœud 12 du trie. Le failure pop du nœud 10 est donc $[##c]$ et son failure link pointe vers le nœud 12.

Calcul du failure pop et du failure link du nœud 11

Le nœud 11 représente la chaîne $##cdy$. Cette chaîne appartient au vocabulaire. MaxMatch produit donc directement le jeton $##cdy$. Après son retrait, le suffixe restant est vide et correspond au nœud 2. Le failure pop du nœud 11 est $[##cdy]$ et son failure link pointe vers le nœud 2.

Calcul du failure pop et du failure link du nœud 12

Le nœud 12 représente la chaîne $##d$. Aucun préfixe non vide de cette chaîne n'appartient

au vocabulaire. D'après la définition, le failure pop est donc vide. Par conséquent, le failure link est également vide.

Calcul du failure pop et du failure link du nœud 13

Le nœud 13 représente la chaîne `##dz`. Cette chaîne appartient au vocabulaire ; MaxMatch produit donc directement le jeton `##dz`. Après son retrait, le suffixe restant est vide et correspond au nœud 2. Le failure pop du nœud 13 est `[##dz]` et son failure link pointe vers le nœud 2.

6.2.2.3 Exemples sur la tokenisation en utilisant l'approche optimisée

Dans les exemples suivants, on considère le vocabulaire

$V = a, ##b, ##c, ##cdy, ##dz, abcdx$.

Les deux exemples utilisent ce même vocabulaire ; seule la chaîne à tokeniser diffère d'un exemple à l'autre.

Conformément à l'algorithme WordPiece, si une chaîne ne peut pas être entièrement tokenisée à l'aide des jetons du vocabulaire, son encodage est remplacé par le jeton `[UNK]`.

Les exemples suivants utilisent les failure links et les failure pops calculés dans l'exemple précédent afin d'illustrer le fonctionnement de l'algorithme LinMaxMatch lors de la tokenisation.

Exemple 1

Supposons que la chaîne à tokeniser soit `abcdz`.

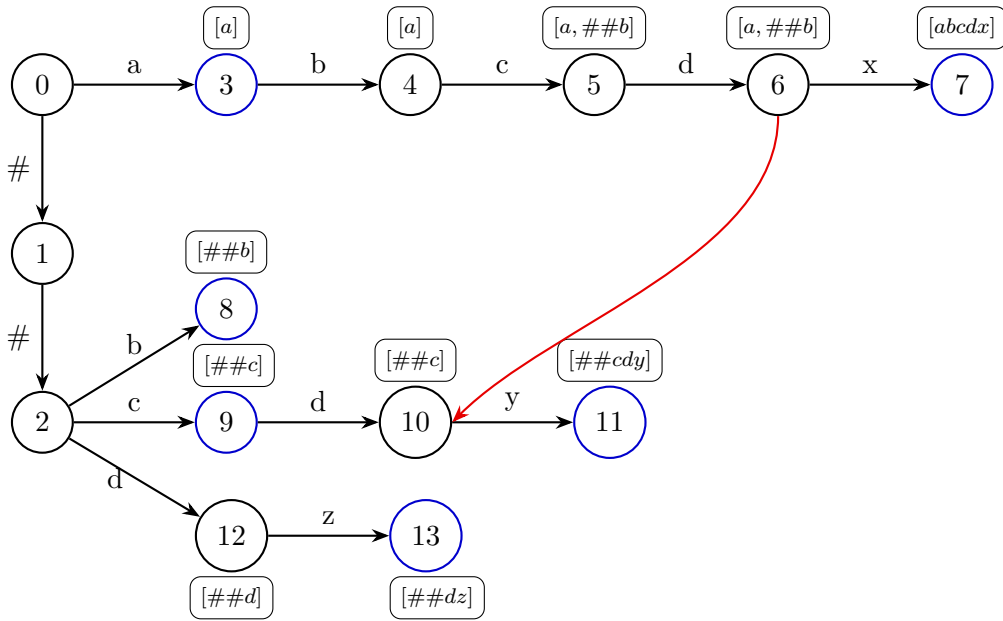
La tokenisation commence à la racine du trie (nœud 0). On lit successivement les caractères de la chaîne :

- le caractère `a` conduit au nœud 3 ;
- le caractère `b` conduit au nœud 4 ;
- le caractère `c` conduit au nœud 5 ;
- le caractère `d` conduit au nœud 6.

Le caractère suivant est `z`. Aucune arête sortante du nœud 6 n'est étiquetée par ce caractère. La recherche ne peut donc plus se poursuivre à partir de ce nœud.

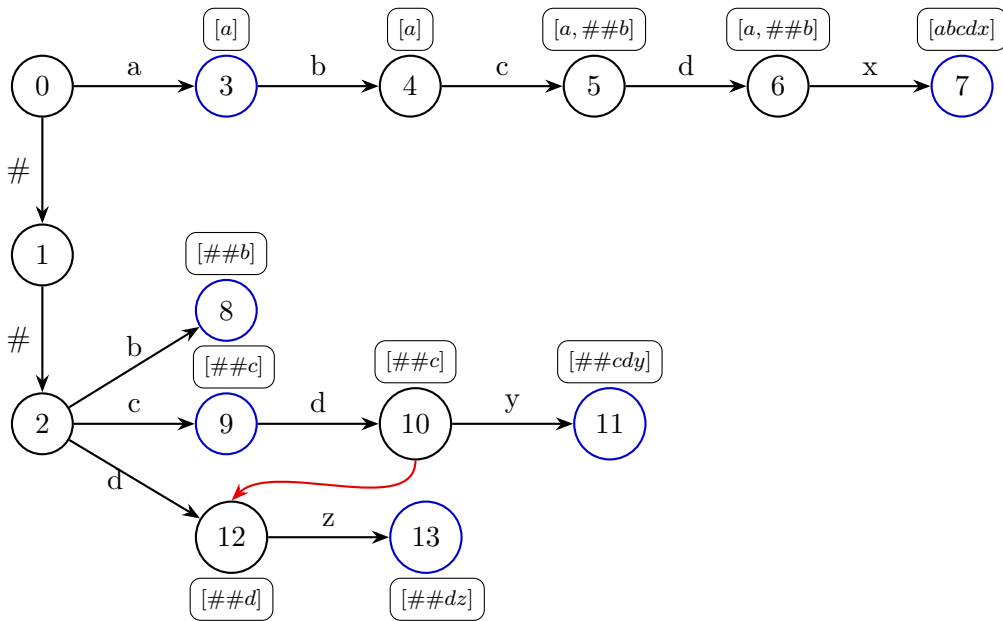
On récupère alors le failure pop du nœud 6, qui est `[a, ##b]`, et on l'ajoute au résultat de la tokenisation. Les deux premiers jetons ayant été produits, le suffixe restant à traiter est `##cdz`.

La recherche reprend ensuite directement à partir du failure link du nœud 6, qui est le nœud 10.



Depuis le nœud 10, le caractère suivant à lire est z. Aucune arête sortante de ce nœud n'est étiquetée par z. On récupère alors le failure pop du nœud 10, qui est $[##c]$, et on l'ajoute au résultat, qui devient : $[a, ##b, ##c]$.

Après avoir retiré ce jeton, le suffixe restant est $##dz$. La recherche reprend alors à partir du failure link du nœud 10, qui est le nœud 12.



Depuis le nœud 12, le caractère suivant est z. Une arête étiquetée z conduit au nœud 13, qui représente le jeton $##dz$ appartenant au vocabulaire. Ce jeton est ajouté au résultat : $[a, ##b, ##c, ##dz]$.

Tous les caractères de la chaîne ayant été consommés, la tokenisation est terminée. Le résultat final est donc : $[a, ##b, ##c, ##dz]$.

Exemple 2

Supposons maintenant que la chaîne à tokeniser soit $abcz$.

La recherche commence à la racine du trie.

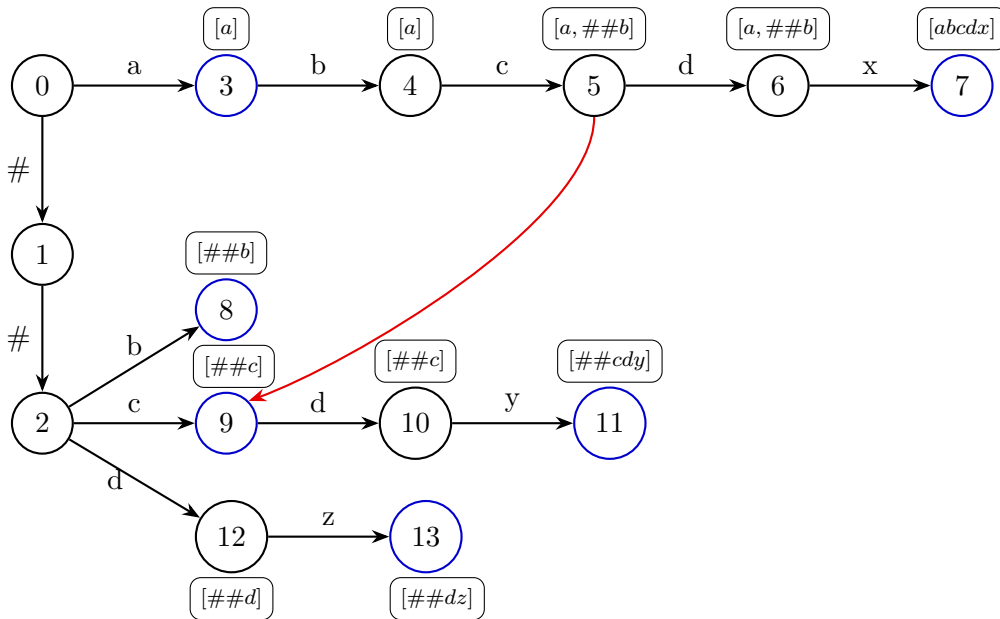
Les caractères a, b et c conduisent successivement aux nœuds 3, 4 puis 5.

Le caractère suivant est z. Aucune arête sortante du nœud 5 n'est étiquetée par ce caractère.

La recherche ne peut donc plus se poursuivre.

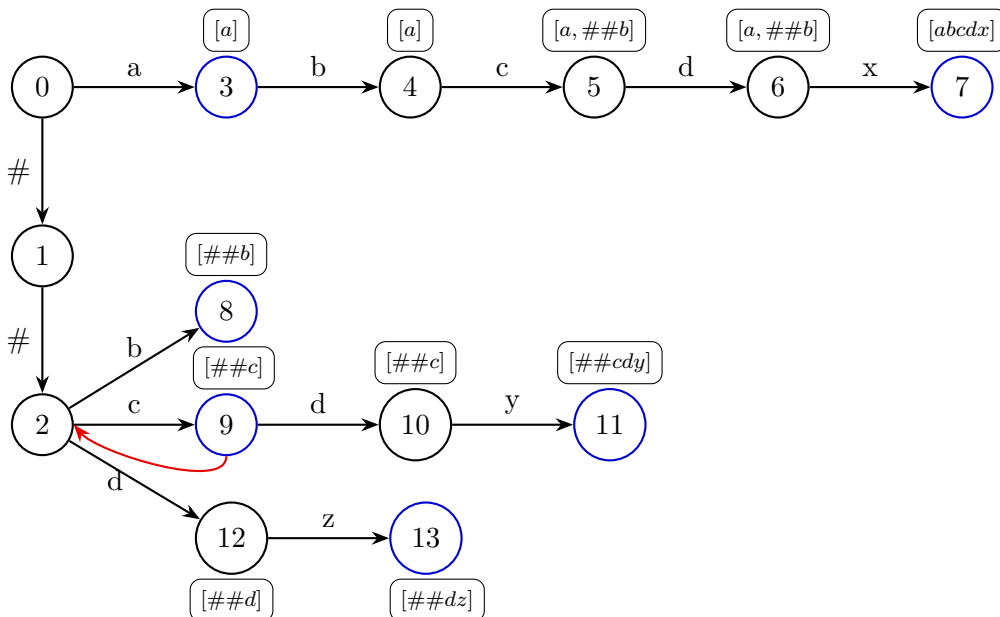
On récupère alors le failure pop du nœud 5, qui est $[a, \#\#b]$, et on l'ajoute au résultat. Le suffixe restant à traiter est alors $\#\#cz$.

La recherche reprend ensuite à partir du failure link du nœud 5, qui est le nœud 9.



Depuis le nœud 9, le caractère suivant est z. Aucune arête sortante de ce nœud n'est étiquetée par ce caractère. On récupère alors le failure pop du nœud 9, qui est $\#\#c]$, et on l'ajoute au résultat, qui devient : $[a, \#\#b, \#\#c]$.

Après avoir retiré ce jeton, le suffixe restant est $\#\#z$. La recherche reprend alors à partir du failure link du nœud 9, qui est le nœud 2.



Depuis le nœud 2, aucune arête sortante n'est étiquetée par z. Le failure pop du nœud 2 est vide et son failure link est également vide. Il n'est donc plus possible de poursuivre la recherche dans le trie.

Le résultat partiel obtenu est : [a, ##b, ##c].

Cependant, la chaîne n'a pas été entièrement tokenisée, puisque le suffixe ##z ne peut pas être représenté par des jetons du vocabulaire. Conformément à l'algorithme WordPiece, la tokenisation de la chaîne est alors remplacée par le jeton unique : [UNK].

6.2.3 Fonctionnement

1. Construction du trie

Le trie est d'abord construit à partir du vocabulaire.

Les arêtes du trie dont l'étiquette est un caractère de ponctuation sont supprimées. Cette suppression permet d'éviter qu'une ponctuation soit intégrée à un token plus long lors de l'encodage d'un texte. Ensuite, pour chaque caractère de ponctuation considéré par le tokenizer, on ajoute un nœud directement accessible depuis la racine du trie. Ces nœuds permettent de reconnaître les ponctuations, qu'elles appartiennent ou non au vocabulaire.

2. Calcul des failureLink et des failurePop

Pour chaque nœud v, on considère la chaîne représentée par le chemin allant de la racine jusqu'à v.

- Si la chaîne appartient au vocabulaire, alors la chaîne représentée par v constitue directement un token. Le failurePop de v est alors une liste contenant cette chaîne comme élément. Le failureLink de v est défini vers le nœud représentant le suffixe indicateur '##'.

Autrement dit, lorsqu'on atteint un token complet, celui-ci peut être ajouté aux résultats et la recherche peut se poursuivre en considérant que les prochains caractères appartiennent au milieu d'un mot.

- Sinon, comme la chaîne représentée par u, le nœud parent de v, est le préfixe de la chaîne représentée par v, le calcul de la failurePop de v utilise le failureLink et le failurePop déjà calculés pour u. L'algorithme suit le failureLink de u et recherche, à partir du nœud obtenu, une transition portant le caractère qui relie u à v. Si cette transition n'existe pas, il suit le failureLink du nœud courant et répète cette recherche. À chaque fois que l'algorithme suit le failureLink du nœud courant, les éléments du failurePop de ce nœud sont ajoutés à une collection que l'on va noter Z. Lorsqu'une transition est trouvée, le failureLink de v est défini vers le nœud atteint par cette transition et son failurePop est constitué du failurePop de u suivi des éléments de Z.
- Les nœuds représentant les ponctuations ont un traitement particulier. On leur attribue un failureLink vers un nœud spécial. Ce nœud spécial n'a pas de parent, n'a pas d'enfant, ne correspond à aucune chaîne normale du vocabulaire, possède un failurePop et un failureLink vide. Le rôle de ce nœud est simplement d'indiquer qu'une

ponctuation vient d'être reconnue. Pour une ponctuation c , si c appartient au vocabulaire alors son `failurePop` est une liste contenant c . Si c n'appartient pas au vocabulaire, alors son `failurePop` est une liste contenant '`<UNK>`'. Le `failureLink` vers ce nœud spécial permet donc de distinguer le cas où la correspondance s'est arrêtée après avoir reconnu une ponctuation.

3. Encodage d'un mot

Pour encoder un mot, un espace est ajouté à sa fin afin de disposer d'un caractère permettant de détecter la fin de la séquence. Le `tokenizer` parcourt ensuite le mot caractère par caractère en suivant les transitions du trie. Lorsqu'une transition ne peut plus être suivie depuis le nœud courant, les éléments de son `failurePop` sont ajoutés aux tokens produits, puis la recherche se poursuit à partir de son `failureLink`.

Cette procédure est répétée jusqu'à ce que l'une des deux situations suivantes se produise :

- **Le mot est entièrement tokenisé**

Cela se produit lorsque tous les caractères du mot ainsi que l'espace ajouté à sa fin ont été parcourus et que la procédure a permis de déterminer les tokens correspondant à l'ensemble du mot. Le mot est alors considéré comme entièrement tokenisé et les tokens obtenus sont retournés.

- **Le mot ne peut pas être entièrement tokenisé**

Si, lors de la recherche, aucune transition ne permet de poursuivre la correspondance et/ou que le nœud courant ne possède pas de `failureLink`, alors aucun suffixe approprié ne peut être trouvé dans le trie pour poursuivre la tokenisation. Une partie du mot ne peut donc pas être représentée par les tokens du vocabulaire. Dans ce cas, le mot entier est représenté par le token '`<UNK>`'.

Pourquoi ajouter un espace à la fin ?

L'espace n'appartient normalement pas au vocabulaire. Il permet donc de provoquer volontairement l'arrêt de la correspondance lorsque la totalité du mot a été parcourue.

4. Encodage d'un texte

Pour l'encodage d'un texte complet, on évite de réaliser séparément un prétraitement du texte en mots puis une tokenisation `WordPiece` de chaque mot. Ces deux opérations sont réalisées en un seul parcours du texte.

Le `tokenizer` parcourt directement le texte caractère par caractère et utilise les espaces, les caractères blancs et les ponctuations pour déterminer les frontières entre les séquences à tokeniser. Lorsqu'un espace ou un caractère blanc est rencontré, la séquence de caractères précédemment parcourue est traitée selon la procédure d'encodage d'un mot. Les tokens obtenus sont ensuite ajoutés au résultat et le parcours reprend après ce séparateur. Lorsqu'une ponctuation est rencontrée, la séquence de caractères qui la précède est d'abord traitée selon la procédure d'encodage d'un mot. La ponctuation est ensuite traitée séparément comme une unité, en utilisant son `failurePop` pour déterminer le token correspondant, puis le parcours se poursuit avec la séquence de caractères qui la suit.

Ainsi, la présence d’une ponctuation permet de séparer deux unités à tokeniser, même lorsqu’aucun espace ne les sépare dans le texte.

Ainsi, le texte n’est pas préalablement découpé en une collection intermédiaire de mots avant l’application de WordPiece.

Cette approche permet notamment d’éviter la création explicite d’une collection de mots intermédiaires, un parcours supplémentaire du texte pour appliquer la tokenisation WordPiece aux mots obtenus lors du prétraitement et une partie des traitements nécessaires pour parcourir séparément le texte afin d’identifier les frontières entre les mots.

Pourquoi traiter les ponctuations séparément ?

Les ponctuations sont traitées comme des unités distinctes afin de reproduire le comportement de la prétraitement sans effectuer un parcours supplémentaire du texte.

La ponctuation est donc tokenisée séparément du mot qui la précède et de celui qui la suit, même lorsqu’aucun espace ne les sépare.

Rôle du nœud spécial

Lorsqu’un nœud représente une ponctuation, son `failureLink` pointe vers le nœud spécial. Celui-ci ne possède ni parent ni enfant, et son `failureLink` ainsi que son `failurePop` sont vides. Cela permet d’arrêter la recherche à la ponctuation et d’empêcher les caractères suivants d’être considérés comme la continuation de la même séquence.

6.2.4 Réalisation

Cette section présente l’implémentation de l’optimisation de l’encodage WordPiece dans Pharo.

6.2.4.1 Création de la classe

Une classe `WordPieceEncoder` est créée afin de réaliser l’encodage d’un texte à partir du vocabulaire appris lors de l’entraînement de WordPiece.

```
WordPieceTrain << #WordPieceEncoder
slots: { #trie . #failureLink . #failurePop . #punctuationNode };
package: 'Tokenizer'
```

Elle possède les variables d’instance suivantes :

- `trie` : contient le trie construit à partir du vocabulaire appris lors de l’entraînement de WordPiece.
- `failureLink` : contient un dictionnaire associant chaque nœud du trie à son `failureLink`. La clé du dictionnaire est un nœud et la valeur associée est le nœud vers lequel son `failureLink` pointe.

- **failurePop** : contient un dictionnaire associant chaque nœud du trie à son **failurePop**. La valeur associée à un nœud est une collection de tokens à produire lorsque le **failureLink** de ce nœud est utilisé.
- **punctuationNode** : est un nœud spécial de type **CTTrieNode** qui n'appartient pas au trie construit à partir du vocabulaire. Il ne possède ni parent ni enfant et son **failureLink** ainsi que son **failurePop** ne sont pas définis. Il est utilisé comme **failureLink** des nœuds représentant les ponctuations dans le trie.

La classe **WordPieceEncoder** est une sous-classe de **WordPieceTrain**. Elle hérite donc des méthodes et des variables d'instance définies dans **WordPieceTrain**. La variable d'instance **vocabulary**, héritée de **WordPieceTrain**, permet à **WordPieceEncoder** de disposer du vocabulaire sauvegardé lors de l'entraînement afin de réaliser l'encodage.

Ainsi, **WordPieceEncoder** utilise le vocabulaire appris par **WordPieceTrain** pour construire le trie et calculer les **failureLink** et les **failurePop** nécessaires à l'encodage optimisé.

6.2.4.2 Structure de données utilisée

- Une instance de **CTTrie** est utilisée pour représenter le vocabulaire sous la forme d'un trie. Chaque chemin du trie correspond à une séquence de caractères représentant un sous-mot du vocabulaire.
- Une instance de **Dictionary** est utilisée pour associer chaque nœud du trie à son **failureLink**. La clé est le nœud et la valeur est le nœud vers lequel son **failureLink** pointe.
- Une instance de **Dictionary** est utilisée pour associer chaque nœud du trie à son **failurePop**. La clé est le nœud et la valeur est une **OrderedCollection** contenant les tokens à ajouter au résultat lors de l'utilisation du **failureLink** de ce nœud.
- Une **OrderedCollection** est utilisée comme valeur de **failurePop** afin de conserver l'ordre des tokens qui doivent être ajoutés au résultat.
- Une instance de **CTTrieNode**, appelée **punctuationNode**, est utilisée comme nœud spécial pour les **failureLink** des nœuds représentant les ponctuations. Ce nœud spécial n'appartient pas au trie et ne possède ni parent ni enfant.

6.2.4.3 Construction du trie

On commence par construire un trie représentant le vocabulaire. Cette opération est réalisée par la méthode **buildTrieAndInitializeFailureData**. Pour chaque sous-mot du vocabulaire, celui-ci est inséré dans le trie à l'aide de l'envoi **at:put:**. Le trie obtenu constitue ainsi une représentation arborescente des sous-mots du vocabulaire, où les préfixes communs sont partagés entre plusieurs sous-mots.

En plus de construire le trie, la méthode ajoute également les caractères de ponctuation lorsqu'ils ne sont pas déjà présents dans le vocabulaire. Ces caractères sont donc eux aussi représentés par des nœuds du trie. Lorsqu'une ponctuation est ajoutée comme un sous-mot constitué d'un seul caractère, le nœud correspondant est un enfant de la racine.

Une fois le trie construit, la méthode initialise les structures nécessaires à la gestion des échecs lors de la recherche dans le trie. Pour chaque nœud, `failureLink` est initialisé à `nil`, tandis que `failurePop` est initialisé avec une liste vide. À ce stade, ces structures ne contiennent donc encore aucune information ; elles seront calculées ultérieurement.

Pour parcourir l'ensemble des nœuds du trie, la méthode utilise une file et effectue un parcours en largeur (Breadth-First Search, BFS). La racine est d'abord placée dans la file. À chaque itération, le premier nœud de la file est retiré. Si ce nœud possède des enfants, ceux-ci sont ajoutés à la fin de la file. Le parcours se poursuit jusqu'à ce que la file soit vide, ce qui permet de visiter tous les nœuds du trie et d'initialiser leurs structures `failureLink` et `failurePop`.

6.2.4.4 Calcul des `failureLink` et `failurePop`

La méthode `computeFailureLinkAndFailurePop` est chargée de calculer les valeurs de `failureLink` et de `failurePop` pour les nœuds du trie. Ces informations sont utilisées lors de la recherche d'une correspondance dans le trie lorsqu'une transition directe ne peut pas être effectuée.

Voici l'implémentation de `computeFailureLinkAndFailurePop` dans Pharo :

```
computeFailureLinkAndFailurePop
|queue currentNode r prefixIndicatorNode|
queue := CTQueue new .
queue enqueue: trie rootNode .
r := trie rootNode findChildWithString: $# .
prefixIndicatorNode := r findChildWithString: $# .
queue enqueue: prefixIndicatorNode .
failureLink
at: punctuationNode
put: nil .

[ queue isEmpty ] whileFalse: [
currentNode := queue dequeue .

currentNode children valuesDo: [ :child|
child ~= prefixIndicatorNode ifTrue: [
self
computeFailureLinkAndFailurePopForANode: child
parent: currentNode
prefixIndicatorNode: prefixIndicatorNode.
queue enqueue: child ] .
] .
]
```

La méthode utilise une file afin de parcourir les nœuds du trie selon un parcours en largeur (Breadth-First Search, BFS). La racine du trie est d'abord ajoutée à la file. Le nœud correspondant à la chaîne `##` est également ajouté explicitement à cette file.

Dans WordPiece, le préfixe `##` est utilisé pour indiquer qu'un sous-mot correspond à une continuation d'un mot. Dans le trie utilisé par cette implémentation, le nœud correspondant à `##` joue un rôle particulier dans la construction des `failureLink`.

Tant que la file n'est pas vide, le premier nœud est retiré. Ses enfants sont ensuite parcourus. Pour chaque enfant différent du nœud représentant `##`, la méthode `computeFailureLinkAndFailurePopForANode:parent:prefixIndicatorNode:` est appelée afin de calculer ses informations de failure, puis l'enfant est ajouté à la file. Le test `child ~= prefixIndicatorNode` empêche ainsi l'ajout d'une seconde fois du nœud `##`, qui a déjà été placé dans la file au début du parcours.

Calcul des `failureLink` et `failurePop` d'un nœud

La méthode `computeFailureLinkAndFailurePopForANode:parent:prefixIndicatorNode:` reçoit trois paramètres. Le premier, `aCTTrieNode`, désigne le nœud courant, c'est-à-dire le nœud pour lequel les valeurs de `failureLink` et `failurePop` doivent être calculées. Le deuxième, `parent`, désigne son nœud parent, c'est-à-dire le nœud directement relié au nœud courant par une arête. Le troisième, `prefixIndicatorNode`, correspond au nœud représentant la chaîne `##`.

Voici l'implémentation de `computeFailureLinkAndFailurePopForANode:parent:prefixIndicatorNode:` dans Pharo :

```
computeFailureLinkAndFailurePopForANode: aCTTrieNode parent: parent
prefixIndicatorNode: aCTTrieNode1
| string failureNode tokens|
string := trie labelOfPathFromRootToANode: aCTTrieNode .

string isPunctuation
  ifTrue: [
    failureLink
      at: aCTTrieNode
      put: punctuationNode .

    (vocabulary includes: string)
      ifTrue: [ (failurePop at: aCTTrieNode ) add: string ]
      ifFalse: [ (failurePop at: aCTTrieNode ) add: '<UNK>' ] ]
  ifFalse: [
    ( vocabulary includes: string )
      ifTrue: [
        failureLink
          at: aCTTrieNode
          put: aCTTrieNode1.
```

```

        (failurePop at: aCTTrieNode ) add: string ]
    ifFalse: [
        failureNode := ( failureLink at: parent ) .
        tokens := OrderedCollection new .

        [ failureNode isNotNil
        and: [ ( failureNode findChildWithString: aCTTrieNode character ) isNil ] ]
        whileTrue:[
            tokens := tokens addAllLast: ( failurePop at: failureNode ) .
            failureNode := ( failureLink at: failureNode ) ] .

    failureNode ifNotNil: [
        failureLink
            at: aCTTrieNode
            put: ( failureNode findChildWithString: aCTTrieNode character ) .
        ( failurePop at: aCTTrieNode ) addAll: ( failurePop at: parent ) .
        ( failurePop at: aCTTrieNode ) addAll: tokens
    ] .
].
].
].

```

La méthode commence par déterminer la chaîne représentée par le nœud courant à l'aide de `labelOfPathFromRootToANode:.` Cette méthode reconstruit la chaîne correspondant au chemin allant de la racine jusqu'au nœud courant, en concaténant les étiquettes des arêtes de ce chemin.

Cas où la chaîne représentée par le nœud courant est une ponctuation

Si la chaîne représentée par le nœud courant est une ponctuation, son `failureLink` est fixé au nœud spécial `punctuationNode`. Son `failurePop` est une liste contenant la ponctuation elle-même si cette ponctuation appartient au vocabulaire, et '`<UNK>`' sinon.

Cas où la chaîne représentée par le nœud courant n'est pas une ponctuation

Si la chaîne représentée par le nœud courant n'est pas une ponctuation et appartient au vocabulaire, son `failureLink` est fixé au nœud représentant `##`, et son `failurePop` est une liste contenant la chaîne.

Lorsque la chaîne représentée par le nœud courant n'appartient pas au vocabulaire, l'algorithme doit déterminer vers quel nœud le `failureLink` du nœud courant doit être dirigé. Pour cela, il utilise d'abord le `failureLink` de son parent.

Supposons que l'étiquette de l'arête reliant le nœud parent du nœud courant au nœud courant soit `c`. L'algorithme examine le nœud indiqué par le `failureLink` du nœud parent et recherche à partir de ce nœud un enfant relié par une arête étiquetée avec le caractère `c`. Si cet enfant existe, il constitue le nœud vers lequel le `failureLink` du nœud courant sera dirigé. Sinon, l'algorithme remonte au `failureLink` du nœud actuellement examiné et répète la même

recherche.

À chaque fois qu'un nœud est parcouru sans posséder de transition correspondant au caractère *c*, son `failurePop` est ajouté à une liste *Z*. *Z* conserve donc les éléments de `failurePop` rencontrés pendant la remontée des `failureLink`.

La recherche se poursuit en suivant successivement les `failureLink` et en recherchant, à chaque étape, une transition sortante étiquetée par *c*. Si une telle transition est trouvée, le `failureLink` du nœud courant est défini comme le nœud atteint par cette transition. Le `failurePop` du nœud courant est alors complété avec les éléments du `failurePop` de son parent, puis avec les éléments de *Z*, accumulés lors de la remontée des `failureLink`.

Si aucune transition étiquetée par *c* n'est trouvée et que la remontée des `failureLink` aboutit à nil, aucun nouveau `failureLink` n'est attribué dans et aucun élément supplémentaire n'est ajouté au `failurePop` du nœud courant. Dans l'implémentation actuelle, le `failureLink` et le `failurePop` du nœud courant conservent alors les valeurs qui leur avaient été attribuées lors de leur initialisation.

La méthode `labelOfPathFromRootToANode` :

La méthode `labelOfPathFromRootToANode` : permet d'obtenir la chaîne représentée par un nœud du trie en parcourant le chemin reliant la racine à ce nœud et en concaténant les caractères rencontrés. Voici l'implémentation de `labelOfPathFromRootToANode` : dans `Pharo`. Elle est implémentée dans la classe `CTTrie`.

```
labelOfPathFromRootToANode: aCTTrieNode
|path|
path := self pathFromRootToANode: aCTTrieNode .
^ (String streamContents: [:stream |
  path do: [:each |
    stream nextPutAll: each character asString ]])
```

La méthode `pathFromRootToANode` :

La méthode `pathFromRootToANode` : permet de trouver le chemin entre la racine du trie et le nœud donné en paramètre. L'implémentation de cette méthode dans la classe `CTTrie` est la suivante.

```
pathFromRootToANode: aCTTrieNode
|currentNode path|
currentNode := aCTTrieNode .
path := OrderedCollection new .

currentNode isRoot ifTrue: [ ^ nil ].

[ currentNode isRoot ] whileFalse: [
  path add: currentNode .
```

```
currentNode := currentNode parent ] .
```

```
^ path reverse
```

6.2.4.5 Encodage d'un mot

La méthode `encodeWord:` permet d'encoder un mot à l'aide de l'algorithme `WordPiece`. Elle ajoute d'abord un espace à la fin du mot, puis utilise la méthode `matchLoopFrom:in:` pour rechercher les tokens correspondant au mot dans le trie.

L'espace ajouté à la fin joue un rôle particulier : il permet de détecter la fin du mot. Comme l'espace n'appartient pas aux tokens du vocabulaire, aucune transition du trie ne peut être suivie avec ce caractère. L'algorithme ne pouvant pas suivre une transition normale correspondant à l'espace, il récupère le `failurePop` du nœud courant puis suit son `failureLink`. Il répète cette opération jusqu'à atteindre un nœud dont le `failureLink` est nil.

L'implémentation de `encodeWord` : est la suivante :

```
encodeWord: aString
|string result tokens node i r|
string := aString, ' ' .
result := self matchLoopFrom: 1 in: string .
tokens := result first .
node := result second .
i := result third .
r := trie rootNode findChildWithString: $# .
r := r findChildWithString: $# .

( i <= aString size
or: [ ( { trie rootNode . r } asSet includes: node ) not ] )
  ifTrue: [ tokens := OrderedCollection with: '<UNK>' ] .

^ tokens
```

La méthode `matchLoopFrom:in:` retourne trois informations : les tokens produits pendant le parcours, le dernier nœud atteint dans le trie et la position à laquelle le parcours s'est arrêté.

Ces trois informations permettent ensuite à `encodeWord:` de déterminer si le mot a été entièrement tokenisé. Deux conditions doivent être satisfaites pour considérer la tokenisation comme réussie :

- tous les caractères du mot ainsi que l'espace ajouté à sa fin doivent avoir été parcourus ;
- le dernier nœud atteint doit être soit la racine du trie, soit le nœud représentant '###'.

Si l'une de ces conditions n'est pas satisfaite, le mot ne peut pas être entièrement tokenisé par WordPiece et le résultat est remplacé par une liste contenant le token '<UNK>'.

Fonctionnement de la methode matchLoop : in :

La méthode `matchLoopFrom:in:` parcourt le trie à partir d'une position donnée dans une chaîne. Elle traite les caractères de gauche à droite et cherche, pour chaque caractère, une transition correspondante dans le trie. Lors de l'encodage d'un mot, cette méthode est appelée à partir de la première position du mot. La position de départ n'est cependant pas fixée directement dans la méthode afin de permettre sa réutilisation pour l'encodage d'un texte.

L'implémentation de cette méthode est la suivante :

```
matchLoopFrom: aStartingIndex in: aString
|currentNode tokens i|
currentNode := trie rootNode .
tokens := OrderedCollection new .
i := aStartingIndex .

[ i <= aString size ]
whileTrue: [

    [ ( currentNode findChildWithString: ( aString at: i ) ) isNil ]
    whileTrue: [

        ( failureLink at: currentNode ) ifNil: [ ^ { tokens . currentNode . i } ].

        tokens addAll: ( failurePop at: currentNode ) .
        currentNode := ( failureLink at: currentNode )

    ] .

    currentNode := ( currentNode findChildWithString: ( aString at: i ) ) .
    i := i + 1.

] .

^ { tokens . currentNode . i }
```

Le parcours commence à la racine du trie. Pour chaque caractère de la chaîne, l'algorithme recherche une arête sortante du nœud courant portant le caractère à traiter.

Si une telle arête existe, elle est suivie et le caractère est considéré comme consommé. L'algorithme passe alors au caractère suivant de la chaîne.

Sinon, l'algorithme vérifie si le nœud courant possède un `failureLink`.

Si aucun `failureLink` n'est défini, le parcours s'arrête. La méthode retourne alors les tokens obtenus jusqu'à présent, le dernier nœud atteint et la position du caractère qui n'a pas pu être consommé.

Si un `failureLink` existe, il ajoute d'abord au résultat les tokens contenus dans le `failurePop` du nœud courant, puis suit le `failureLink` vers un autre nœud du trie. Le caractère qui n'a pas pu être consommé est conservé et sera recherché à nouveau à partir de ce nouveau nœud.

Cette recherche peut nécessiter un ou plusieurs déplacements successifs vers les nœuds indiqués par les `failureLink`. À chaque étape, les tokens associés au `failurePop` du nœud courant sont ajoutés au résultat, puis la recherche se poursuit à partir du nœud indiqué par son `failureLink`. Le même caractère est alors recherché à partir de ce nouveau nœud. Le processus continue jusqu'à ce qu'une transition correspondant au caractère courant soit trouvée ou qu'un nœud dont le `failureLink` est nil soit atteint. Lorsque la transition recherchée est trouvée, elle est suivie et le caractère est consommé. Le parcours reprend ensuite avec le caractère suivant de la chaîne.

Condition d'arrêt de `matchLoop` : in :

Le parcours peut se terminer de deux façons.

Si tous les caractères de la chaîne ont été parcourus, la méthode retourne les tokens collectés ainsi que les informations concernant le dernier nœud atteint et la position finale.

Si, pour un caractère donné, aucune transition correspondante ne peut être trouvée après avoir suivi les `failureLink`, le parcours s'arrête dès qu'un nœud dont le `failureLink` est nil est atteint. La méthode retourne alors les tokens collectés, le dernier nœud atteint et la position du caractère qui n'a pas pu être consommé.

Rôle de l'espace ajouté à la fin du mot

Lors de l'encodage d'un mot, `encodeWord` : ajoute un espace à la fin de celui-ci avant d'appeler `matchLoopFrom :in :`. Cet espace ne fait pas partie du vocabulaire et aucune arête du trie ne lui est donc associée.

Lorsque le parcours atteint cet espace, aucune transition directe ne peut être suivie. L'algorithme utilise alors les `failureLink` et les `failurePop` pour terminer le parcours.

Cette étape permet notamment de vérifier que tous les caractères du mot ont été couverts par les tokens produits. `encodeWord` : utilise ensuite les informations retournées par `matchLoopFrom :in :` pour déterminer si la tokenisation est complète. Si le mot n'a pas pu être entièrement couvert par les tokens du vocabulaire, le résultat est remplacé par '<UNK>'.

6.2.4.6 Encodage d'un texte

Contrairement à `encodeWord` :, qui traite un seul mot, cette méthode permet de tokeniser l'ensemble d'un texte en parcourant ses caractères de gauche à droite.

La méthode commence le parcours à la première position du texte. Elle utilise ensuite `matchLoopFrom :in :` pour rechercher les tokens correspondant à la portion du texte située à partir de la position courante.

Voici son implémentation :

```
encodeText: aText
|result text i tokens currentNode r prefixIndicatorNode result1|
result := OrderedCollection new .
text := aText , ' ' .
i := 1 .
r := trie rootNode findChildWithString: $# .

prefixIndicatorNode := r findChildWithString: $# .
[ i <= text size ]
whileTrue: [
result1 := self matchLoopFrom: i in: text .
tokens := result1 first .
currentNode := result1 second .
i := result1 third .

( ( self isWordBoundaryAt: i in: text ) not
or: [ ( { trie rootNode . prefixIndicatorNode . punctuationNode } asSet
includes: currentNode ) not ] )
ifTrue: [
tokens := OrderedCollection with: '<UNK>' ] .

(currentNode = prefixIndicatorNode
and: [tokens isEmpty])
ifTrue: [

(vocabulary includes: '##')
ifTrue: [
tokens := OrderedCollection with: '##'
]
ifFalse: [ tokens := OrderedCollection with: '<UNK>' ]

].

result addAll: tokens .

[ i <= text size
and: [ ( self isWordBoundaryAt: i in: text ) not ] ]
whileTrue: [i := i + 1 ] .
```



```

[ i <= text size
and: [ ( text at: i ) isSeparator ] ]
  whileTrue: [ i := i + 1 ] .
] .

^ result

```

Parcours du texte

La méthode ajoute d'abord un espace à la fin du texte. Cet espace permet de disposer d'un caractère qui ne fait pas partie du vocabulaire afin de terminer correctement le parcours de la dernière portion du texte.

Le parcours commence à la première position du texte. Tant qu'il reste des caractères à traiter, la méthode appelle `matchLoopFrom:in:` à partir de la position courante.

La méthode `matchLoopFrom:in:` retourne trois informations :

- les tokens produits pendant le parcours ;
- le dernier nœud atteint dans le trie ;
- la position à laquelle le parcours s'est arrêté.

Ces informations sont utilisées par `encodeText:` pour déterminer si la portion parcourue a été correctement tokenisée et pour savoir à quelle position poursuivre le parcours du texte.

Détermination d'une frontière entre les mots

Après le retour de `matchLoopFrom:in:`, la méthode vérifie si la position où le parcours s'est arrêté correspond à une frontière entre les mots.

Dans l'article original, une position est considérée comme une frontière lorsqu'elle correspond notamment :

- à un espace ou à une tabulation ;
- à une ponctuation ;
- à une position située après la fin du texte ;
- à un caractère qui suit directement une ponctuation.

Cette vérification permet de déterminer si le parcours s'est arrêté à une position qui correspond à une frontière entre deux mots. Si ce n'est pas le cas, cela signifie que `matchLoopFrom:in:` s'est arrêté au milieu d'une portion du texte. La tokenisation de cette portion est alors considérée comme incorrecte et le résultat est remplacé par '<UNK>'.

Vérification de la réussite de la tokenisation

La méthode vérifie ensuite deux conditions.

Premièrement, la position à laquelle `matchLoopFrom:in:` s'est arrêté doit correspondre à une frontière entre les mots.

Deuxièmement, le dernier nœud atteint doit être l'un des nœuds indiquant une terminaison valide du parcours : la racine du trie, le nœud représentant '###' ou le `punctuationNode`.

Si l'une de ces conditions n'est pas satisfaite, la portion de texte traitée ne peut pas être correctement tokenisée. Les tokens éventuellement obtenus sont alors remplacés par '<UNK>'.

Cette vérification est importante car `matchLoopFrom:in:` peut avoir reconnu une partie du texte sans avoir réussi à couvrir entièrement la portion considérée.

Cas particulier du token '##'

La méthode traite ensuite un cas particulier : lorsque le dernier nœud atteint est celui qui représente '##' et qu'aucun token n'a été produit. Dans cette situation, elle vérifie si '##' appartient au vocabulaire.

S'il appartient au vocabulaire, '##' est ajouté comme token. Sinon, le résultat est remplacé par '<UNK>'.

Ajout des tokens au résultat final

Les tokens obtenus pour la portion courante du texte sont ensuite ajoutés au résultat global. Le processus peut alors continuer avec la partie restante du texte.

Passage à la prochaine portion du texte

Après avoir ajouté les tokens obtenus, la méthode avance dans le texte.

La première boucle après l'ajout des tokens au résultat permet de dépasser les caractères qui ne constituent pas une frontière entre les mots et qui n'ont pas été consommés lors du parcours précédent.

La seconde boucle après l'ajout des tokens au résultat permet ensuite de dépasser les caractères séparateurs, notamment les espaces et les tabulations, afin de commencer le traitement à la prochaine partie pertinente du texte.

Ainsi, `encodeText:` ne recommence pas systématiquement le parcours à la position immédiatement suivante. Il détermine d'abord où commence la prochaine portion du texte qui doit être analysée.

Rôle des ponctuations

Lors de la construction du trie, les caractères de ponctuation sont ajoutés individuellement afin de pouvoir les reconnaître séparément lors de la tokenisation. Cette représentation permet de traiter chaque ponctuation comme une unité indépendante des autres sous-mots.

Un nœud spécial, appelé `punctuationNode`, est également utilisé pour les nœuds représentant les ponctuations. Le `failureLink` de chacun de ces nœuds pointe vers `punctuationNode`. Ce dernier ne possède pas d'enfant et son `failureLink` est nil.

Lorsqu'une ponctuation apparaît après une séquence de caractères ne contenant pas de ponctuations, `matchLoopFrom:in:` ne peut pas poursuivre le parcours à partir du nœud représentant cette séquence, car les ponctuations sont représentées séparément dans le trie. La ponctuation n'est donc pas consommée lors de cet appel. Le parcours s'arrête à sa position. `encodeText:` reprend ensuite le traitement à cette même position lors de l'itération suivante, ce qui permet de reconnaître la ponctuation comme une unité indépendante.

Lorsqu'une ponctuation est elle-même parcourue, le parcours atteint le nœud qui la représente. Ce nœud ne possède pas d'enfant. Si le caractère suivant est recherché à partir de ce nœud, aucune transition correspondante ne peut donc être suivie. L'algorithme suit alors son `failureLink`, qui conduit au `punctuationNode`. Comme ce dernier ne possède pas de transition et que son `failureLink` est nil, `matchLoopFrom:in:` s'arrête. La position retournée correspond

alors au caractère suivant la ponctuation, qui n’a pas encore été consommé. `encodeText` : pourra donc reprendre le traitement à cette position lors de l’itération suivante.

6.3 Benchmarking

Cette section présente le benchmark des différentes implémentations de WordPiece développées dans ce travail.

6.3.1 Entraînement

Chaque implementation a été exécutée 100 fois pour chaque taille de vocabulaire n afin de réduire l’impact des fluctuations liées au système d’exploitation et au garbage collector. Les temps d’exécution obtenus ont ensuite été moyennés (division par 100) afin d’obtenir des valeurs représentatives, reportées dans les tableaux et figures ci-dessous. Les expérimentations ont été réalisées sur deux types de données issues du système Pharo : les noms de classes et les noms de méthodes.

Les tableaux 6.1 et 6.2 présentent les temps d’exécution des différentes implémentations du WordPiece sur les noms de classes et les noms de méthodes.

n	WordPiece	WordPieceTrain
50	32.9 ms	17.26 ms
100	435.22 ms	34.91 ms
200	1 169.15 ms	68.68 ms
400	2 494.65 ms	128.17 ms
600	3 747.46 ms	230 ms
807	5 118.11 ms	302.13 ms

TABLE 6.1 – Benchmark des temps d’entraînement des implémentations du WordPiece sur les noms de classes de Pharo en fonction de la taille du vocabulaire n .

n	WordPiece	WordPieceTrain
50	36.74 ms	77.54 ms
100	895.89 ms	91.54 ms
200	3 946.76 ms	181.94 ms
400	9 660.52 ms	378.01 ms
600	14 951.26 ms	512.71 ms
800	18 569.91 ms	678.92 ms
1 000	24 888.52 ms	853.40 ms
1 500	34 325.84 ms	1 221.51 ms
2 000	45 848.83 ms	1 803.79 ms
2 500	52 866.96 ms	2 444.75 ms
3 000	62 230.11 ms	2 864.26 ms
3 519	71 949.98 ms	3 018 ms

TABLE 6.2 – Benchmark des temps d’entraînement des implémentations du WordPiece sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n .

Les figures 6.10 et 6.11 présentent les temps d'exécution des différentes implémentations du WordPiece sur les noms de classes et les noms de méthodes.

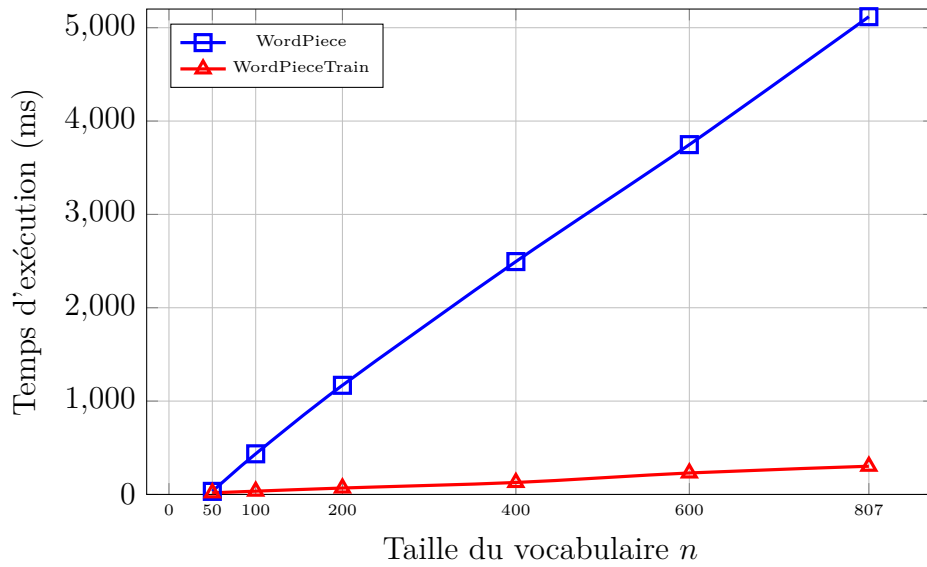


FIGURE 6.10 – Benchmark des temps d'entraînement des implémentations du WordPiece sur les noms de classes de Pharo en fonction de la taille du vocabulaire n .

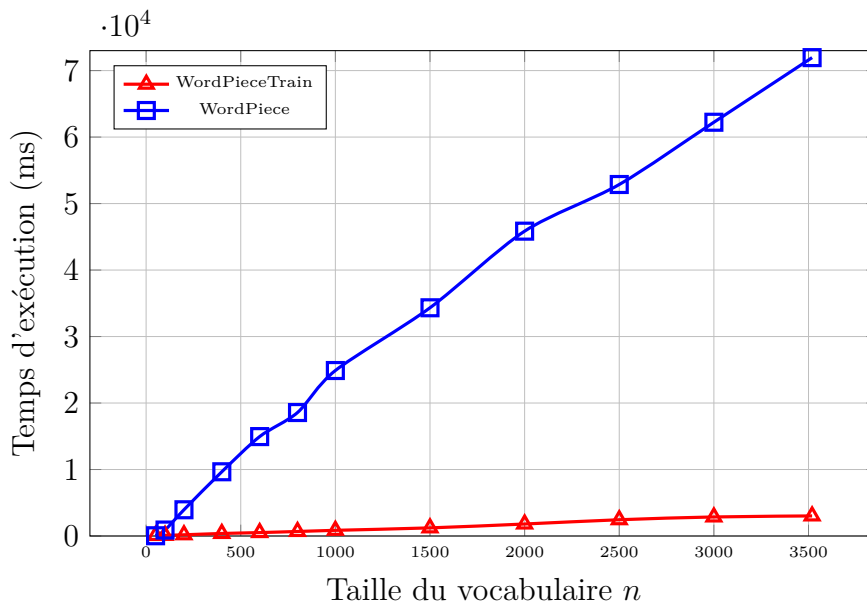


FIGURE 6.11 – Benchmark des temps d'entraînement des implémentations du WordPiece sur les noms de méthodes de Pharo en fonction de la taille du vocabulaire n .

On remarque que, pour une petite taille de vocabulaire, notamment $n = 50$, les temps d'exécution restent faibles pour les deux implémentations. Cependant, les résultats diffèrent selon le corpus considéré. Pour les noms de classes, la version optimisée nécessite 17,26 ms, contre 32,9 ms pour la version naïve. Elle est donc déjà plus rapide pour cette taille de vocabulaire. Pour les noms de méthodes, en revanche, la version optimisée nécessite 77,54 ms, contre 36,74 ms pour la version naïve. Dans ce cas, l'optimisation est donc moins performante pour cette petite taille de vocabulaire. Cette différence peut notamment s'expliquer par le coût des structures de

données et des opérations supplémentaires introduites par l’optimisation, dont le bénéfice peut être insuffisant lorsque la taille du vocabulaire reste faible.

Lorsque la taille du vocabulaire augmente, l’avantage de la version optimisée devient beaucoup plus important. Pour les noms de classes, à $n = 400$, la version optimisée est environ 19,5 fois plus rapide que la version naïve, avec respectivement 128,17 ms et 2 494,65 ms. À $n = 807$, elle est environ 17 fois plus rapide, avec 302,13 ms contre 5 118,11 ms. Pour les noms de méthodes, l’écart est encore plus important : à $n = 400$, la version optimisée est environ 25,6 fois plus rapide, avec 378,01 ms contre 9 660,52 ms, et à $n = 3,519$, elle est environ 23,8 fois plus rapide, avec 3 018 ms contre 71 949,98 ms. Ces résultats montrent que l’avantage de la version optimisée devient particulièrement marqué lorsque la taille du vocabulaire augmente. Pour les grandes tailles de vocabulaire considérées dans ces expériences, la réduction du temps d’exécution par rapport à la version naïve est donc très importante.

On observe également que les temps d’exécution obtenus sur les noms de méthodes sont globalement supérieurs à ceux obtenus sur les noms de classes. Cette différence est notamment liée à la taille du corpus utilisé dans les deux expériences : le corpus constitué des noms de méthodes contient davantage de données à traiter. Il nécessite donc davantage d’opérations pendant l’entraînement. La taille du corpus constitue ainsi un facteur susceptible d’influencer le temps d’exécution, en plus de la taille du vocabulaire.

6.3.2 Encodage

D’après les tableaux 6.3 et 6.4, ainsi que les figures 6.12 et 6.13, la taille du vocabulaire semble avoir une influence relativement faible sur le temps d’exécution des deux implémentations de WordPiece, optimisée et naïve. En effet, pour une taille de corpus donnée, les temps d’exécution restent globalement du même ordre de grandeur lorsque la taille du vocabulaire varie de 100 à 2000 tokens. Les variations observées d’une taille de vocabulaire à une autre peuvent notamment être liées aux fluctuations inhérentes aux mesures de temps d’exécution.

En revanche, la taille du corpus a une influence beaucoup plus importante sur le temps d’encodage. Pour les deux implémentations, le temps d’exécution augmente lorsque le nombre de méthodes à encoder augmente. Par exemple, pour un vocabulaire de 100 tokens, le temps de `WordPieceEncoder` passe de 3,71 ms pour un corpus de 100 méthodes à 19,94 ms pour 500 méthodes, puis à 43,79 ms pour 1083 méthodes. De manière similaire, le temps de `WordPiece` passe de 8,56 ms à 49,84 ms, puis à 105,89 ms pour les mêmes tailles de corpus.

Les heatmaps permettent ainsi de visualiser que les variations de temps sont principalement associées à la taille du corpus plutôt qu’à la taille du vocabulaire dans les configurations étudiées. Cette observation suggère que, pour l’encodage considéré, le nombre de séquences à traiter constitue un facteur plus déterminant que la taille du vocabulaire.

Taille du corpus	Taille du vocabulaire				
	100	500	1000	1500	2000
100	3.71 ms	3.46 ms	3.82 ms	3.59 ms	3.64 ms
500	19.94 ms	17.64 ms	18.56 ms	18.30 ms	18.59 ms
1083	43.79 ms	45.05 ms	42.57 ms	40.66 ms	41.78 ms

TABLE 6.3 – Temps d’exécution de WordPieceEncoder en fonction de la taille du corpus et de la taille du vocabulaire.

Taille du corpus	Taille du vocabulaire				
	100	500	1000	1500	2000
100	8.56 ms	8.32 ms	7.45 ms	8.23 ms	7.38 ms
500	49.84 ms	46.15 ms	44.80 ms	46.34 ms	42.35 ms
1083	105.89 ms	99.33 ms	91.36 ms	106.40 ms	89.33 ms

TABLE 6.4 – Temps d’exécution de WordPiece en fonction de la taille du corpus et de la taille du vocabulaire.

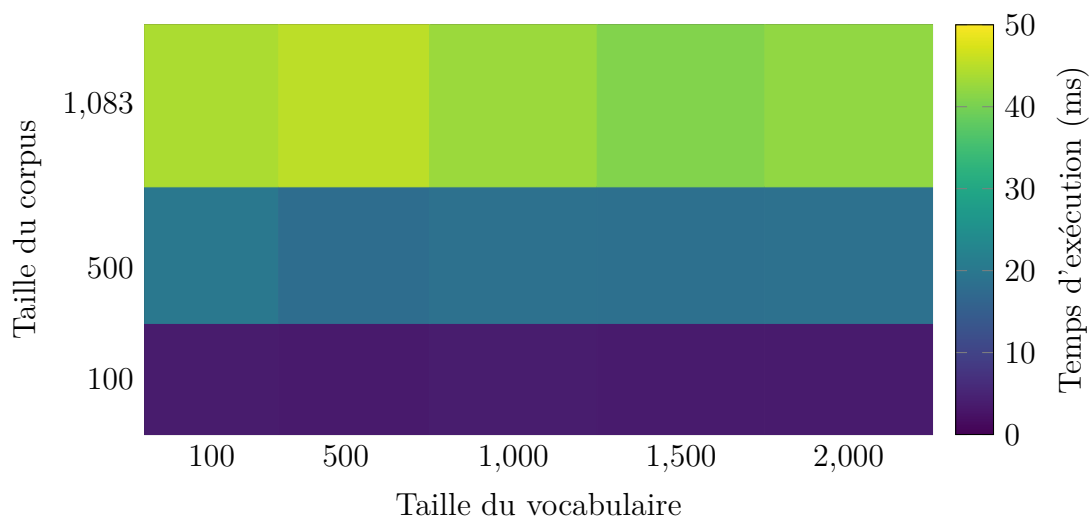


FIGURE 6.12 – Heatmap des temps d’encodage de WordPieceEncoder.

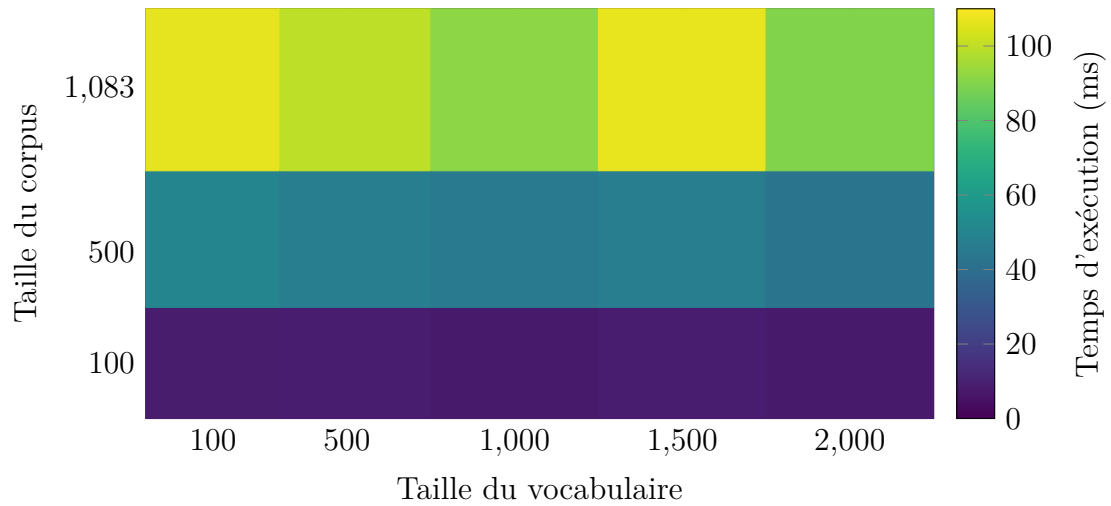


FIGURE 6.13 – Heatmap des temps d’encodage de WordPiece.

D’après le tableau 6.5 et la figure 6.14, la version optimisée `WordPieceEncoder` est systématiquement plus rapide que la version naïve `WordPiece` pour toutes les configurations considérées. Le facteur d’accélération varie entre 1,95 et 2,62 selon la taille du corpus et la taille du vocabulaire. Ainsi, dans les configurations étudiées, `WordPieceEncoder` est généralement environ deux fois plus rapide que `WordPiece`.

Le facteur d’accélération ne suit pas une évolution monotone avec la taille du vocabulaire ou celle du corpus.

Taille du corpus	Taille du vocabulaire				
	100	500	1000	1500	2000
100	2,31	2,41	1,95	2,29	2,03
500	2,50	2,62	2,41	2,53	2,28
1083	2,42	2,20	2,15	2,62	2,14

TABLE 6.5 – Facteur d’accélération de `WordPieceEncoder` par rapport à `WordPiece`.

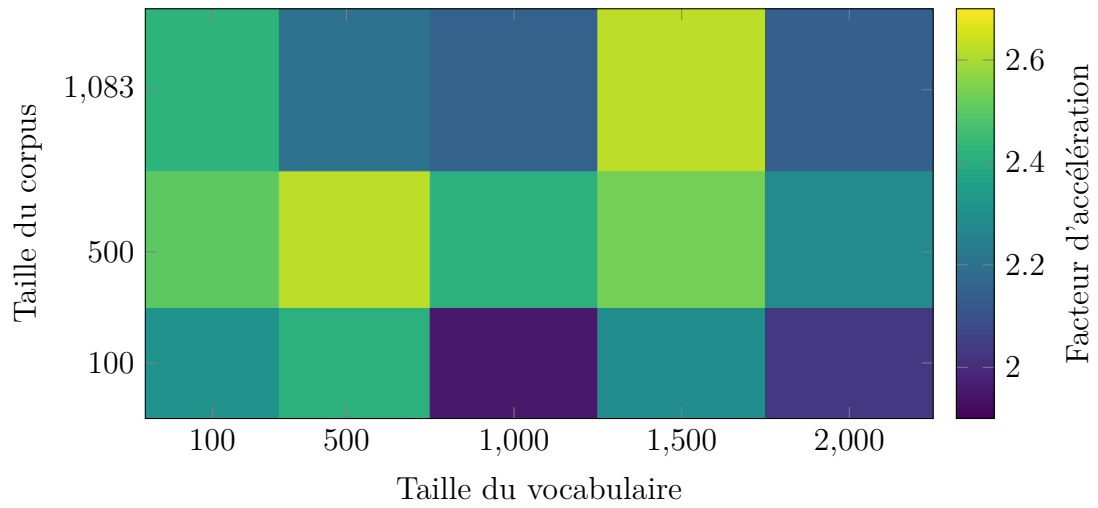


FIGURE 6.14 – Facteur d’accélération de `WordPieceEncoder` par rapport à `WordPiece` en fonction de la taille du corpus et de la taille du vocabulaire.

6.4 Comparaison théorique

Chapitre 7

Implémentation

7.1 Implémentation de BPE

7.2 Implémentation de WordPiece

7.3 Implémentation de Unigram

Chapitre 8

Résultats et Analyse

8.1 Expérimentations

8.2 Comparaison des performances

Chapitre 9

Conclusion et Perspectives

Annexe A

Annexes

Bibliographie

- [1] A. Kathuria, J. Chaudhary *Emerging Paradigms and Challenges in Multidisciplinary Education and Research* , Iterative International Publisher, 2024, pp. 100-101.
- [2] Choudhary A, Pamidimokkala S, R. K and R. M. B *Impact of Natural Language Processing models on diagnosis and decision-making in healthcare, business, education, and sports : a review*, Front. Artif. Intell, 8 :1706369, 5 february 2026, doi : 10.3389/frai.2025.1706369.
- [3] A. Kathuria, J. Chaudhary *Emerging Paradigms and Challenges in Multidisciplinary Education and Research* , Iterative International Publisher, 2024, pp. 103.
- [4] Daniel Jurafsky, James H. Martin *Speech and Language Processing*, 3rd edition, draft version, 2025, pp. 158
- [5] Ian Goodfellow, Yoshua Bengio, Aaron Courville *Deep Learning*, MIT Press, 2016, pp. 373